Chapter 7
Public-Key Encryption

Stefan Dziembowski
www.crypto.edu.pl/Dziembowski

University of Warsaw

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. IND-CCA security
9. Practical considerations
10. Theoretical overview

“Handbook RSA” encryption

Take Z_N^* (where $N = pq$ and p, q are **two distinct odd primes**), defined as follows:

$e \leftarrow Z_{\varphi(N)}^*$

$d = e^{-1} \bmod \varphi(N)$

Let $pk = (N, e)$ and $sk = (N, d)$

Handbook RSA encryption scheme:

messages and **ciphertexts**: Z_N

- $\mathrm{Enc}_{N,e}(m) = m^e \bmod N$
- $\mathrm{Dec}_{N,d}(c) = c^d \bmod N$

Is it secure?

Issues with the "handbook RSA"

1. It is **deterministic**.
2. It has some "**algebraic properties**".
3. It is defined over Z_N^* and not over Z_N.

this is not really a problem (exercise)

Algebraic properties of RSA

1. **RSA** is homomorphic:

$$\mathbf{RSA}_{e,N}(m_0 \cdot m_1) = (m_0 \cdot m_1)^e$$

$$= m_0^e \cdot m_1^e$$

$$= \mathbf{RSA}_{e,N}(m_0) \cdot \mathbf{RSA}_{e,N}(m_1)$$

why is it bad?

By checking if $c = c_0 \cdot c_1$ the adversary can check if the messages m, m_0, m_1 corresponding to c, c_0, c_1 satisfy:

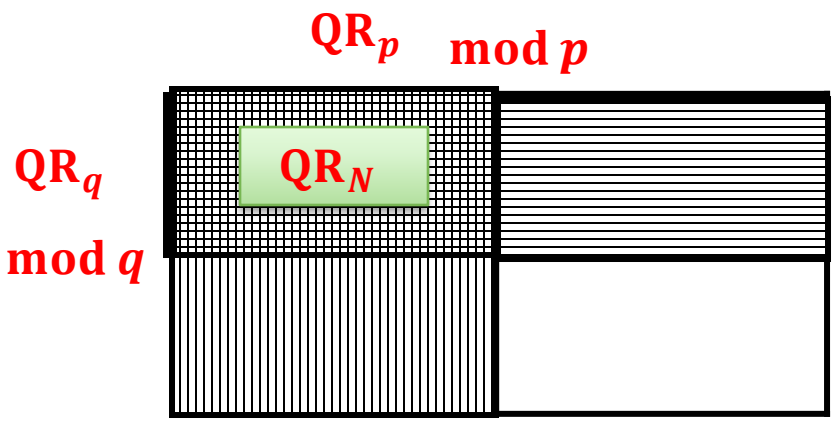
$$m = m_0 \cdot m_1$$

2. The **Jacobi symbol** leaks.

Jacobi Symbol (from the Chapter 6)

for any prime p define $J_p(x) := \begin{cases} +1 & \text{if } x \in \mathrm{QR}_p \\ -1 & \text{otherwise} \end{cases}$

for $N = pq$ define $J_N(x) := J_p(x) \cdot J_q(x)$



$J_N(x) :=$

+1	-1
-1	+1

It is a subgroup of Z_N^*

$Z_N^+ := \{x \in Z_N^* : J_N(x) = +1\}$

Jacobi symbol can be computed efficiently!
(even in p and q are unknown)

Fact: the **RSA** function "preserves" the **Jacobi symbol**

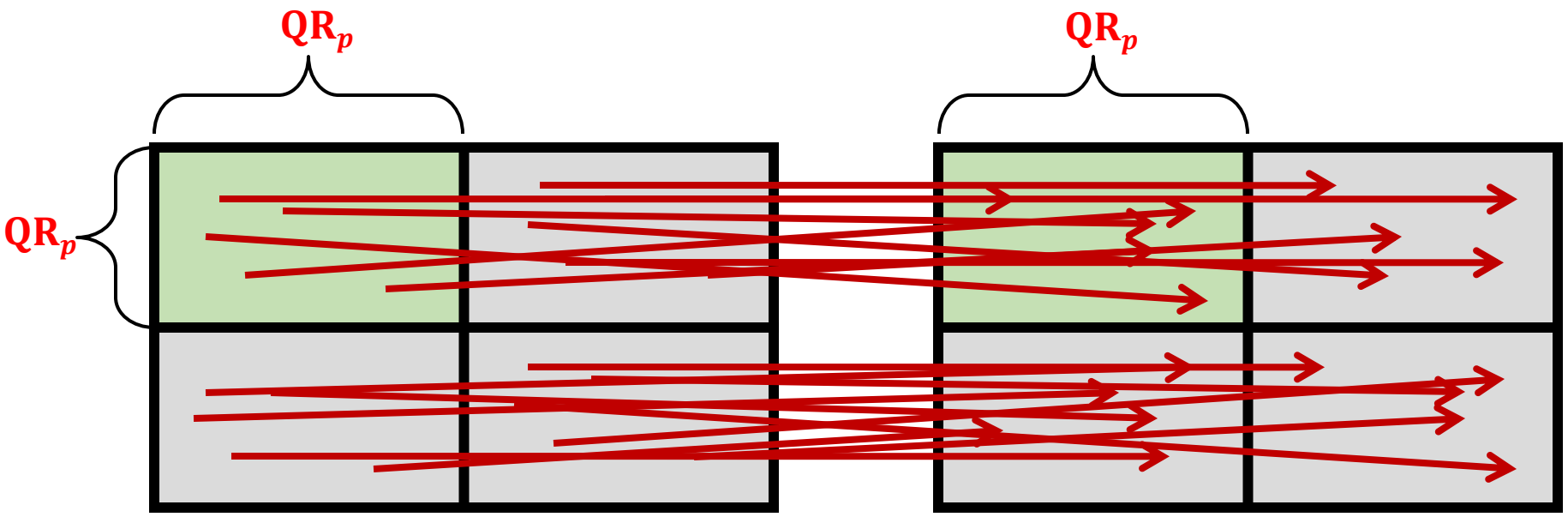
$N = pq$ - **RSA** modulus

e is such that $e \perp \varphi(N)$

$$J_N(x) = J_N(x^e \bmod N)$$

Actually, something even stronger holds:

$\mathbf{RSA}_{N,e}$ is a permutation on each "quarter" of Z_N^*

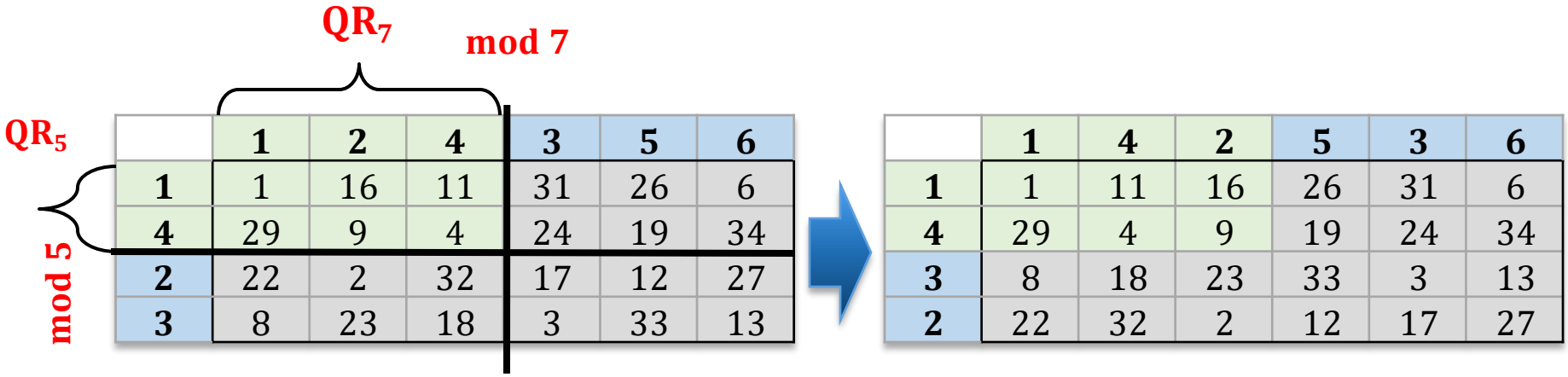


In other words:

- $m \bmod p \in \mathrm{QR}_p$ **iff** $m^e \bmod p \in \mathrm{QR}_p$
- $m \bmod q \in \mathrm{QR}_q$ **iff** $m^e \bmod q \in \mathrm{QR}_q$

Example Z_{35}^*

We calculate $\mathbf{RSA}_{23,35}(m) = m^{23} \bmod 35$



	1	2	4	3	5	6
1	1	16	11	31	26	6
4	29	9	4	24	19	34
2	22	2	32	17	12	27
3	8	23	18	3	33	13

	1	4	2	5	3	6
1	1	11	16	26	31	6
4	29	4	9	19	24	34
3	8	18	23	33	3	13
2	22	32	2	12	17	27

How to prove it?

By the **CRT** and by the fact that p and q are symmetric it is enough to show that

m is a $\mathbf{QR}_p$

iff

m^e is a $\mathbf{QR}_p$

Fact

For an odd e:

$m^e \bmod p$ is a QR_p

iff

$m \bmod p$ is a QR_p

Proof:

Let g be the generator of Z_p^*. Let y be such that $m = g^y$.

Recall that x is a QR_p iff x is an even power of g

We have that

$m^e \bmod p$ is a QR_p

iff

$(g^y)^e \bmod p$ is an **even** power of g

iff

$g^{ye \bmod (p-1)}$ is an **even** power of g

iff

$g^y \bmod p$ is an **even** power of g.

remember that p and e are **odd**

$= m \bmod p$

QED

Conclusion

The Jacobi symbol "leaks", i.e.:

from $\boldsymbol{c}$

one can compute $\boldsymbol{J_N(\mathrm{Dec}_{N,d}(c))}$

(without knowing the factorization of $\boldsymbol{N}$)

Is it a big problem?

Depends on the application...

Plan for the next slides

1. Provide a formal security definition of public key encryption.

2. Modify **RSA** so that it is secure according to this definition.

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. IND-CCA security
9. Practical considerations
10. Theoretical overview

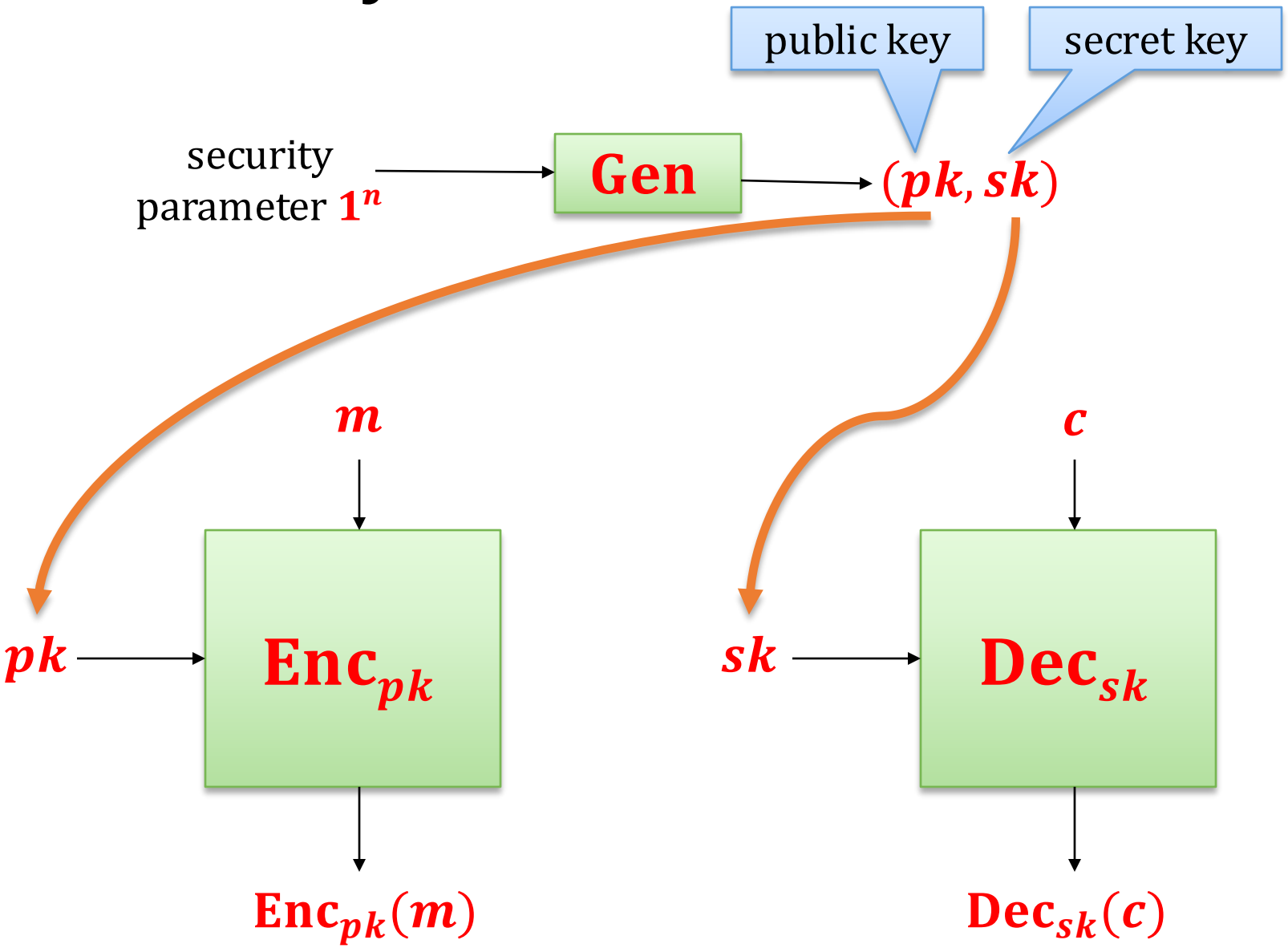
A mathematical view

A **public-key encryption (PKE)** scheme is a triple **(Gen, Enc, Dec)** of poly-time algorithms, where

- **Gen** is a **key-generation** randomized algorithm that takes as input a security parameter $\mathbf{1^n}$ and outputs a key pair $(\boldsymbol{pk}, \boldsymbol{sk}) \in (\{0,1\}^*)^2$.
- **Enc** is an **encryption** algorithm that takes as input the **public key** $\boldsymbol{pk}$ and a **message** $\boldsymbol{m}$ (from some set that **may depend** on $\boldsymbol{pk}$), and outputs a **ciphertext** $\boldsymbol{c}$,
- **Dec** is a **decryption** algorithm that takes as input the **private key** $\boldsymbol{sk}$ and the **ciphertext** $\boldsymbol{c}$, and outputs a **message** $\boldsymbol{m'} \in \{0,1\}^* \cup \{\perp\}$.

We will sometimes write $\mathbf{Enc}_{pk}(m)$ and $\mathbf{Dec}_{sk}(c)$ instead of $\mathbf{Enc}(pk, m)$ and $\mathbf{Dec}(sk, c)$.

Pictorially



Correctness

We will require that it always holds that

$$P(\mathbf{Dec}_{sk}(\mathbf{Enc}_{pk}(m)) \neq m) \text{ is negligible in } n$$

assuming that:

- $(pk, sk) \leftarrow \mathbf{Gen}(1^n)$
- and m is a "legal" plaintext for pk.

The security definition

Remember the symmetric-key case?

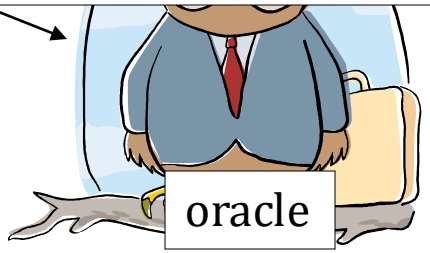
We considered a **chosen-plaintext attack**.

How would it look in the case of the **public-key encryption**?

IND-CPA in the **symmetric** settings

security parameter
1^n

1. chooses random k
2. chooses random $b \leftarrow \{0,1\}$



oracle

chooses m'_1

m'_1

$c_1 = \mathrm{Enc}(k, m'_1)$

$\vdots$

chooses m'_t

m'_t

$c_t = \mathrm{Enc}(k, m'_t)$

challenge phase:

chooses m_0, m_1

m_0, m_1

$c = \mathrm{Enc}(k, m_b)$

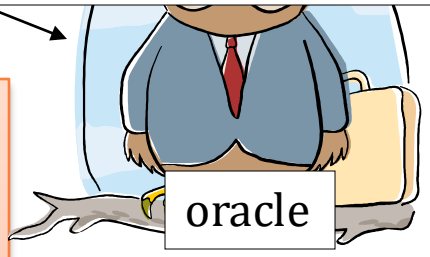
the interaction continues . . .

has to guess b

IND-CPA in the asymmetric settings

security parameter
1^n

1. generates $(pk, sk) \leftarrow Gen(1^n)$
2. chooses random $b \leftarrow \{0,1\}$



pk

chooses m'_1

m'_1

$c_1 = Enc(pk, m'_1)$

chooses m'_t

m'_t

$c_t = Enc(pk, m'_t)$

challenge phase:

chooses m_0, m_1

m_0, m_1

$c = Enc(pk, m_b)$

the interaction continues . . .

has to guess b

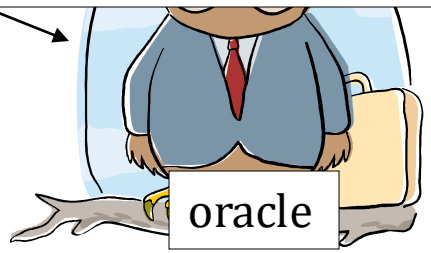
This is not needed.
Why?
Because if **Eve** knows pk she can compute all these ciphertexts herself!

The game after simplifications

security parameter
1^n

1. generates $(pk, sk) \leftarrow \mathbf{Gen}(1^n)$
2. chooses random $b \leftarrow \{0,1\}$

pk



oracle

challenge phase:

chooses m_0, m_1

m_0, m_1

$c = \mathbf{Enc}(pk, m_b)$

has to guess b

IND-CPA security

Alternative name: IND-CPA-secure

Security definition:

We say that **(Gen, Enc, Dec) has indistinguishable encryptions under a chosen-plaintext attack (IND-CPA)** if any

randomized polynomial time adversary

guesses b correctly

with probability at most $1/2 + \varepsilon(n)$, where ε is negligible.

Is the "**handbook RSA**" IND-CPA-secure?

$N = pq$, such that p and q are random primes, and $|p| = |q|$
e – random such that $e \perp (p-1)(q-1)$
d – random such that $ed = 1 \pmod{(p-1)(q-1)}$
$pk := (N, e)$ $sk := (N, d)$

$$\mathrm{Enc}_{pk}(m) = m^e \bmod N.$$
$$\mathrm{Dec}_{sk}(c) = c^d \bmod N.$$

Not IND-CPA-secure!

In fact: **no deterministic encryption scheme is secure.**

How can the adversary win the game?

1. he chooses any m_0, m_1,
2. computes $c_0 = \mathrm{Enc}_{pk}(m_0)$ himself
3. compares the result.

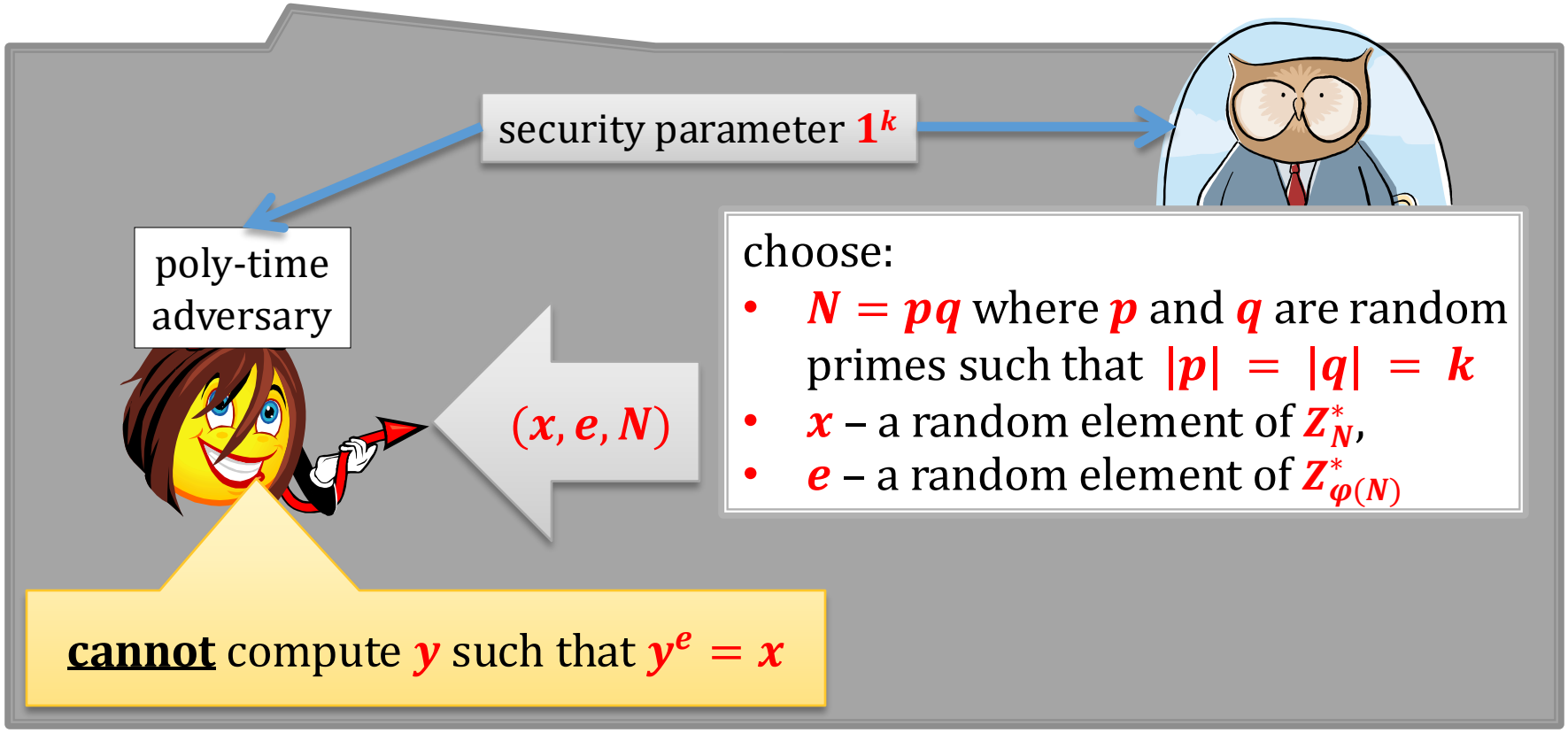
Moral: encryption has to be randomized.

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
    1. theoretical construction
    2. practical construction
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. CCA security
9. Practical considerations
10. Theoretical overview

IND-CPA-secure encryption from the **RSA** assumption

We now show how to construct a provably secure encryption scheme whose security is based on the **RSA assumption**.



RSA hardcore bit

Question: does **RSA** have a bit that is for sure well-hidden?

Answer: if **RSA assumption** doesn't hold, then: **no**.

But what if it holds?

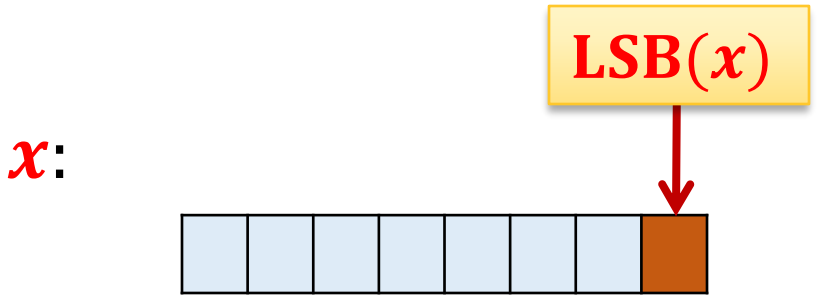
Answer: **yes** – the **least significant bit of the argument is hard to compute**.

Notation

For an integer x we will write

$$\mathbf{LSB}(x)$$

to denote the least significant bit of x.



In other words: $\mathbf{LSB}(x) = x \bmod 2$

Outline of the construction

1. We prove that the **least significant bit** is a **hard to compute** for **RSA**.

2. We show how to **“encrypt using this bit”**

Fact (informally)

LSB is the “hardest bit to compute” in **RSA**.

(it is called a “hard-core bit”).

More precisely:

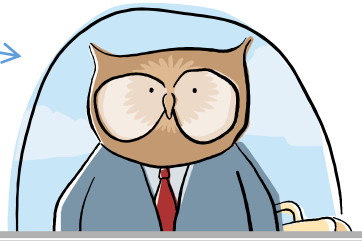
If you can compute **LSB** then you can invert **RSA**.

Note:

In some sense it is a “dual” predicate to Jacobi symbol...

"LSB game":

security parameter 1^k



choose:

- $N = pq$ where p and q are random primes such that $|p| = |q| = k$
- x – a random element of Z_N^*,
- e – a random element of $Z_{\varphi(N)}^*$

(x, e, N)

outputs b

$= x^d \bmod N$

The adversary **wins** if

b is the **least significant bit** of $y = \mathrm{Enc}_{e,N}^{-1}(x)$

Theorem

Suppose the **RSA assumption** holds.
Then every poly-time adversary wins the **LSB game** with a probability at most

$$\mathbf{0.5} + \boldsymbol{\varepsilon}(\boldsymbol{k})$$

where $\boldsymbol{\varepsilon}$ is negligible.

W. Alexi, B. Chor, O. Goldreich, and C.P. Schnorr
On the hardness of the least-signficant bits of the RSA and Rabin functions, 1984

In other words:

The least significant bit is a **hard-core bit for RSA.**

Proof strategy

Suppose we are given a poly-time adversary

that wins the **LSB game**.

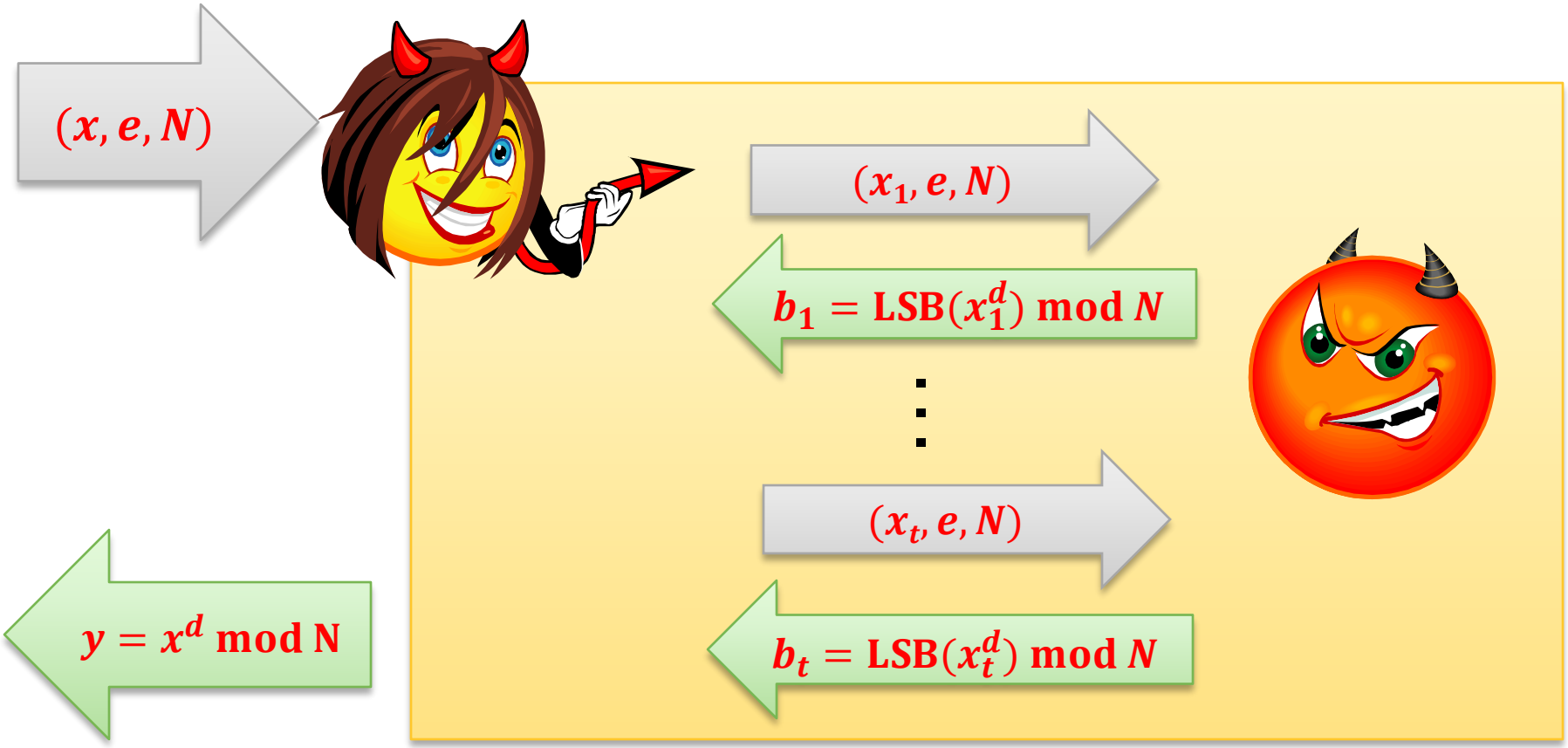
For simplicity suppose that this happens with probability $\mathbf{1}$

(not: $\mathbf{0.5} + \boldsymbol{\varepsilon(k)}$)

We construct a poly-time adversary

that breaks the **RSA assumption**.

Outline of the construction



Observation

Adversary  that can compute

LSB of x^d **mod** N from x

can also be used to compute (for any $c \in Z_N^*$)

LSB of $c \cdot x^d$ **mod** N from x.

How?

$(c^e \cdot x, e, N)$

outputs

$b' = \mathbf{LSB}((c^e \cdot x)^d)$

$= \mathbf{LSB}\,(c^{ed} \cdot x^d)$

$= \mathbf{LSB}\,(c \cdot x^d)$

The method

Let $y := x^d \bmod N$

The adversary  will use  to

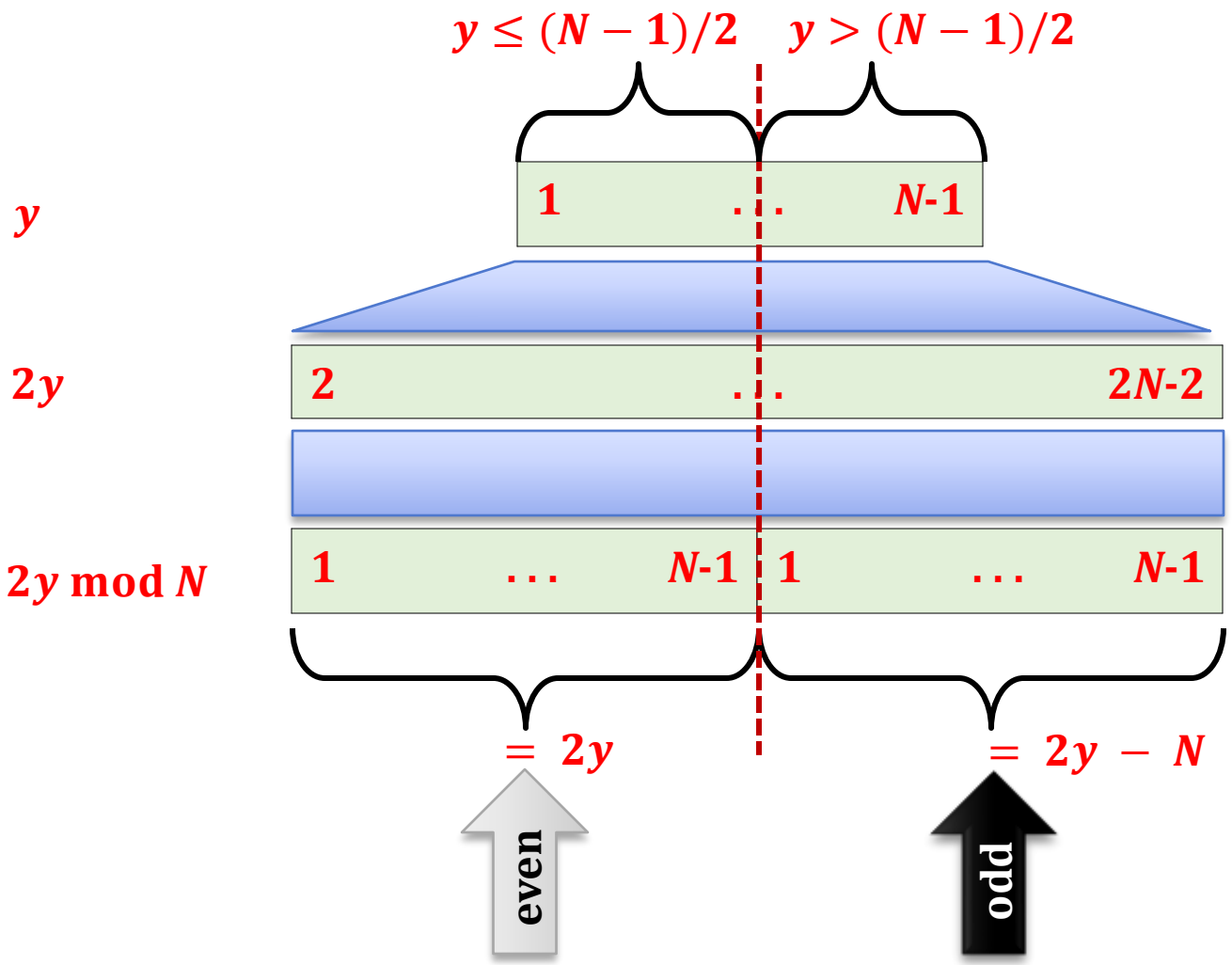
compute:

- $\mathbf{LSB}(2y)$
- $\mathbf{LSB}(4y)$
- $\mathbf{LSB}(8y)$

⋮

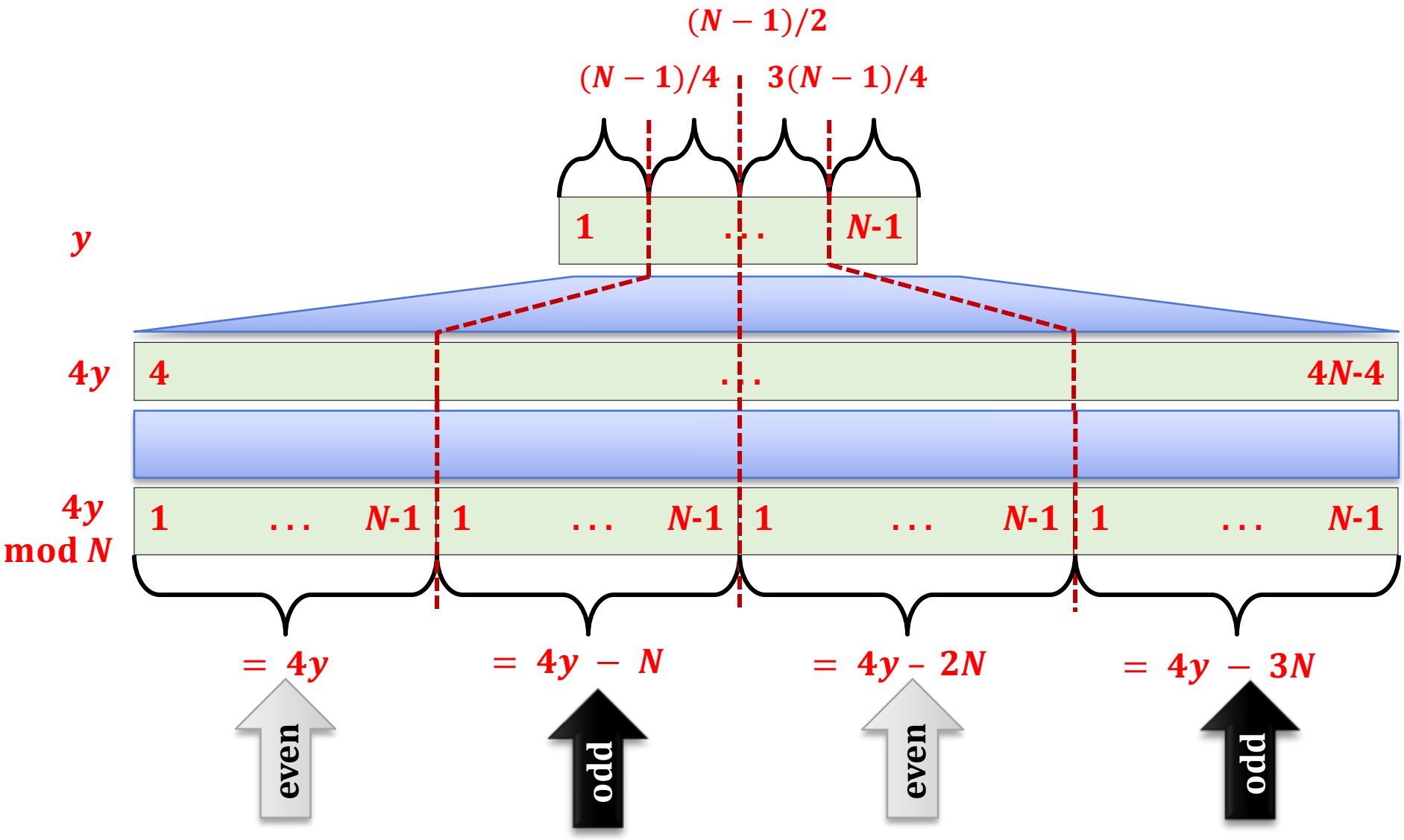
Why is it useful?

Observation

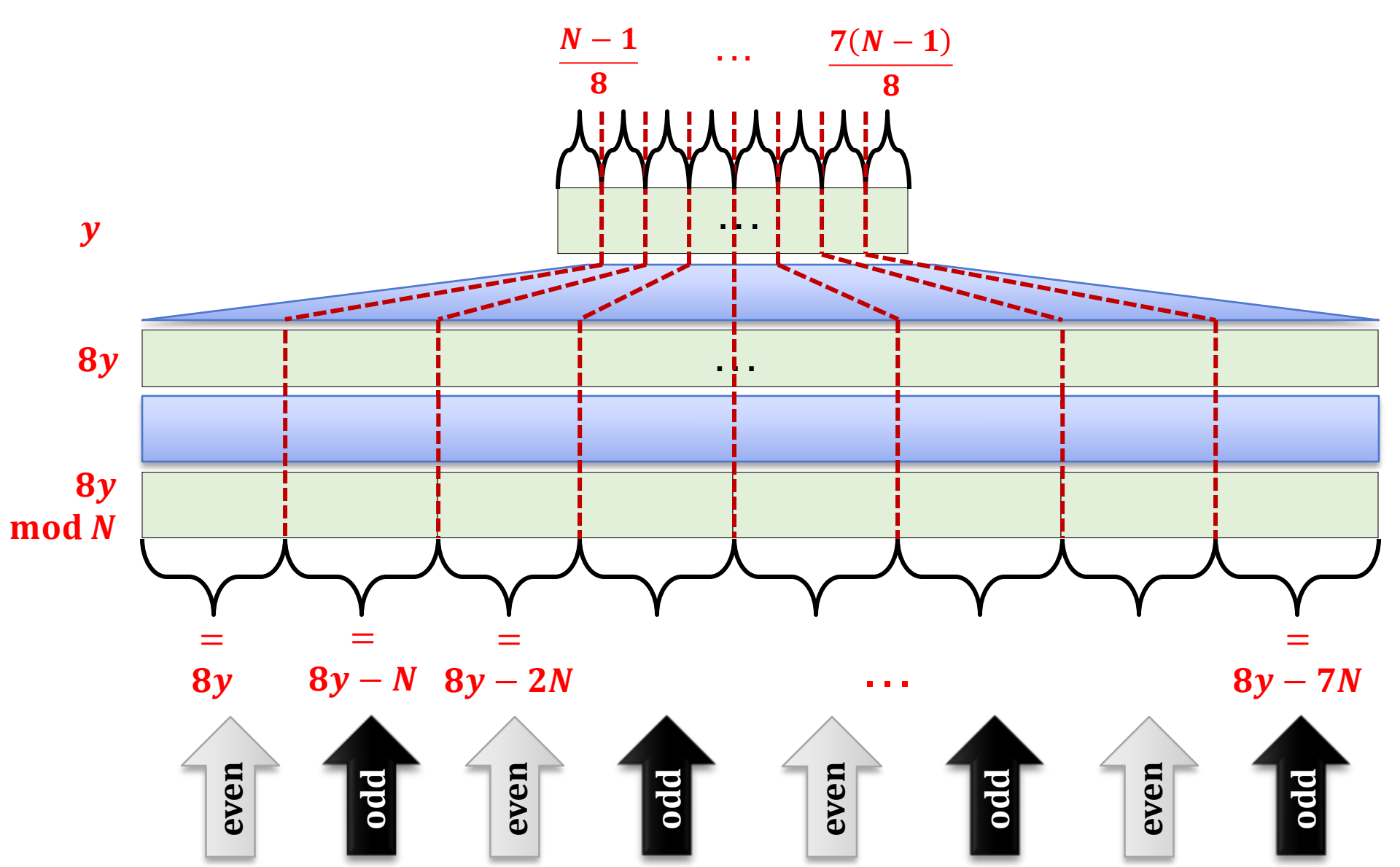


Remember:
$N = pq$ is odd

Moral: $y \in [1, \ldots, (N-1)/2]$ iff $2y \bmod N$ is even



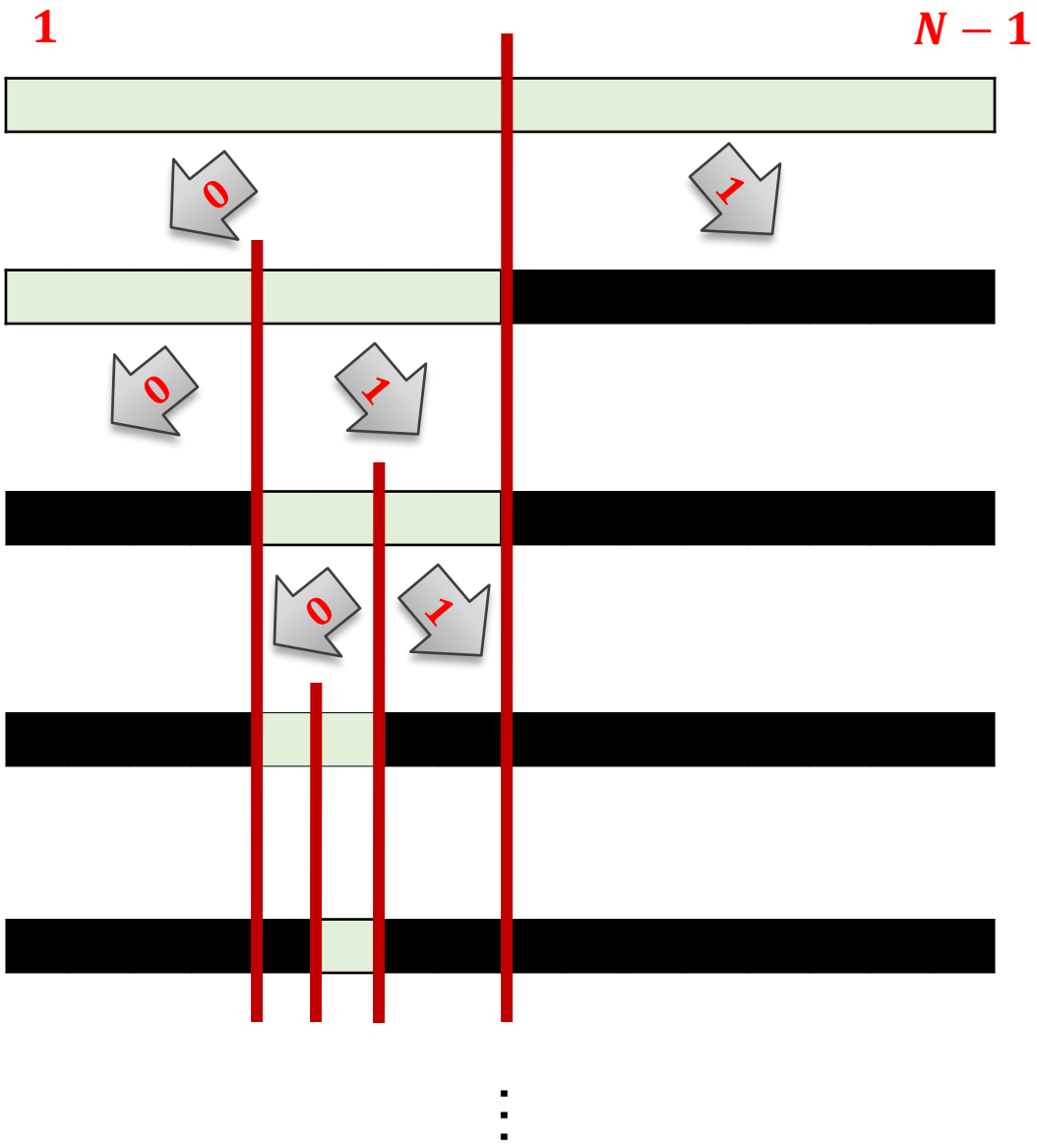
Moral: $y \in \left[1, \ldots, \frac{N-1}{4}\right] \cup \left[\frac{N}{2}+1, \ldots, \frac{3(N-1)}{4}\right]$ iff $4y \bmod N$ is even



Moral: $y \in \left[1, \dots, \frac{N-1}{8}\right] \cup \left[\frac{2N}{8} + 1, \dots, \frac{3(N-1)}{8}\right]$
$\cup \left[\frac{4N}{8} + 1, \dots, \frac{5(N-1)}{8}\right] \cup \left[\frac{6N}{8} + 1, \dots, \frac{7(N-1)}{8}\right]$

iff $8y \bmod N$ is even

So we can use bisection



calculate
$\mathbf{LSB}(2y)$

calculate
$\mathbf{LSB}(4 \cdot y)$

calculate
$\mathbf{LSB}(8 \cdot y)$

calculate
$\mathbf{LSB}(16 \cdot y)$

⋮

QED

Why is it interesting?

We can encrypt **one bit messages** as follows:

(N, e) – **public key**
(N, d) – **private key**

$$\mathbf{Enc}_{e,N}(b) = (\mathbf{LSB}(y) \oplus b, y^e)$$

(where $y \leftarrow Z_N^*$)

$$\mathbf{Dec}_{d,N}(c, x) = \mathbf{LSB}(x^d) \oplus c$$

This is secure **under the RSA assumption**

Lemma

Assume that the **RSA assumption holds**. Then the encryption scheme from the previous slide is **IND-CPA-secure**.

Proof: exercise

How to extend it to longer messages?

Encrypt **bit-by-bit**:

$$\mathbf{Enc}_{e,N}(m_1, \ldots, m_k) = \\ ((\mathbf{LSB}(y_1) \oplus m_1, \ldots, \mathbf{LSB}(y_k) \oplus m_k), (y_1^e, \ldots, y_k^e))$$

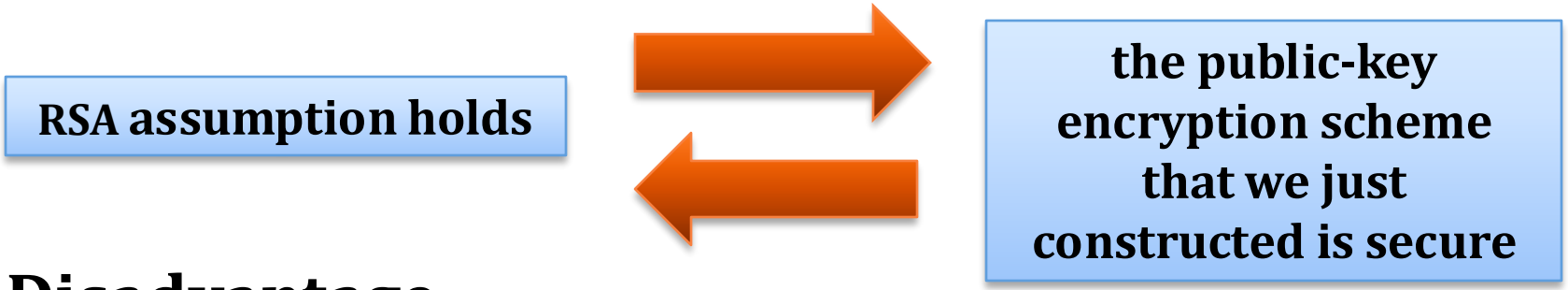
where $y_1, \ldots, y_k \leftarrow Z_N^*$

$$\mathbf{Dec}_{d,N}((c_1, \ldots, c_k), (x_1, \ldots, x_k)) = \\ \left(\mathbf{LSB}(x_1^d) \oplus c_1, \ldots, \mathbf{LSB}(x_k^d) \oplus c_k\right)$$

Conclusion

Advantage:

Security of this scheme is implied by the RSA assumption.



Disadvantage:

The ciphertext is much longer than the plaintext.

It is a rather theoretical construction!

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
    1. theoretical construction
    2. practical construction
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. CCA security
9. Practical considerations
10. Theoretical overview

Encoding (also called: “padding”)

Before encrypting a message we usually **encode it** (adding some randomness).

This has the following advantages:

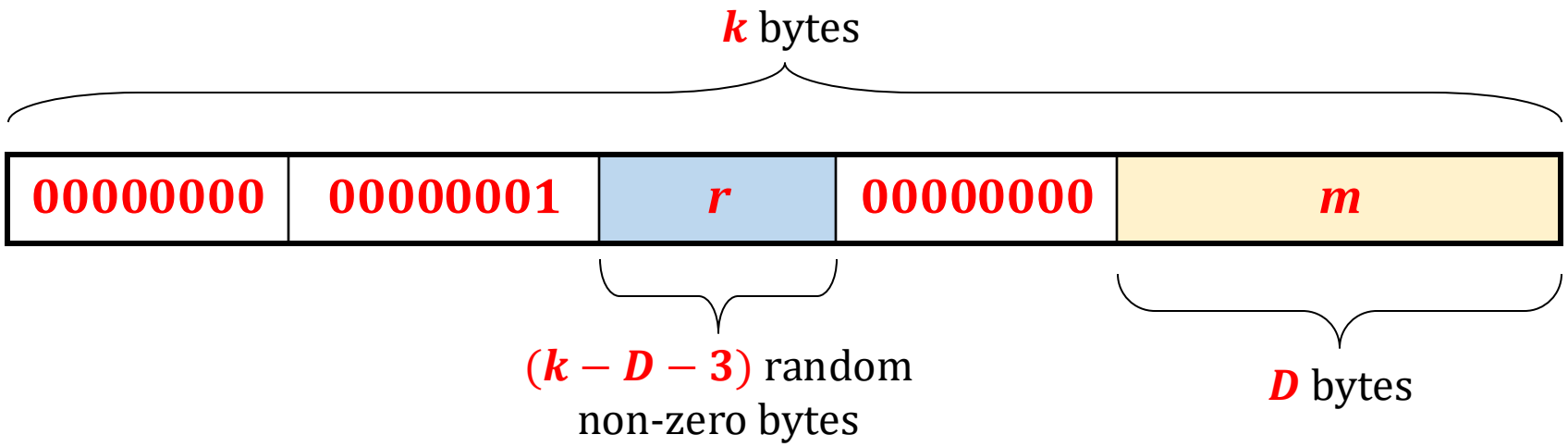
- it makes the encryption **non-deterministic**
- it **breaks the “algebraic properties”** of encryption.

How is it done in real-life?

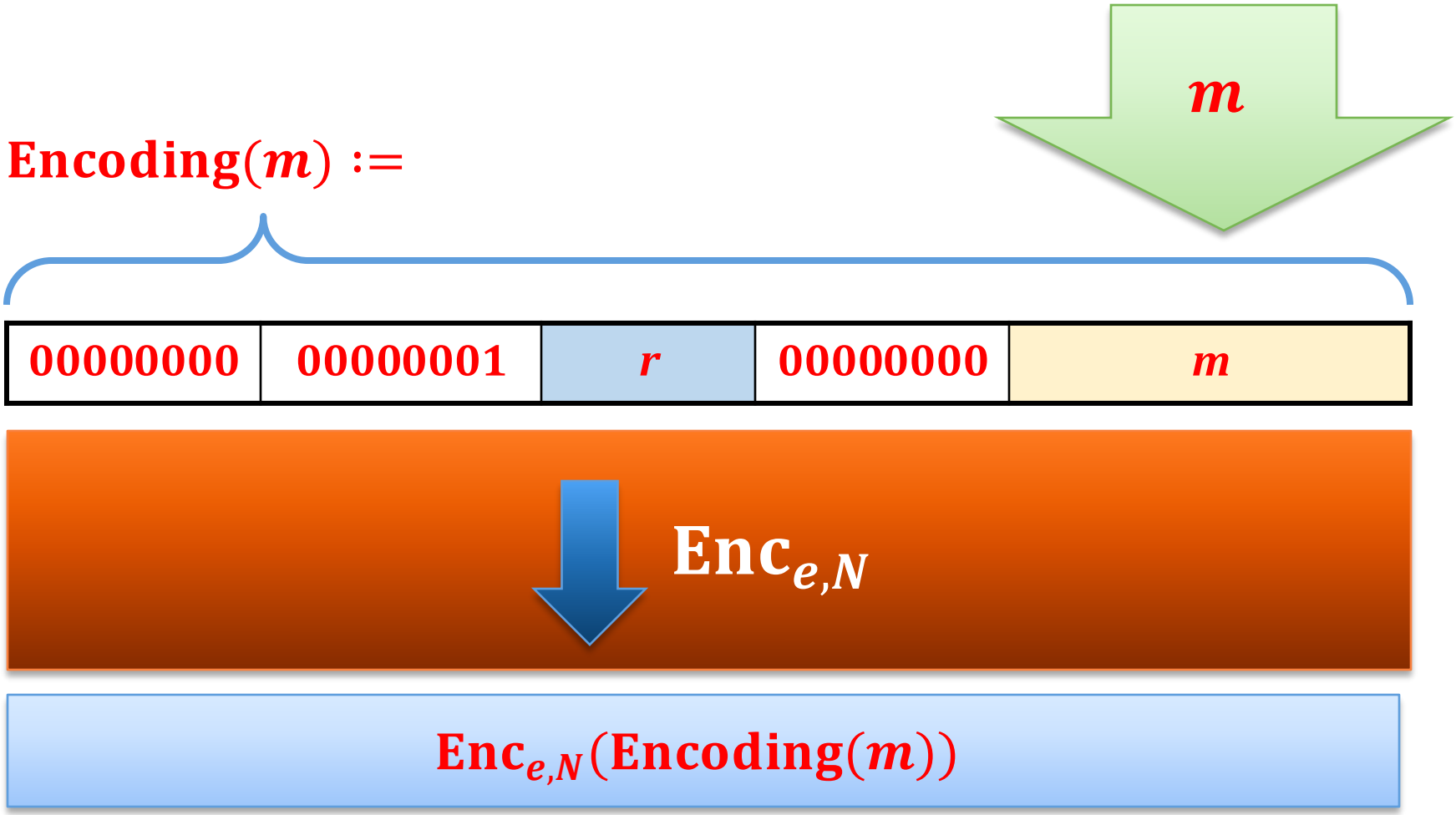
PKCS #1: RSA Encryption Standard Version 1.5:

public-key: (N, e)
k := length on N in bytes.
D := length of the plaintext
requirement: $D \leq k - 11$.

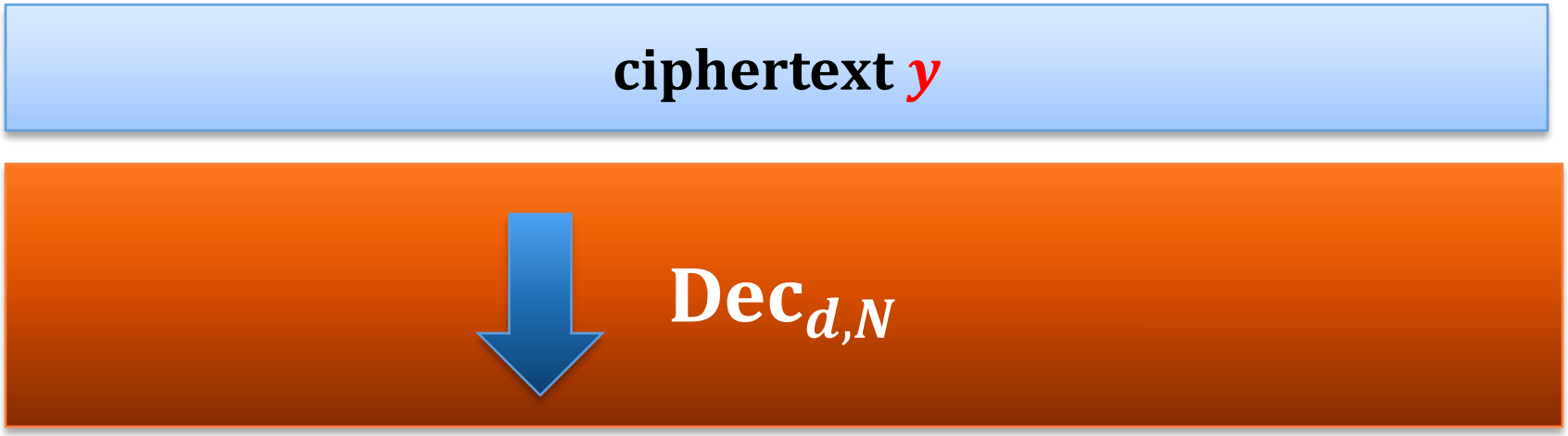
$\mathbf{Enc}((N, e), m) := x^e \bmod N$, where x is equal to:



How to encrypt?



How to decrypt?



check if the format agrees....

00000000	00000001	r	00000000	m

if not then output $\perp$, otherwise

output m

Example

If the adversary can calculate the Jacobi symbol of

most probably it doesn't help him in learning any information about m...

Security of the **PKCS #1: RSA Encryption Standard Version 1.5** – security

It is **believed** to be **IND-CPA-secure**.

(as we will later learn: it's not "**CCA-secure**")

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. CCA security
9. Practical considerations
10. Theoretical overview

The situation

factoring RSA moduli is hard

RSA assumption holds

public-key encryption exists

<u>question</u>:

can we construct PKE based on the "factoring assumption"

<u>Yes:</u>
<u>Rabin encryption</u>

Rabin encryption

Michael O. Rabin
born Wrocław 1931
(in 1935 emigrated to Palestine)

One of the founding fathers of computer science.

- introduced **non-determinism**
- decidability of the **monadic second order logic**
- efficient **primality testing**
- **oblivious transfer**,
- **....**

received **Turing Award** in **1976**

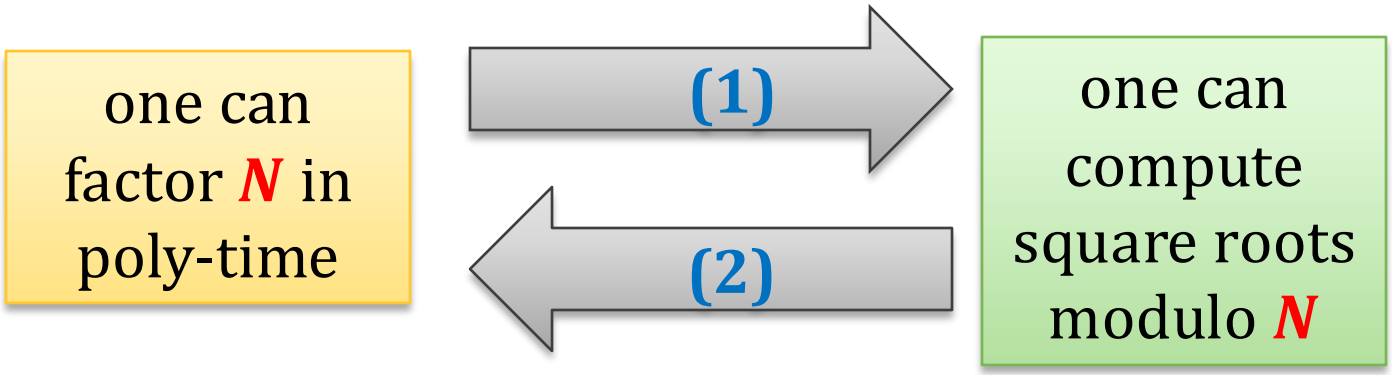
- introduced by **Michael O. Rabin** in 1979
- based on squaring in Z_N^*
- security **equivalent** to factoring

On previous lectures we proven the following

Fact

Let N be a random **RSA** modulus.

The problem of computing square roots (modulo N) of random elements in $\mathbf{QR}_N$ is poly-time equivalent to the problem of factoring N.



In other words

"squaring in $\boldsymbol{Z_N^*}$" is a one-way function (assuming the **factoring RSA moduli** is hard).

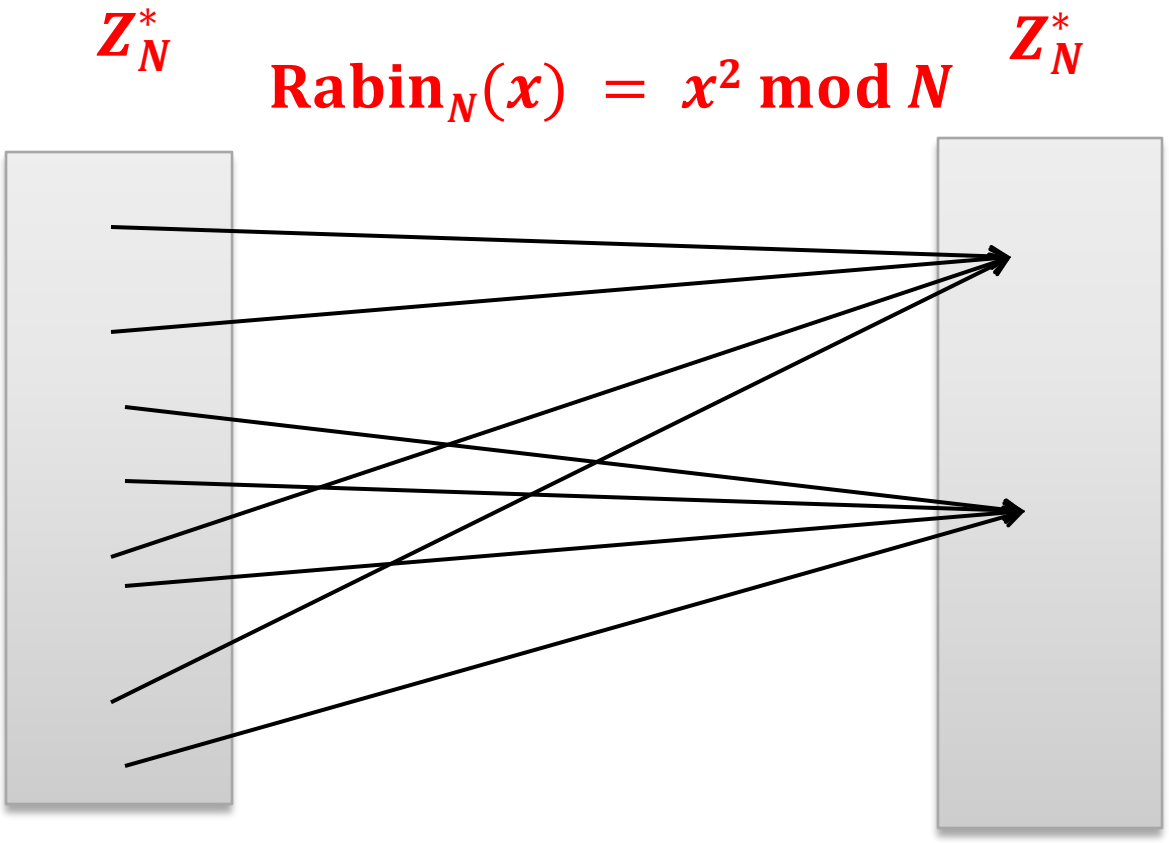
Define:

$$\mathbf{Rabin}: \boldsymbol{Z_N^* \to Z_N^*}$$

as

$$\mathbf{Rabin}(\boldsymbol{x}) \coloneqq \boldsymbol{x^2 \bmod N}$$

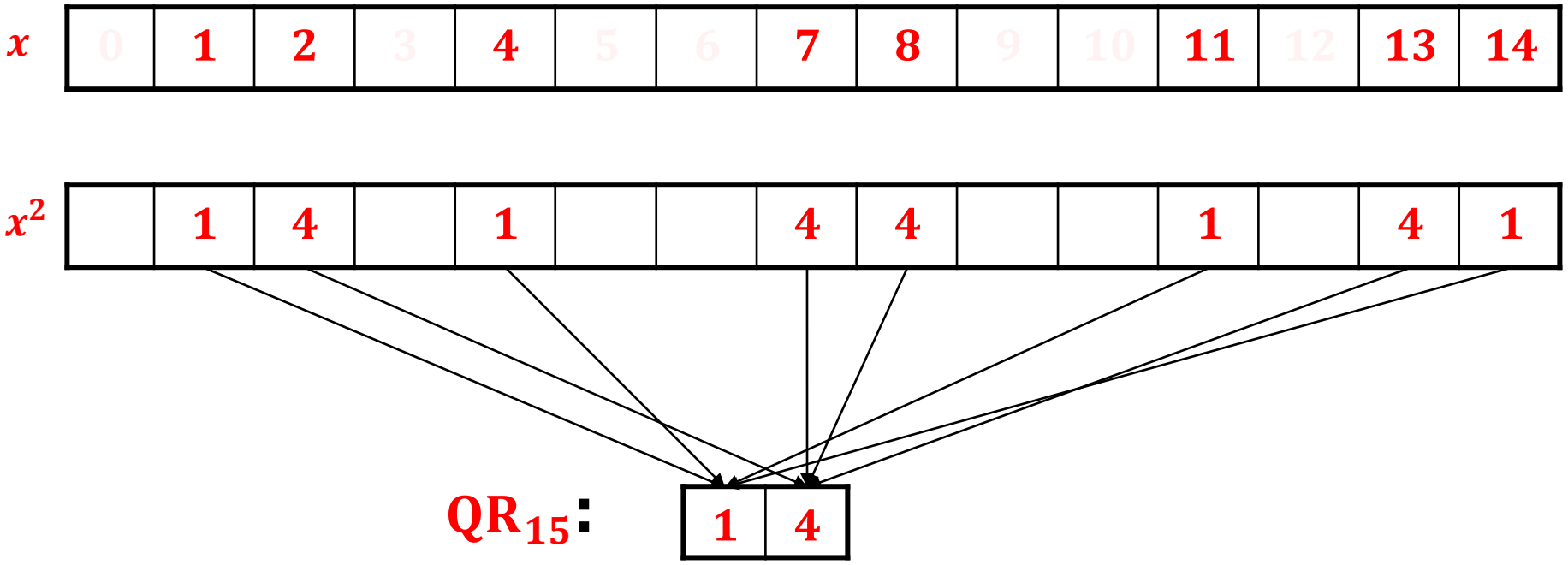
A fact about squaring modulo $N = pq$?



This function "glues" **4** elements together.

Example for $N = 15$

Z_{15}^*



How to base encryption on this?

Idea:

public key: $N = pq$

private key: (p, q)

encryption: $\mathbf{Enc}_N(x) = x^2 \bmod N$

decryption: $\mathbf{Dec}_{(p,q)}(y) = \sqrt{y} \bmod N$

can be computed efficiently if one knows p and q (see **Chapter 6**)

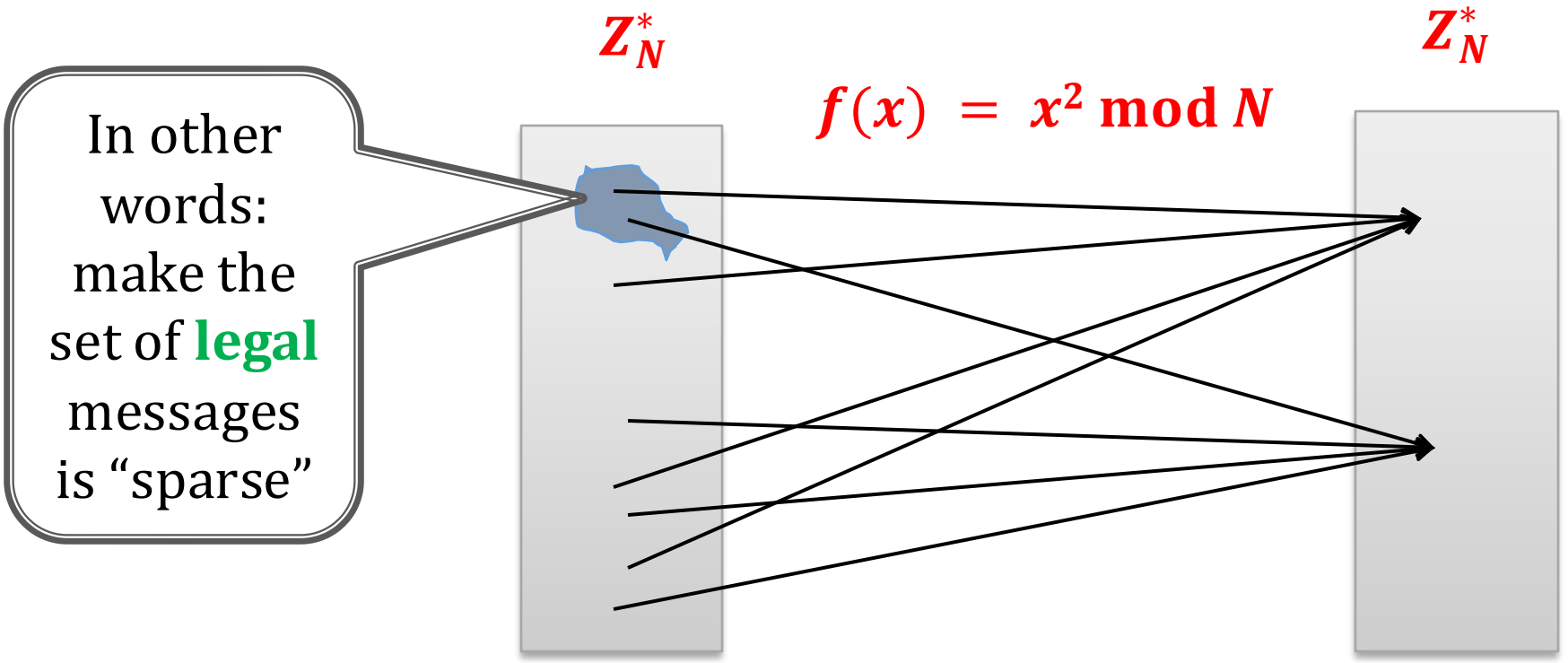
Problem: there are **4** square roots.

Solution: "make the inversion unique".

How to do it?

An ad-hoc method: add an encoding (like in the "real **RSA** encryption").

In such a way that only **1** out of the **4** square roots "make sense".



Another approach

Such an N is called a "**Blum integer**"

Fact

Suppose $N = pq$ where

$$p = q = 3 \pmod 4$$

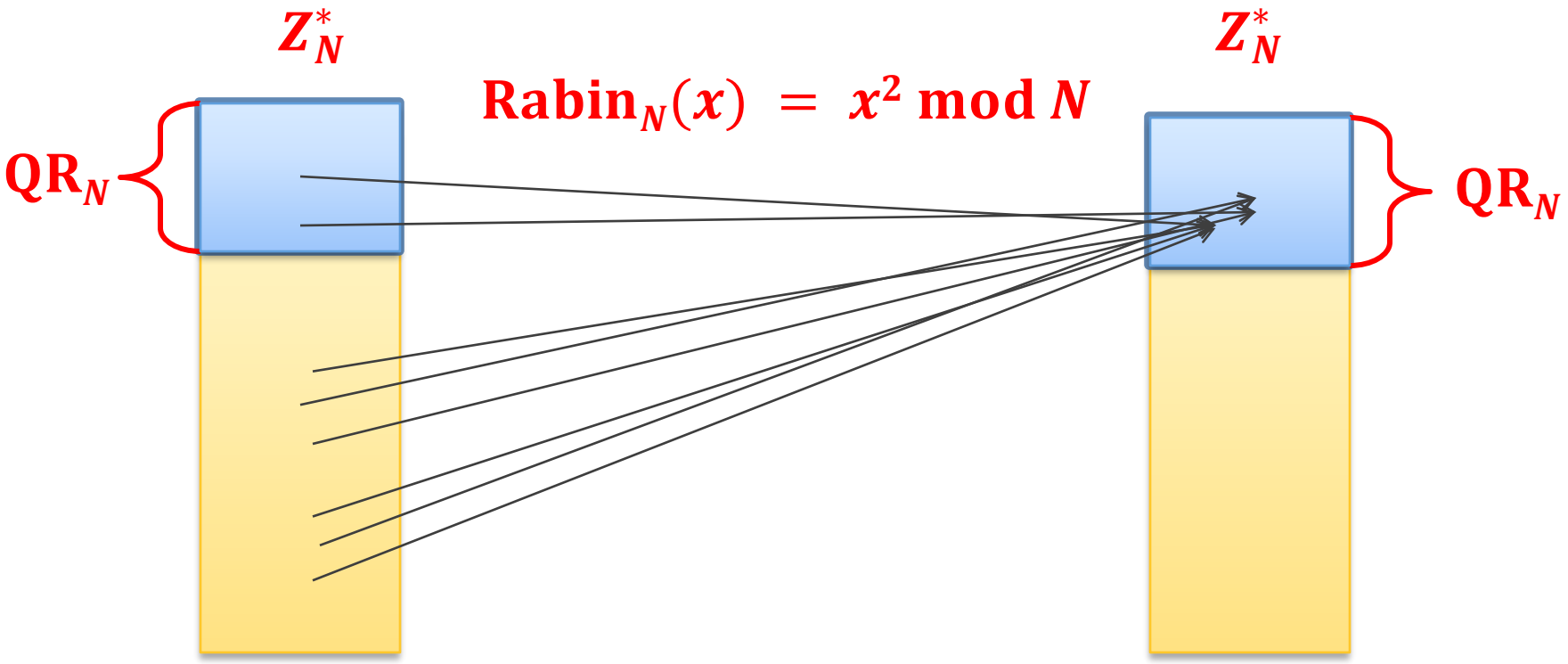
Then the function

$$\mathbf{Rabin}_N(x) = x^2 \bmod N$$

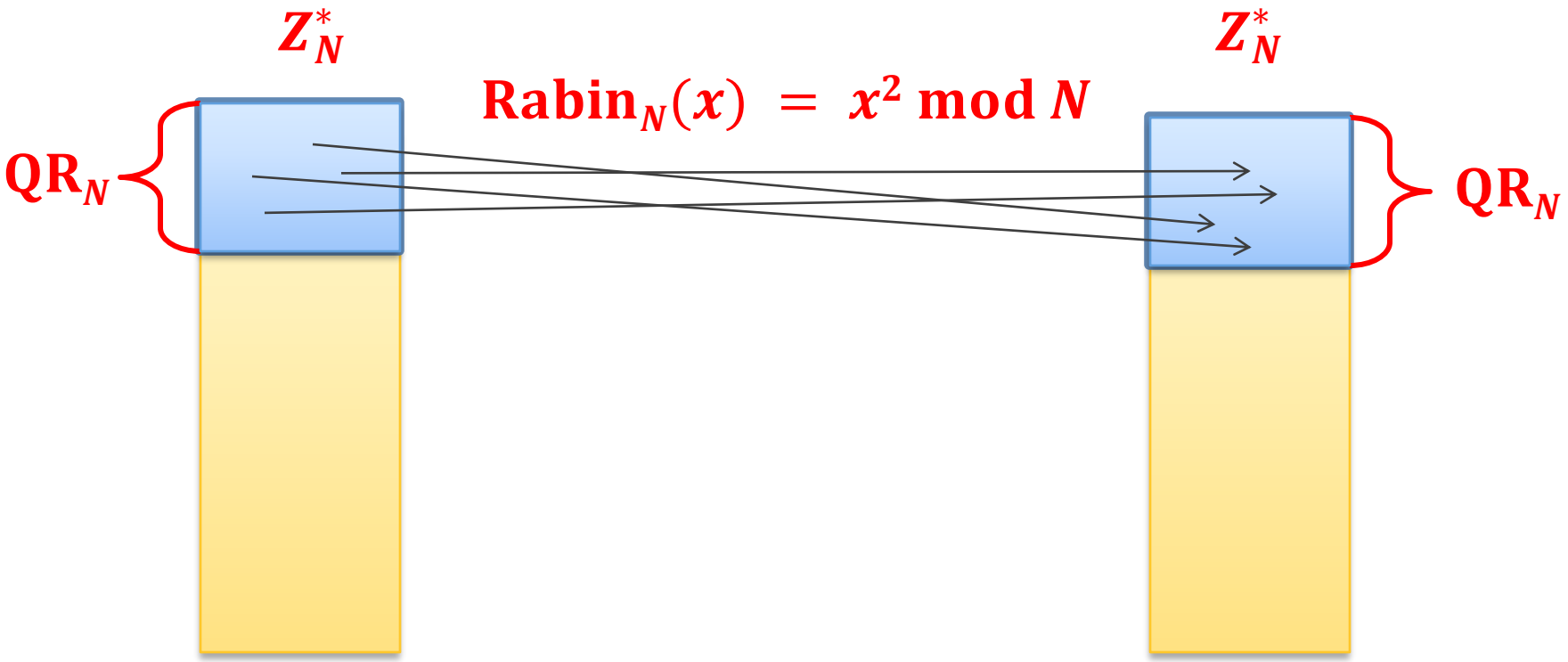
is a permutation when restricted to $\mathbf{QR}_N$

$$\mathbf{Rabin}_N : \mathbf{QR}_N \to \mathbf{QR}_N$$

How does it look?



Rabin restricted to $\mathbf{QR}_N$ is a permutation



Proof that $\mathbf{Rabin}_N(x) = x^2 \bmod N$ restricted to $\mathbf{QR}_N$ is a permutation

($N = pq$, where $p = q = 3 \bmod 4$)

We prove that **Rabin** is injective, i.e. for every $x, y \in \mathbf{QR}_N$ we have that

$$x^2 = y^2 \implies x = y$$

Observation: by **CRT** it is enough to show that

- $x^2 = y^2 \implies x = y \bmod p$ and
- $x^2 = y^2 \implies x = y \bmod q$.

By symmetry it's also enough to show it just for p.

Proof

Let $p = 4k + 3$, where $k \in \mathbf{N}$

Let $i, j \in \mathbf{N}$ be such that

- $x = g^{2i} \bmod p$ and
- $y = g^{2j} \bmod p$

where g is a generator of Z_p^* and

$$0 \leq j \leq i < \frac{p-1}{2}$$

$$= \frac{4k+2}{2}$$

$$= 2k + 1$$

Suppose we have $x, y \in \mathrm{QR_N}$ such that $x^2 = y^2 \bmod N$

$$x^2 = y^2 \bmod p$$

$$g^{4i} = g^{4j} \bmod p$$

$$g^{4(i-j)} = 1 \bmod p$$

$$p - 1 \mid 4(i-j)$$

$$4k + 2 \mid 4(i-j)$$

$$2k + 1 \mid 2(i-j)$$

$$2k + 1 \mid i - j$$

$$i = j$$

$$x = y \bmod p$$

QED

How to encrypt a one-bit message b?

Fact: the least significant bit is a **hard-core bit for the Rabin permutation**.

a Blum integer

N – public key

(p, q) – private key

$$\mathbf{Rabin}_N(x) = x^2 \bmod N$$

$$\mathbf{Rabin}_N : \mathbf{QR}_N \to \mathbf{QR}_N$$

$$\mathbf{Enc}_N(b) = (\mathbf{LSB}(x) \oplus b, \mathbf{Rabin}_N(x)),$$

where $x \in \mathbf{QR}_N$ is random.

this can be computed if one knows p and q

$$\mathbf{Dec}_{p,q}(b', y) = \mathbf{LSB}\left(\mathbf{Rabin}_N^{-1}(y)\right) \oplus b'$$

Moral

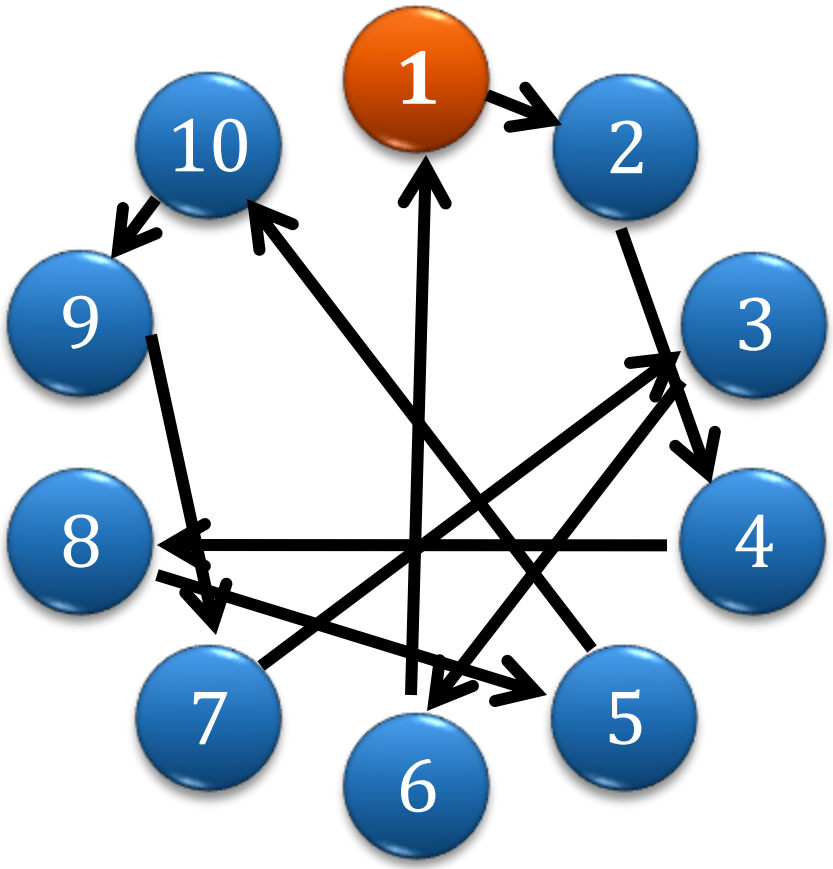
factoring RSA moduli is hard

public-key encryption exists

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. CCA security
9. Practical considerations
10. Theoretical overview

Remember the exponentiation modulo a prime?



x	$2^x \bmod 11$
0	1
1	2
2	4
3	8
4	5
5	10
6	9
7	7
8	3
9	6

2 is a generator of Z_{11}^*

Discrete log

x	g^x
0	1
1	2
2	4
3	8
4	5
5	10
6	9
7	7
8	3
9	6

Function

$$f(x) = g^x \bmod p$$

easy to compute

believed to be **hard to compute** for large p

f^{-1} is also denoted $\log_g$ and called the **discrete logarithm**

Discrete log is hard in many other groups!

How to construct PKE based on the **hardness of discrete log**?

RSA was a trapdoor permutation, so the construction was quite easy...

In case of the **discrete log,** we just have a one-way function.

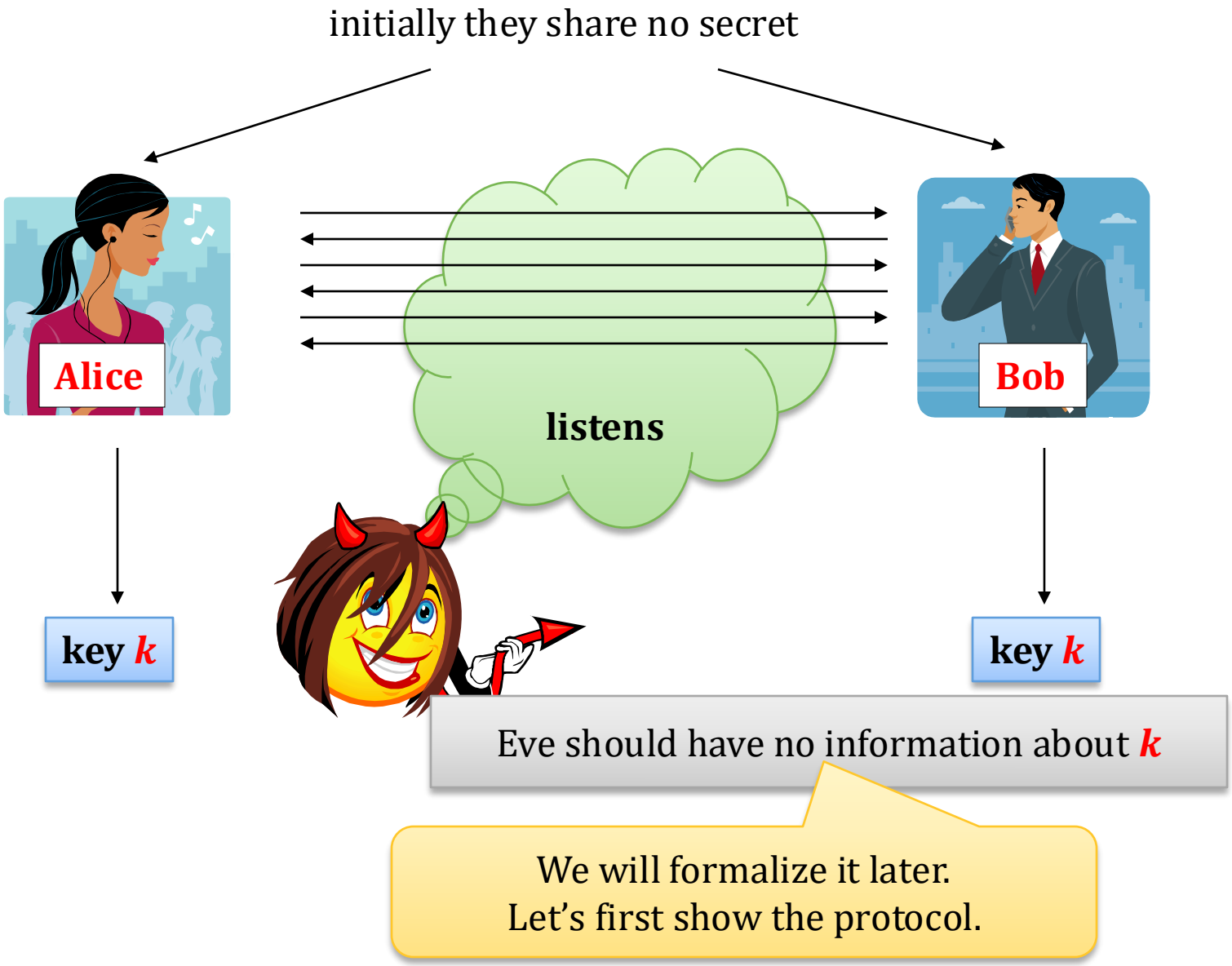
Diffie and Hellman constructed something weaker than PKE: a **key exchange protocol** (also called key **agreement** protocol).

We will now describe it. Then, we will show how to "convert it" into a **PKE**.

Plan

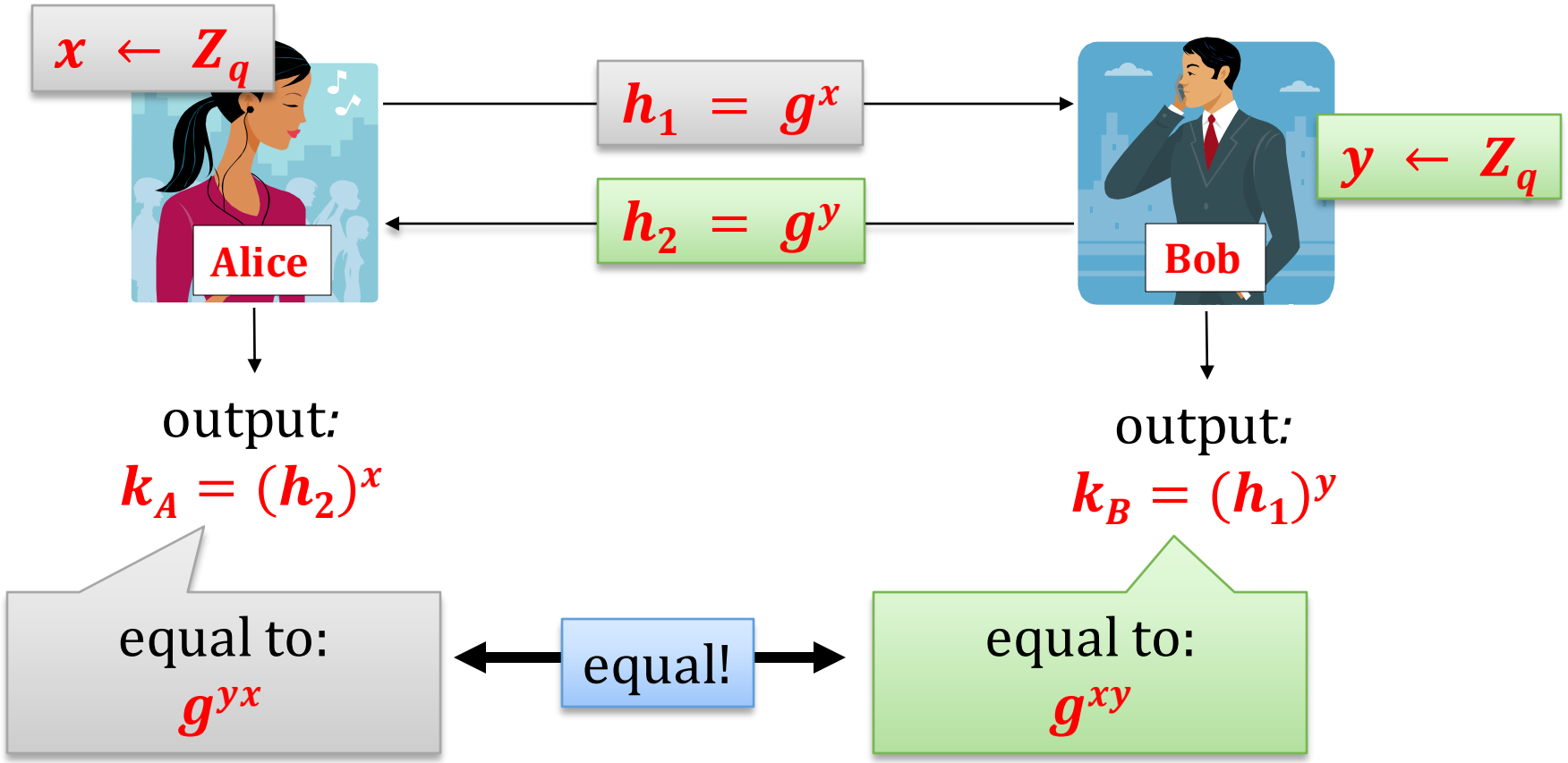
1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
    1. a tool: Diffie-Hellman key exchange
    2. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. CCA security
9. Practical considerations
10. Theoretical overview

Key exchange

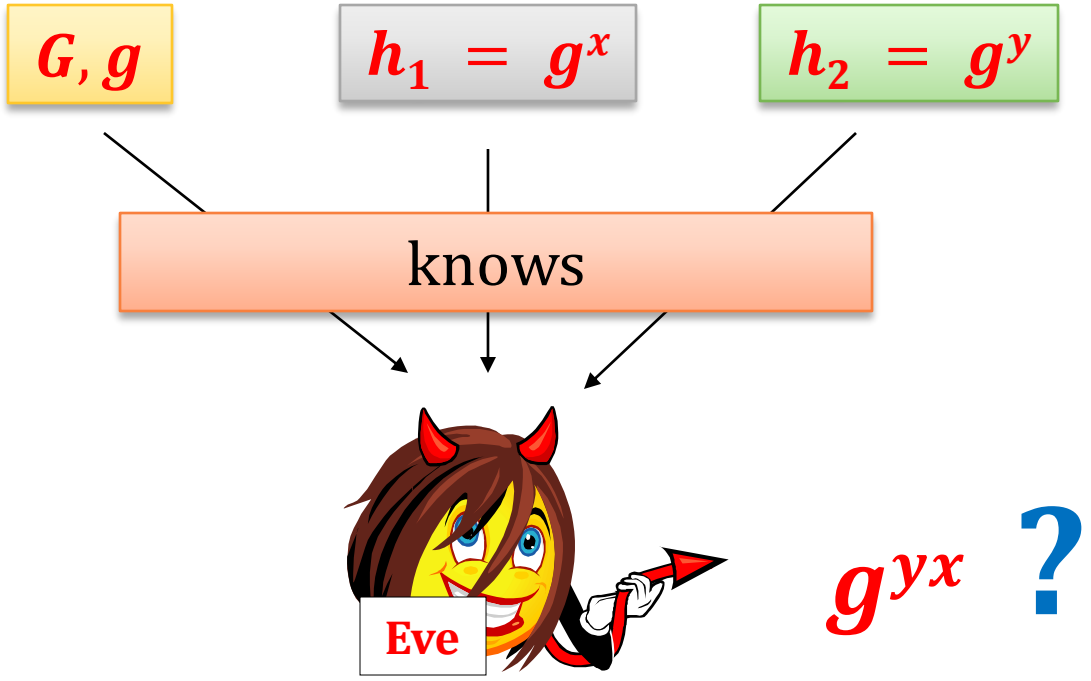


The Diffie-Hellman Key exchange

- G – a group, where **discrete log is believed to be hard**
- $q := |G|$
- g – a generator of G



Security of the Diffie-Hellman key exchange



Eve should have no information about g^{yx}.

Is it secure?

If the **discrete log in G** is easy then the **DH key exchange** is **not secure**.

(because the adversary can compute x and y from g^x and g^y)

If the discrete log in G is hard, then...

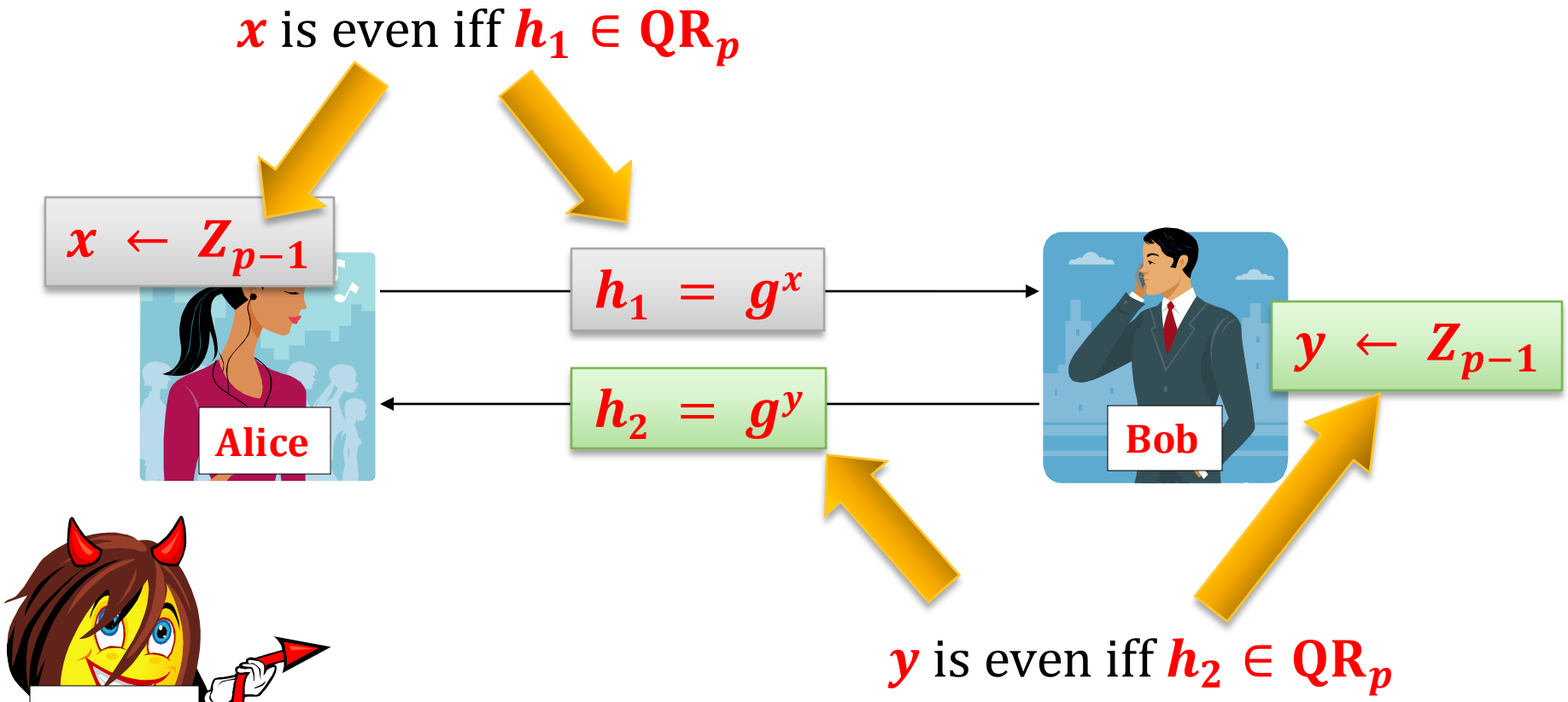
it may also not be secure

Example for $G = Z_p^*$

We use the facts that:

- **quadratic residues** in Z_p^* are **even powers of the generator**, and
- testing **membership in** $\mathbf{QR}_p$ is computationally **easy** (even for large p).

Suppose $G = Z_p^*$



Therefore:

$g^{xy} \in \mathrm{QR}_p$ iff ($h_1 \in \mathrm{QR}_p$ or $h_2 \in \mathrm{QR}_p$)

So, Eve can compute some information about g^{xy} (namely: if it is a **QR**, or not).

Solution (see Chapter 6)

Instead of working in Z_p^* work in its **subgroup**: $\mathbf{QR}_p$

How to find a generator of $\mathbf{QR}_p$?

A practical method: Choose p that is a **strong prime**, which means that:

$$p = 2 \cdot q + 1, \text{ with } q \text{ prime.}$$

Hence: $\mathbf{QR}_p$ has a **prime order** (q).

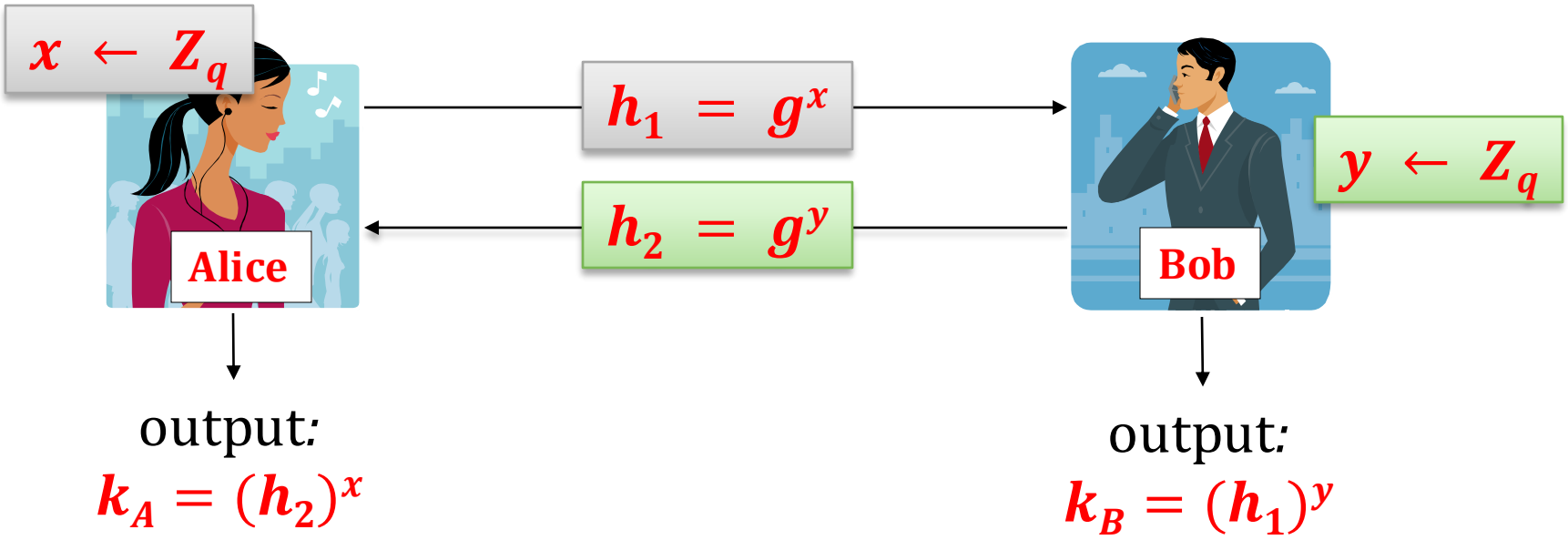
Every element (except of **1**) of a group of a prime order is its **generator**!

Therefore: every element of $\mathbf{QR}_p$ is a generator.

The DH Key exchange over QR group

Take a prime $p = 2 \cdot q + 1$, with q prime.

Take any $h \in Z_p^*$ such that $h \neq \pm 1$ and let $g = h^2 \bmod p$.

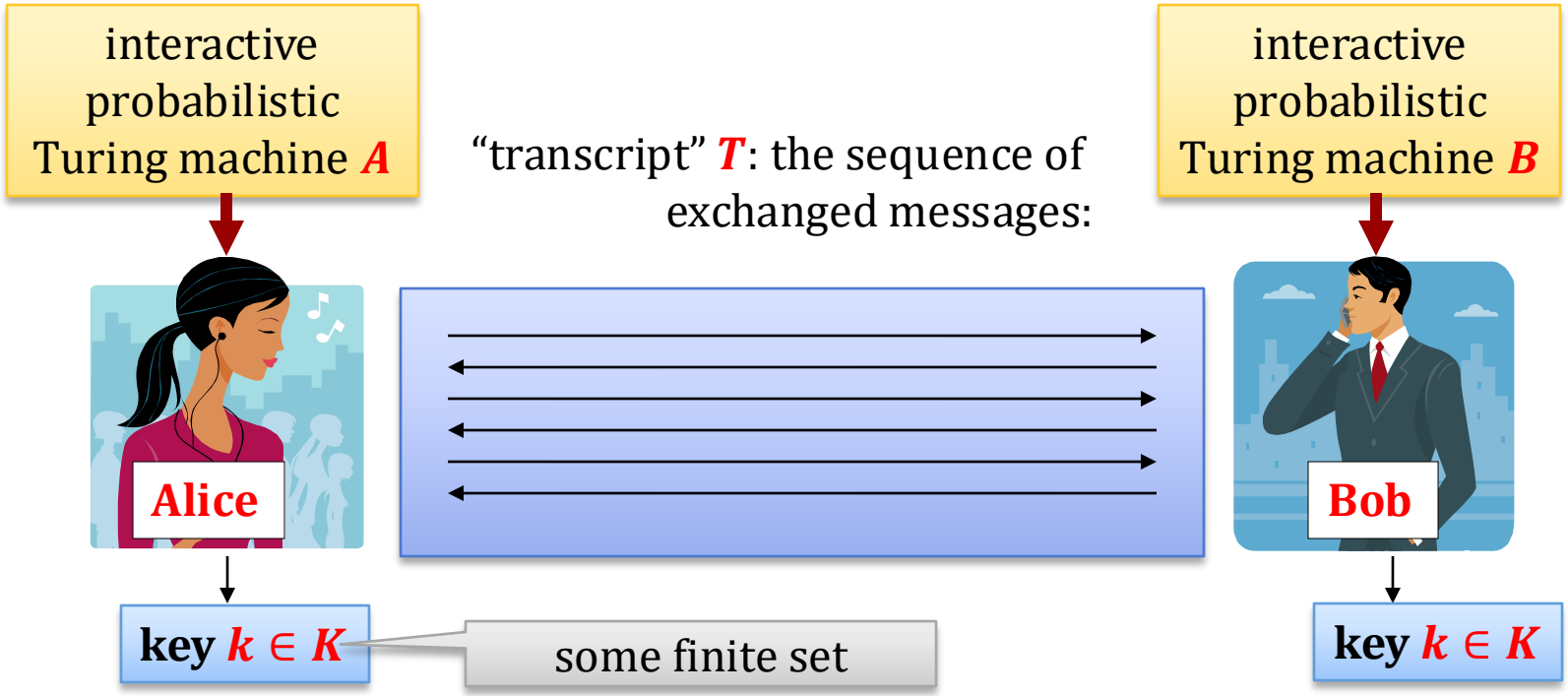


But is the partial information leakage really a problem?

We need to

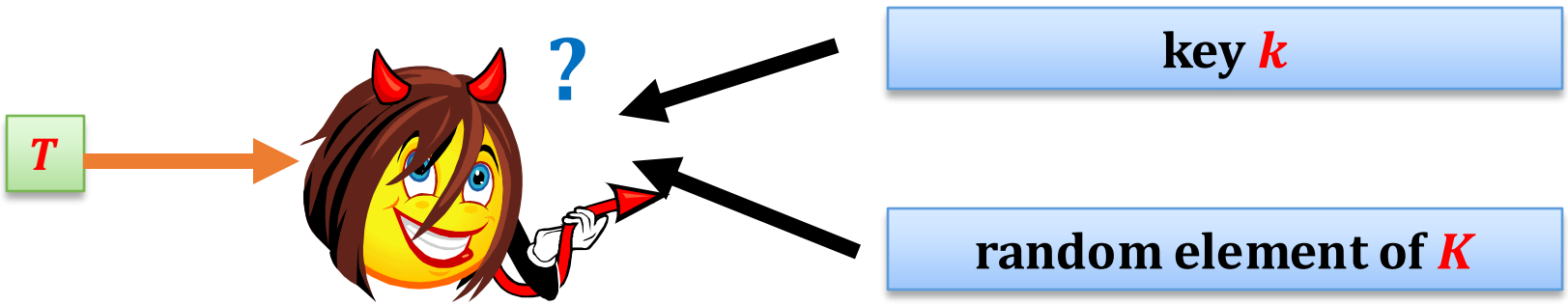
1. **formalize** what we mean by secure key exchange,

2. identify the **assumptions needed** to prove the security.

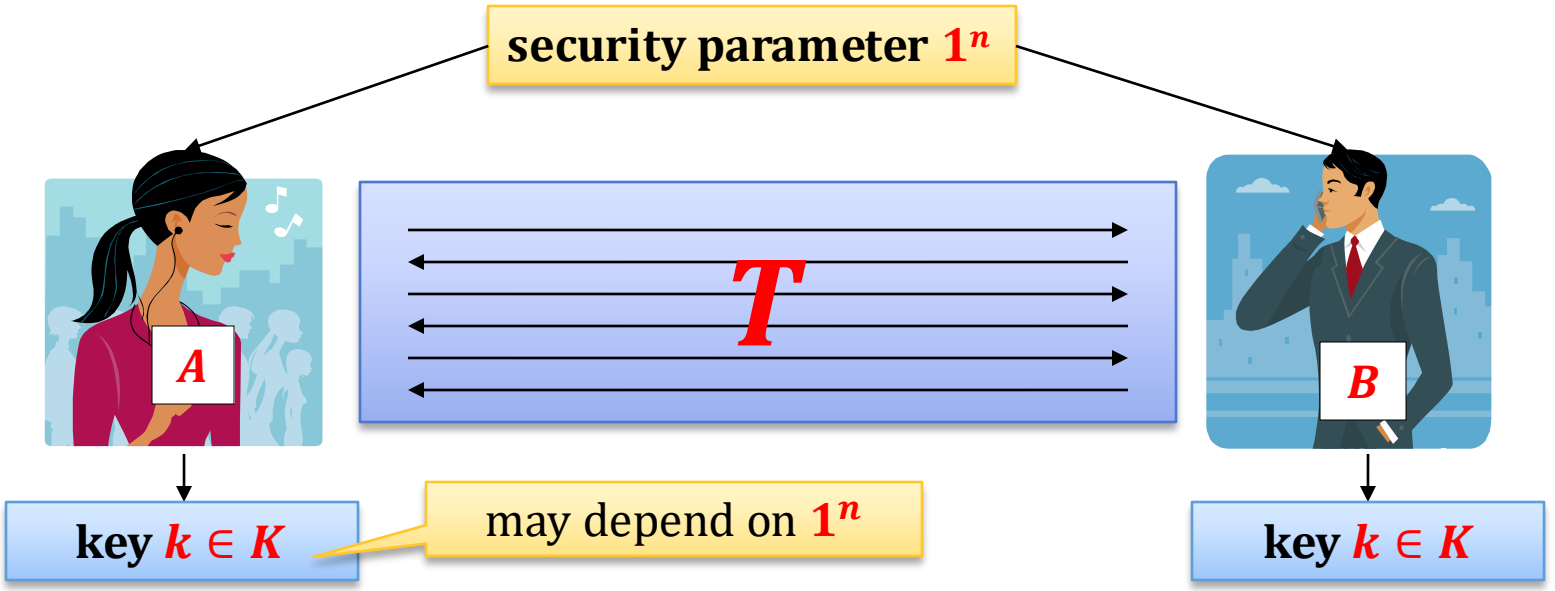


Informal definition:

(A, B) is **secure** if no "efficient adversary" can distinguish k from random, given T, with a "non-negligible advantage".



How to formalize it?



We say (A, B) is secure a secure key-exchange protocol if:
the output of A and B is always the same, and

$$\forall \; |P(M(1^n, T, k) = 1) - P(M(1^n, T, r) = 1)| \leq \mathbf{negl}(n)$$

poly-time M

$r \leftarrow K$

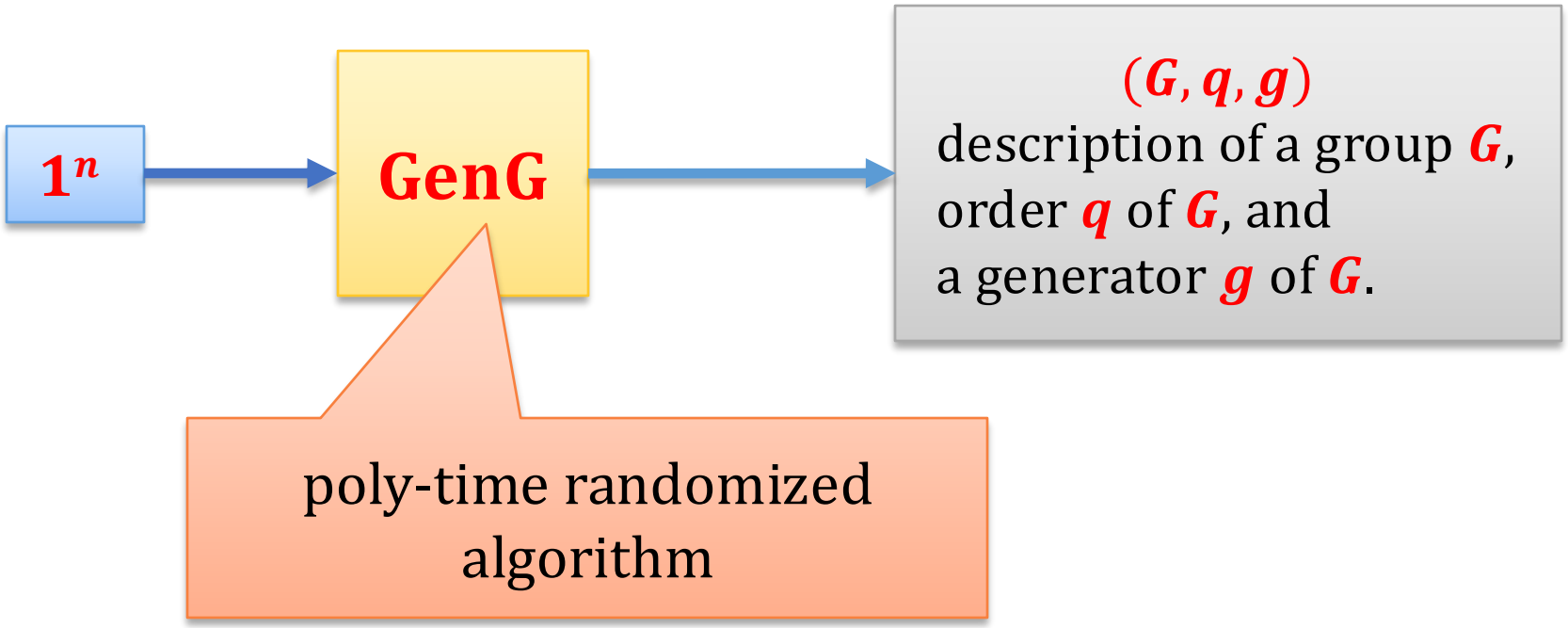
How to make G dependent on 1^n?

In **practice** often a fixed group is used.

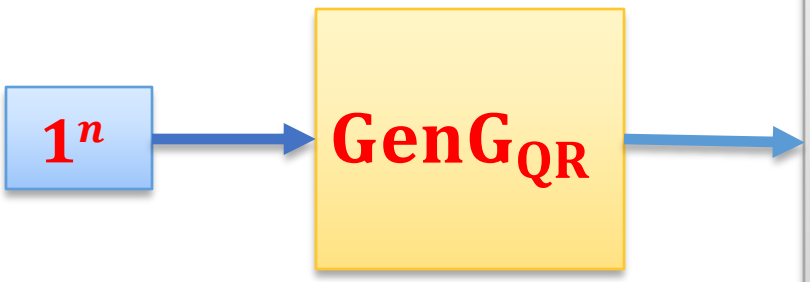
In **theory** we need to have a **new group** G for every value of 1^n.

So, we need to define an algorithm that generates G and its generator g.

Group generating algorithm **GenG**



Example of **GenG**

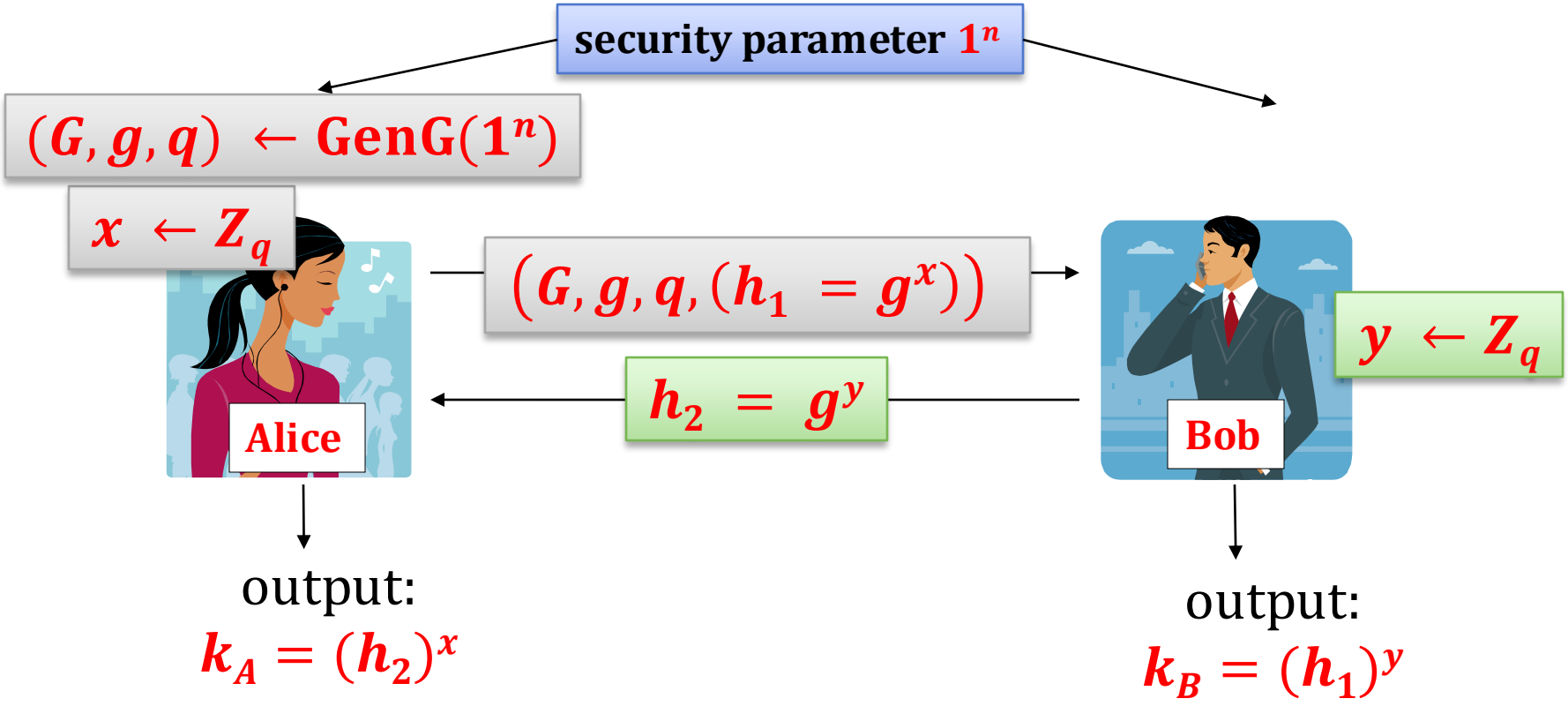


$$(Z_p^*, q, g)$$

where:

- p – random prime such that $|p| = n$
- $q = (p-1)/2$ – a prime
- and $g = h^2 \bmod p$, where $h \leftarrow Z_p^* \setminus \{-1, 1\}$

How does the protocol look now?



If such a key exchange protocol is secure, we say that: the **Decisional Diffie-Hellman (DDH) problem is hard with respect to GenG**)

Formally

Decisional Diffie-Hellman (DDH) problem is hard relative to **GenG** if for every poly-time algorithm A we have that

$$|P(A(G, q, g, g^x, g^y, g^z) = 1) - P(A(G, q, g, g^x, g^y, g^{xy}) = 1)| \leq \mathbf{negl}(n)$$

where

$$(G, q, g) \leftarrow \mathbf{GenG}(1^n)$$

and

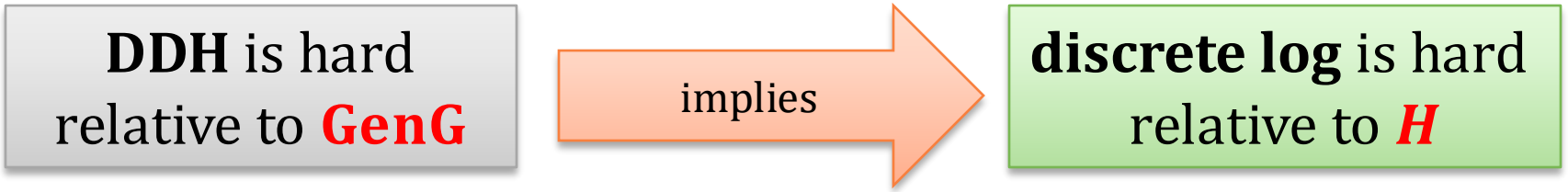
$$x, y, z \leftarrow Z_q$$

Examples

DDH is believed to be hard relative to $\mathbf{GenG_{QR}}$

Other examples: elliptic curves

How does DDH compare to the discrete log assumption



The opposite implication is unknown in most of the cases

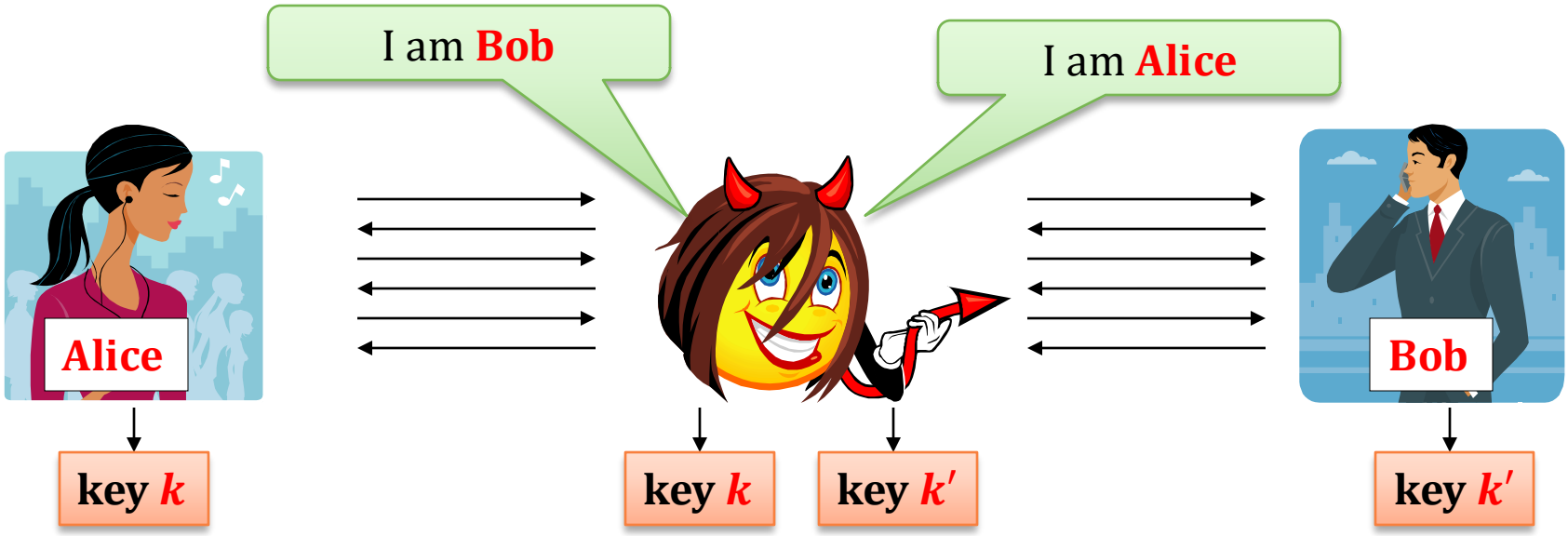
A problem

The protocols that we discussed are secure only against a **passive adversary** (that only eavesdrop).

What if the adversary is **active**?

She can launch a "**man-in-the-middle** attack".

Man in the middle attack



A very realistic attack!

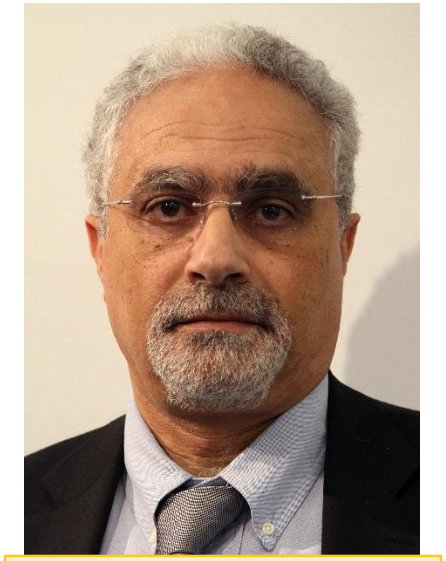
So, is this thing totally useless?
No! (it is useful as a building block)

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
    1. a tool: Diffie-Hellman key exchange
    2. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. CCA security
9. Practical considerations
10. Theoretical overview

ElGamal encryption

ElGamal is another popular public-key encryption scheme.



Taher ElGamal
(1955–)

Introduced in:

[Taher ElGamal "A Public key Cryptosystem and A Signature Scheme based on discrete Logarithms". *IEEE Transactions on Information Theory*. 1985]

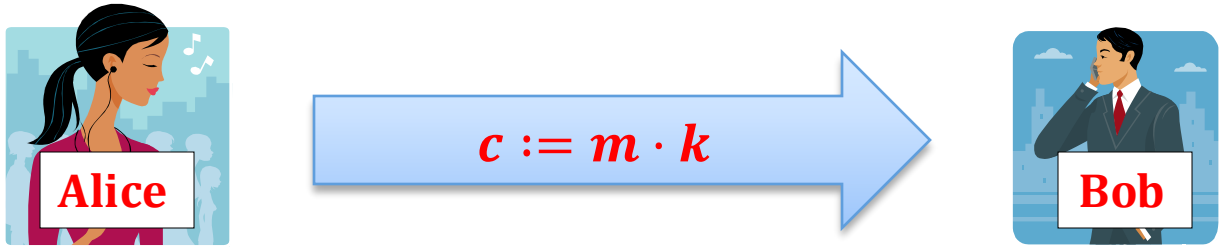
It is based on the **Diffie-Hellman** key-exchange.

First observation

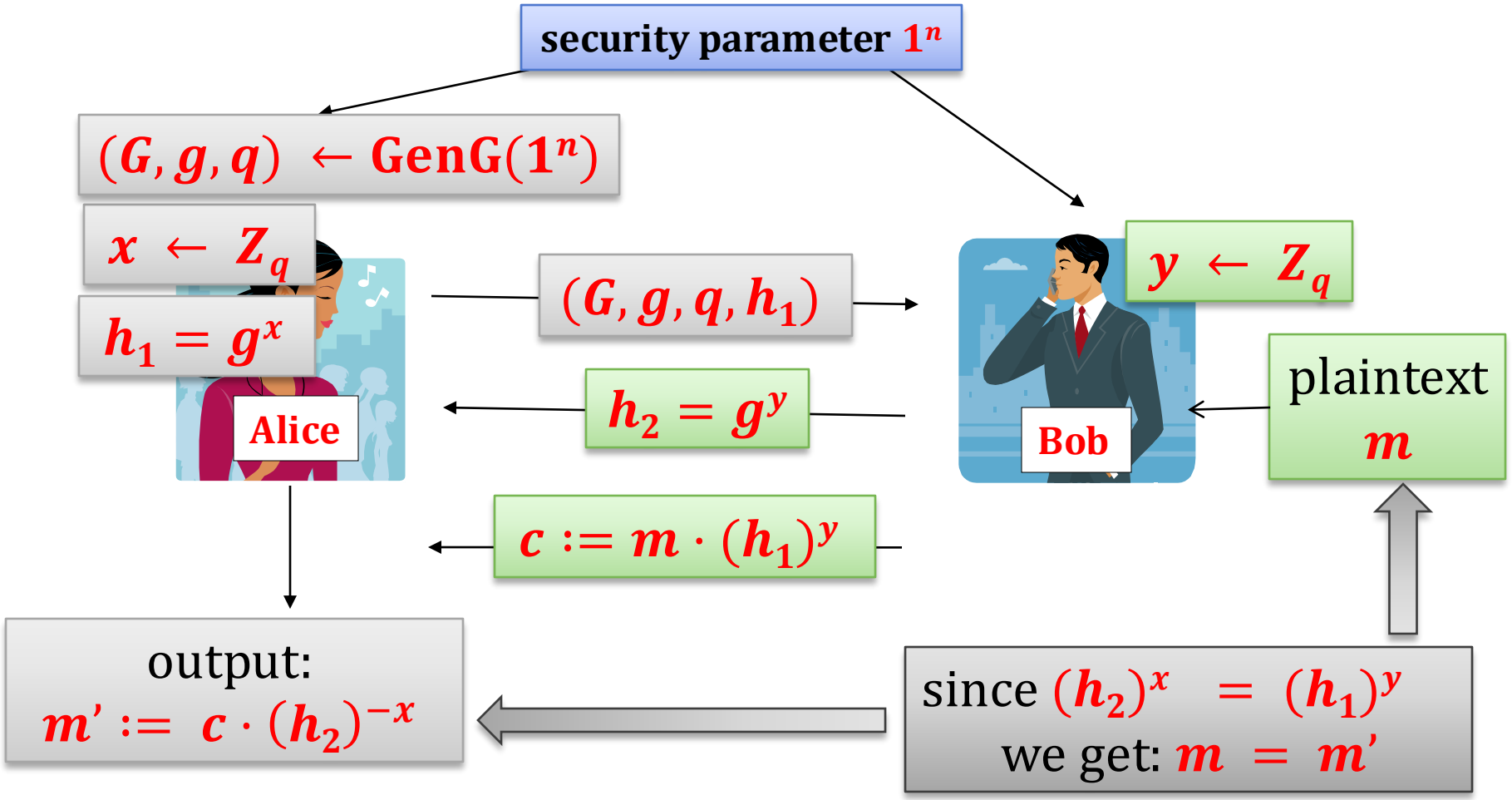
Remember that the one-time pad scheme can be generalized to any group G?

$$\mathcal{K} = \mathcal{M} = C = G$$
$$\mathbf{Enc}(k, m) = m \cdot k$$
$$\mathbf{Dec}(k, m) = m \cdot k^{-1}$$

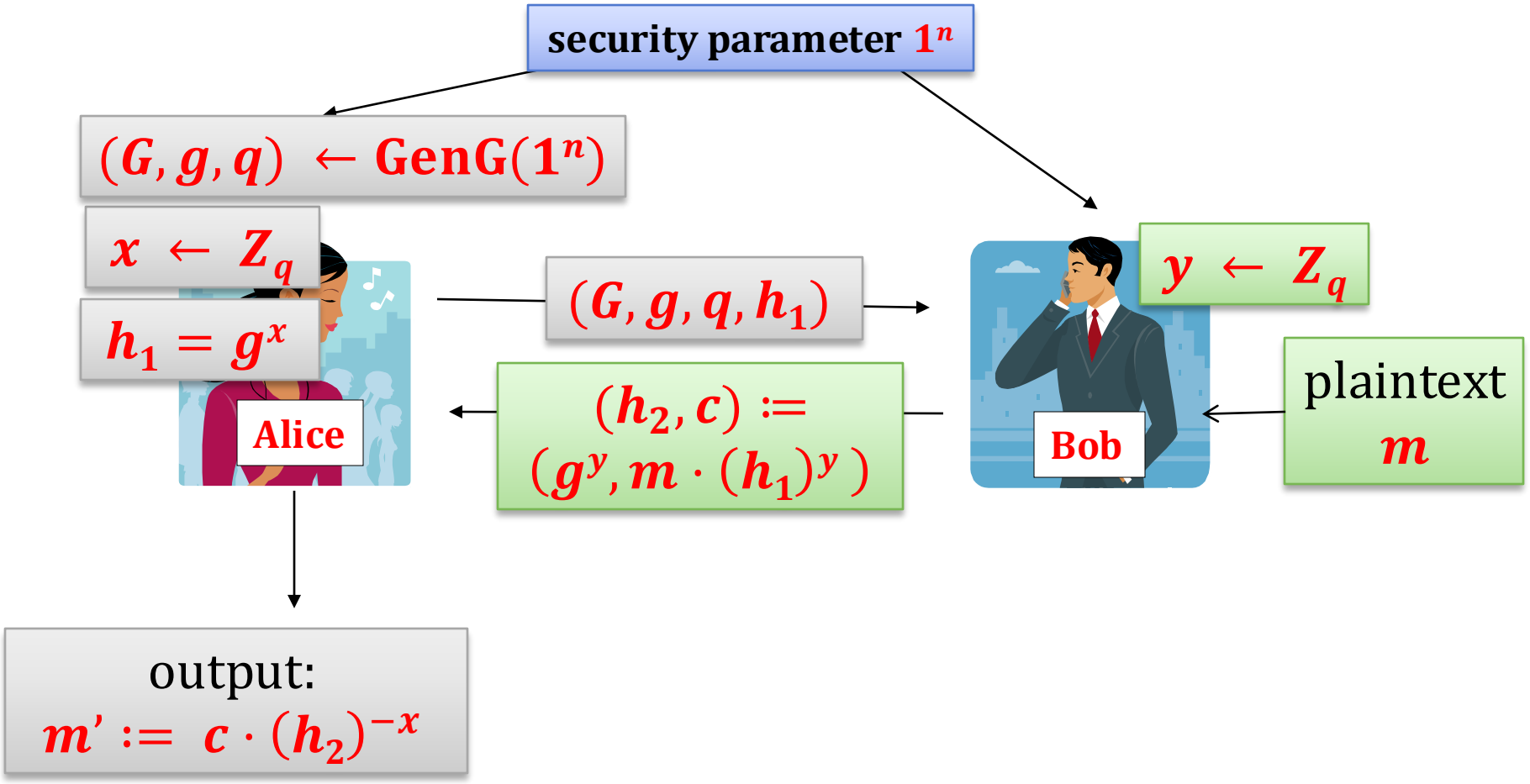
So, if k is the key agreed in the **DH key exchange**, then **Alice** can send a message $M \in G$ to **Bob** "encrypting it with k" by setting: $c := m \cdot k$



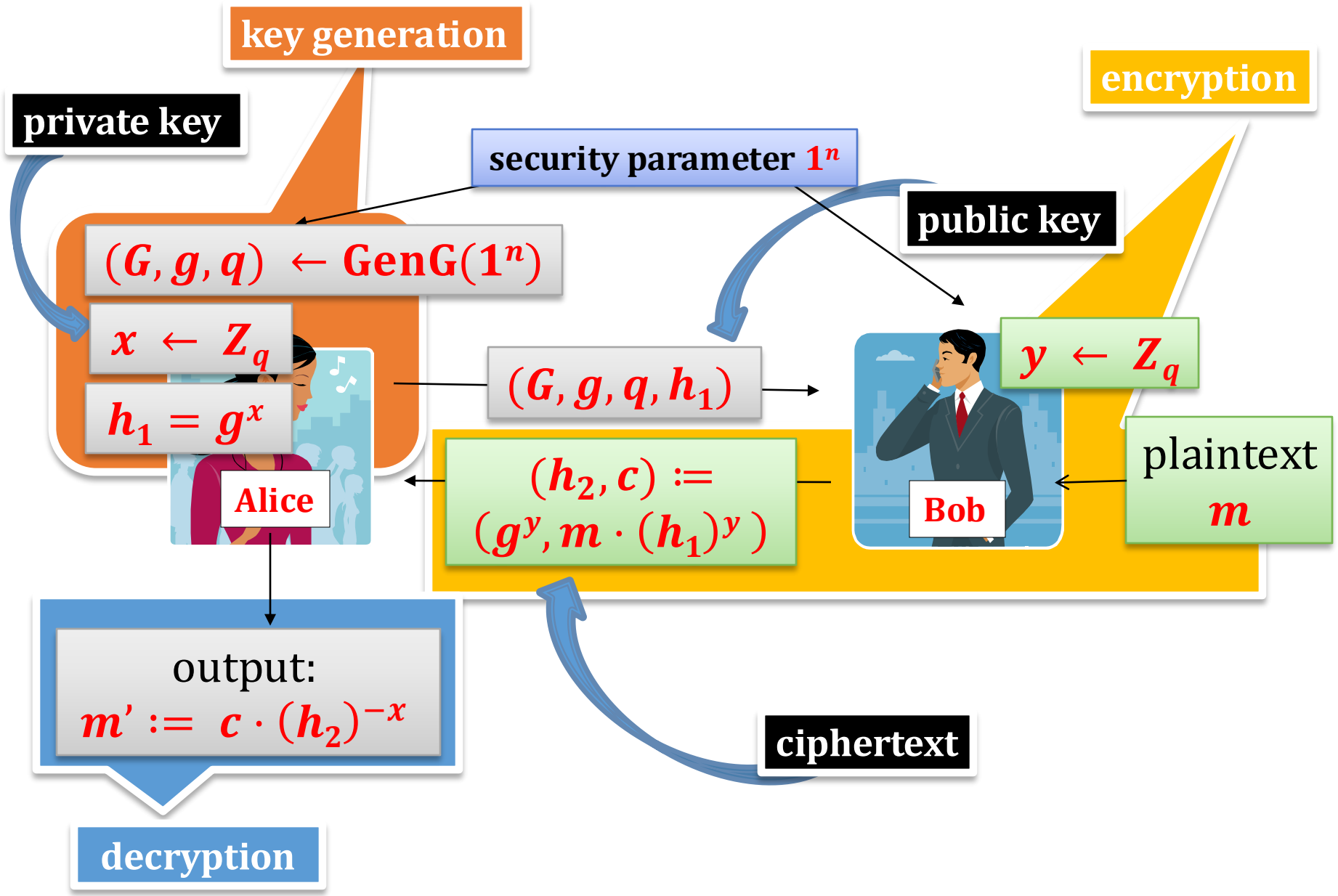
How does it look now?



The last two messages can be sent together



ElGamal encryption



ElGamal encryption

Let **GenG** be such that **DDH** is hard with respect to **GenG**.

Gen(1^n) first runs **GenG** to obtain G, g and q. Then, it chooses $x \leftarrow Z_q$ and computes $h_1 := g^x$.

The public key is (G, g, q, h_1).
The private key is (G, g, q, x).

$\mathbf{Enc}((G, g, q, h_1), m) := (m \cdot h_1^y, g^y)$,
where $m \in G$ and y is a random element of Z_q
(note: it is **randomized by definition**)

$$\mathbf{Dec}\big((G, g, q, x), (c_1, h_2)\big) := c_1 \cdot h_2^{-x}$$

Correctness

$$h = g^x$$

$$\mathbf{Enc}((G, g, q, h), m) = (m \cdot h^y, g^y)$$

$$\begin{aligned}
\mathbf{Dec}\big((G, g, q, x), (c_1, h_2)\big) &= c_1 \cdot h_2^{-x} \\
&= m \cdot h^y \cdot (g^y)^{-x} \\
&= m \cdot (g^x)^y \cdot (g^y)^{-x} \\
&= m \cdot g^{xy} \cdot g^{-yx} \\
&= m
\end{aligned}$$

ElGamal – implementation issues

Which group to choose?

E.g.: $\mathbf{QR}_p$, where p is a strong prime, i.e.: $q = \frac{p-1}{2}$ is also prime.

Plaintext space is a set of integers $\{1, \ldots, q\}$.

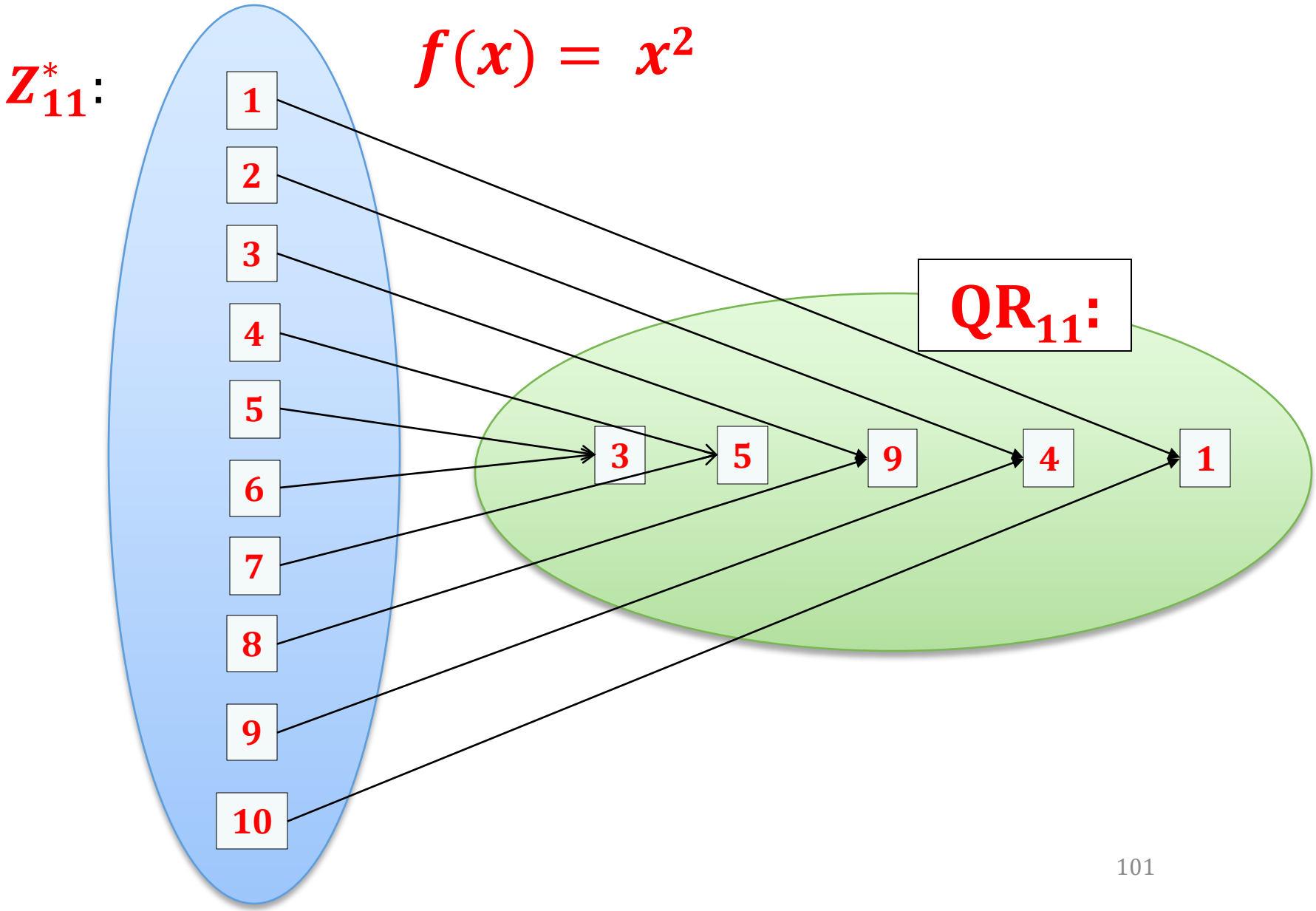
How to map an integer $i \in \{1, \ldots, q\}$ to $\mathbf{QR}_p$?

Just square:

$$f(i) = i^2 \bmod p.$$

Why is it **one-to-one**?

Remember this picture (from previous lectures)?



The mapping

So

$$f(i) = i^2 \bmod p$$

is **one-to-one** (on $\{1, \ldots, q\}$).

Is it also efficiently invertible?

Yes (see Chapter 6)

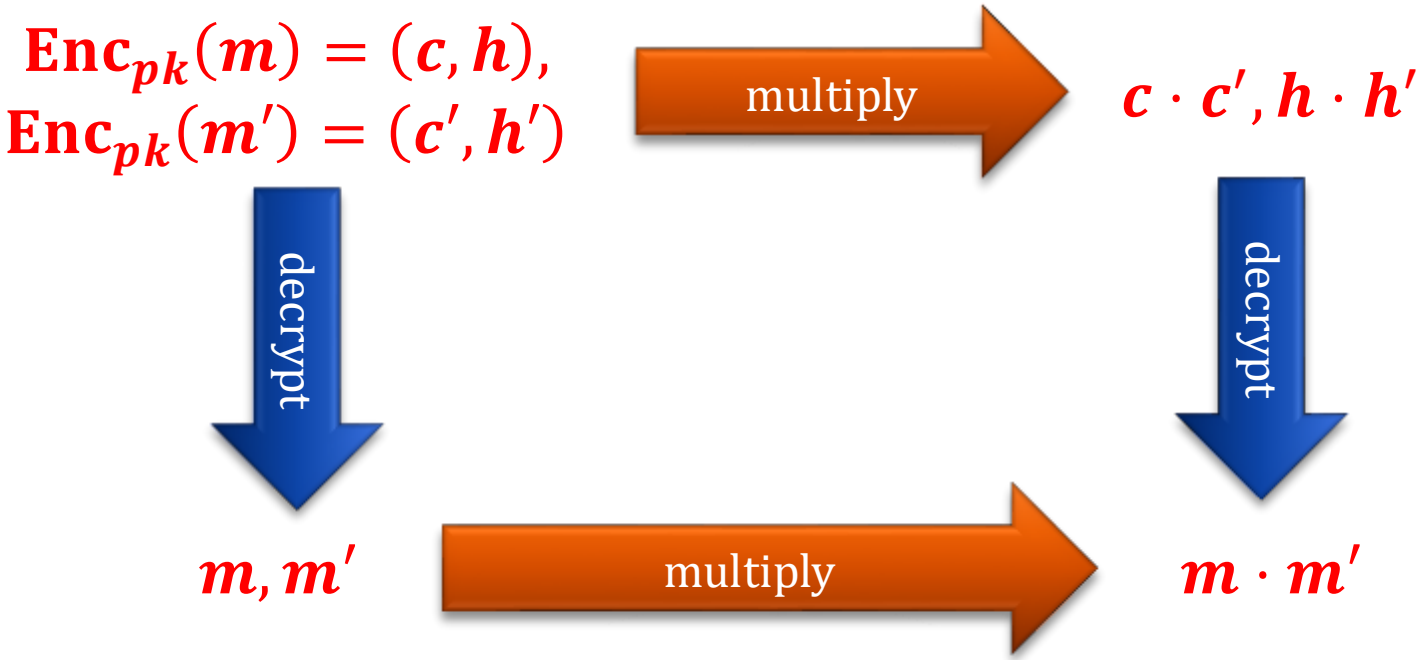
Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. CCA security
9. Practical considerations
10. Theoretical overview

ElGamal has an interesting property

homomorphism with respect to multiplication:

A "product of two ciphertexts" decrypts to a product of their corresponding messages.



Why?

- **public key**: (G, g, q, h)
- **private key**: (G, g, q, x)

$c := \mathbf{Enc}((G, g, q, h), m) := (m \cdot h^y, g^y)$, where $y \leftarrow Z_q$

$c' := \mathbf{Enc}((G, g, q, h), m') := (m' \cdot h^{y'}, g^{y'})$, where $y' \leftarrow Z_q$

product of c and c':

$$(m \cdot m' \cdot h^y \cdot h^{y'}, g^y \cdot g^{y'})$$
$$= \left(m \cdot m' \cdot h^{y+y'}, g^{y+y'}\right)$$

this is an encryption of $m \cdot m'$ with randomness $y + y'$

Homomorphism – good or bad?

Sometimes homomorphism is a security weakness (think of the **CCA security**).

On the other hand: it can also be a plus.

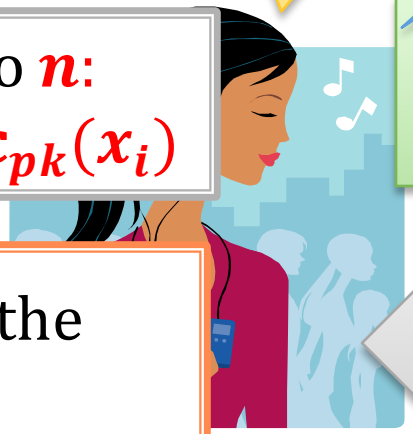
One example: cloud computing

Example: outsourcing computation

has a large set
$\{x_1, \ldots, x_n\} \subseteq \mathrm{QR}_p$
and wants to learn
$x = x_1 \cdot \cdots \cdot x_n \bmod p$

generated a key pair
$pk = (\mathrm{QR}_p, g, q, h)$
$sk = (\mathrm{QR}_p, g, q, x)$

for $i = 1$ to n:
$c_i := \mathrm{Enc}_{pk}(x_i)$



p,
$c_1, \ldots, c_n$

computes
$c := c_1 \cdot \cdots \cdot c_n \bmod p$

c

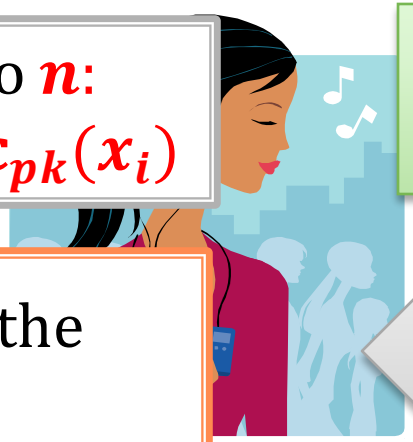
computes the result as:
$x := \mathrm{Dec}_{sk}(c)$

Observe: the server doesn't learn the x_i's!

This can be generalized!

The example on the previous slide was a bit artificial.
But think about the following.

has some data $x_1, \ldots, x_n$ and wants to learn $x = f(x_1, \ldots, x_n)$ for some function f.



for $i = 1$ to n:
$c_i := \mathrm{Enc}_{pk}(x_i)$

pk,
$c_1, \ldots, c_n$

computes c from
$c_1, \ldots, c_n$

c

computes the result as:
$x := \mathrm{Dec}_{sk}(c)$

but how to do it for any f?

Fully homomorphic encryption (FHE)

Constructing encryption scheme that would allow "**homomorphic computation**" of any function ***f*** was an **open problem** until **2009**.

The first such construction was given in:

> **Craig Gentry. Fully Homomorphic Encryption Using Ideal Lattices. ACM Symposium on Theory of Computing (STOC), 2009**.

Working towards construction of **practical FHE** is an active research area.

A natural (but much simpler) question

Can we construct an encryption scheme that is homomorphic **with respect to addition**?

Answer: **Yes, Paillier cryptosystem**

[Pascal Paillier "Public-Key Cryptosystems Based on Composite Degree Residuosity Classes". EUROCRYPT 1999]

Paillier cryptosystem works over $Z^*_{N^2}$, where N is an **RSA modulus**

Let $N := pq$.

public key: N

private key: (p, q)

How does $Z^*_{N^2}$ look like?

Fact:

$$\begin{aligned}\varphi(N^2) &= p(p-1) \cdot q(q-1) \\ &= pq \cdot (p-1)(q-1) \\ &= N \cdot \varphi(N)\end{aligned}$$

Moreover

$Z_{N^2}^*$ is isomorphic to $Z_N \times Z_N^*$ with the following isomorphism

$$f: Z_N \times Z_N^* \to Z_{N^2}^*$$

$$f(a,b) = (1+N)^a \cdot b^N \bmod N^2$$

If $x = f(a,b)$ then we will also write: $x \leftrightarrow (a,b)$

[proof: exercise]

Another fact

Fact: for any integer a we have that

$$(1+N)^a = 1 + a \cdot N \pmod{N^2}$$

Proof:

$$(1+N)^a = 1 + \binom{a}{1} N^1 + \binom{a}{2} N^2 + \cdots + \binom{a}{a} N^a$$

$$= 1 + \binom{a}{1} N \pmod{N^2}$$

$$= 1 + a \cdot N \pmod{N^2}$$

QED

A consequence of this fact

Fact: for any integer a we have that

$$(1+N)^a = 1 + a \cdot N \pmod{N^2}$$

Consequence: order of $1+N$ in $Z_{N^2}^*$ is N.

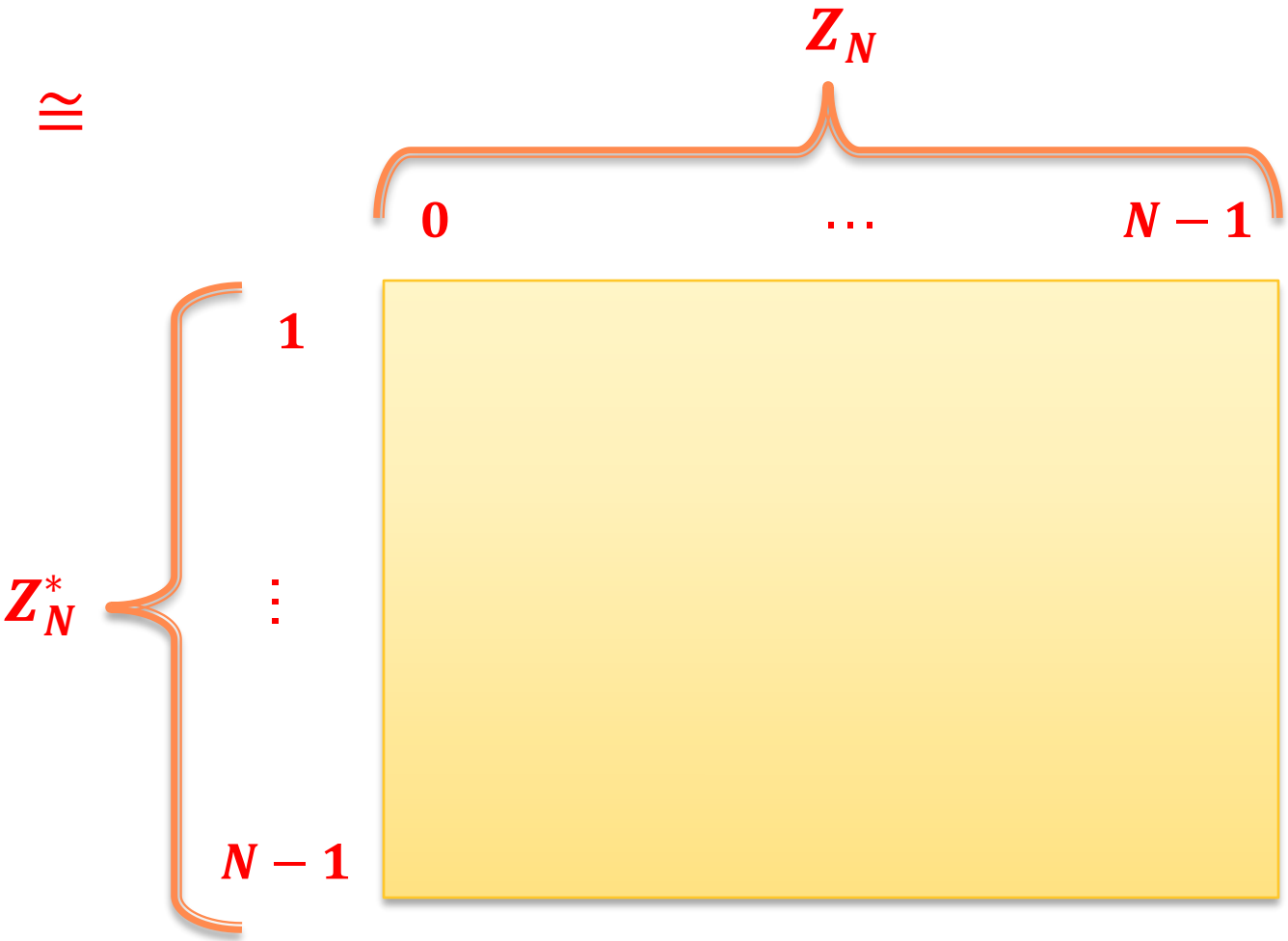
why?

because:

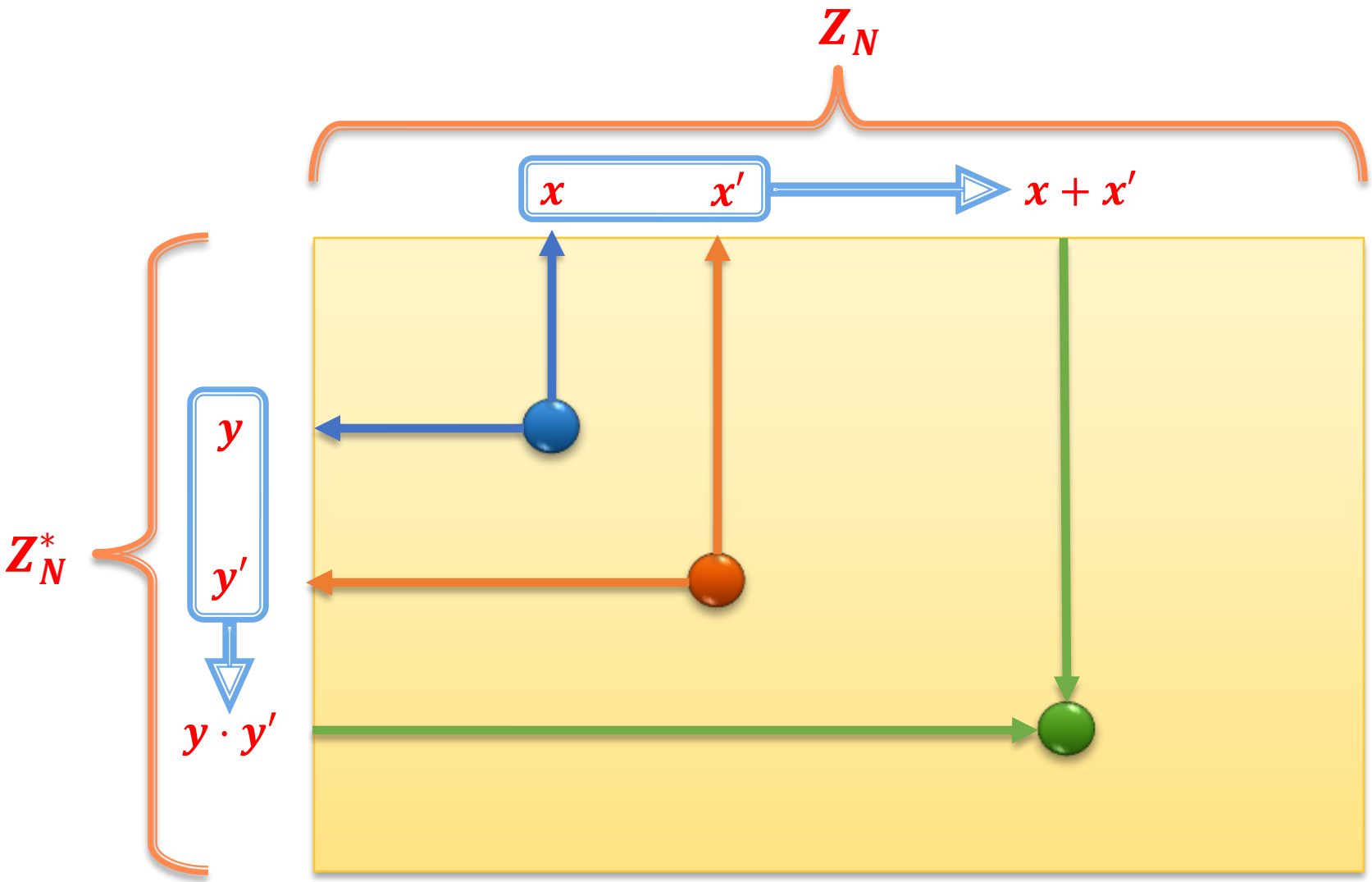
- for $0 < a < N$ we have $1 < 1 + a \cdot N < N^2$
- and $1 + N \cdot N = 1 \pmod{N^2}$

Structure of $Z_{N^2}^*$

$$Z_{N^2}^* \cong$$



Multiplication in $Z^*_{N^2}$



Nth residues in $Z_{N^2}^*$

A number $y \in Z_{N^2}^*$ is called an **Nth residue modulo N^2** if there exists $x \in Z_{N^2}^*$ such that

$$y = x^N \bmod N^2$$

How do the Nth residues look like?

A form of every Nth residue

Suppose $x \leftrightarrow (a, b)$.

Then

$$x^N \leftrightarrow (N \cdot a \bmod N, b^N \bmod N)$$
$$= (0, b^N \bmod N)$$

So every Nth residue is of a form

$$y \leftrightarrow (0, c)$$

Is every element of this form an Nth residue?

Yes!

A proof that every element $(0, c)$ is an Nth residue

Take $y \leftrightarrow (0, c)$. Let $d = N^{-1} \bmod \varphi(N)$.

this is possible because $N \perp \varphi(N)$

[exercise]

For an arbitrary $a \in Z_N$ let x be such that

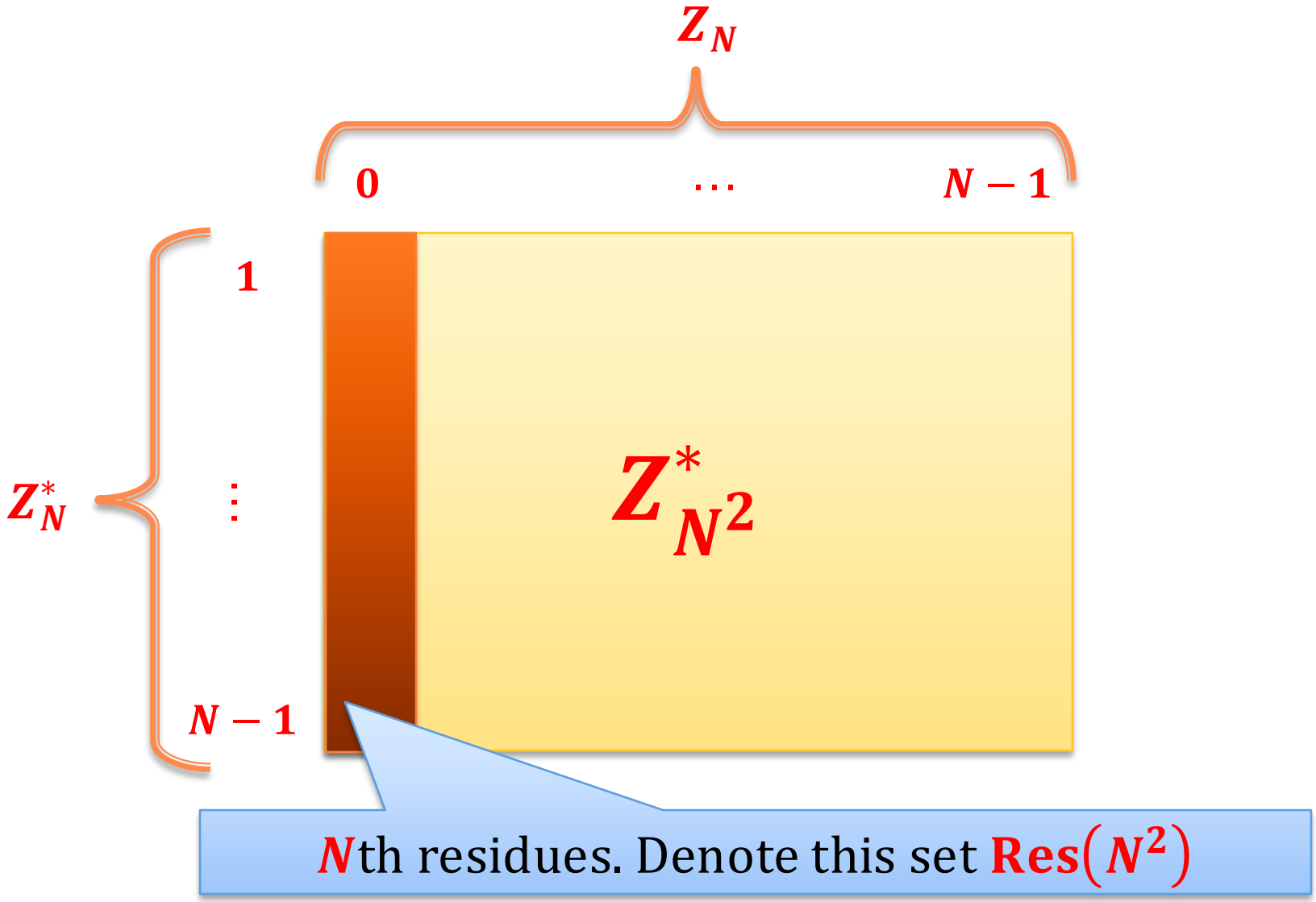
$$x \leftrightarrow \left(a, c^d\right)$$

We have:

$$\begin{aligned} x^N &\leftrightarrow \left(Na \bmod N, c^{dN} \bmod N\right) \\ &= \left(0, c^{dN \bmod \varphi(N)}\right) \\ &= (0, c^1) \\ &= (0, c) \end{aligned}$$

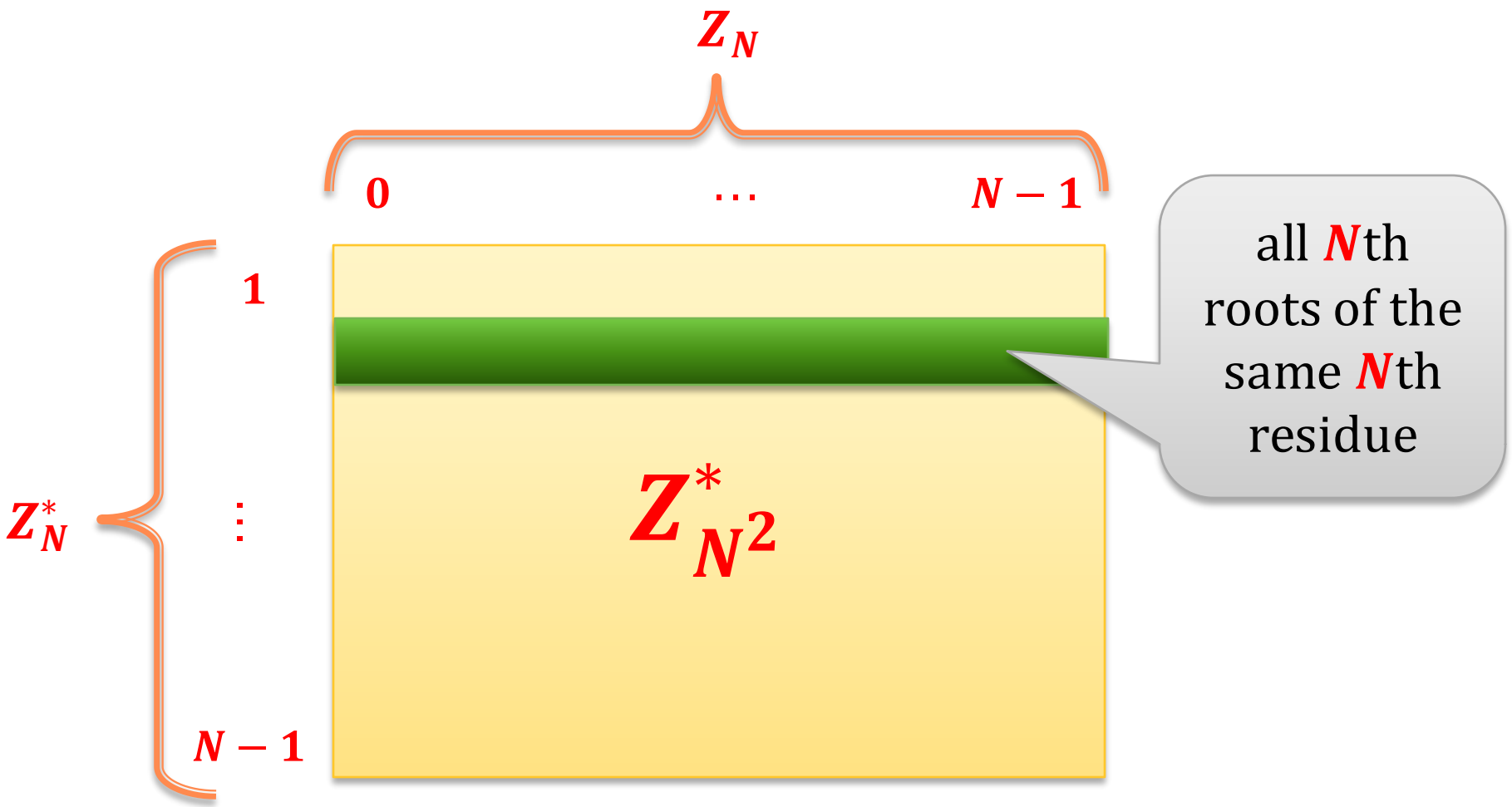
Observe: this also shows that every Nth residue y has exactly N roots $\sqrt[N]{y}$.

The Nth residues pictorially



Also

The Nth roots of every $(0, c)$ have a form (a, c^d):



Corollary

It's easy to choose a random Nth residue:

Just take a random element $x \leftarrow Z^*_{N^2}$ and compute $y = x^N \bmod N^2$.

Which problem is **hard** $Z^*_{N^2}$ (if one doesn't know p and q)?

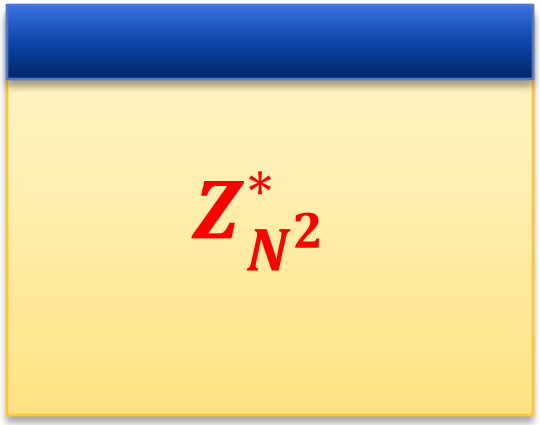
Decisional composite residuosity (DCR) assumption

Informally:

It is hard to distinguish random element of $\mathbf{Res}(N^2)$ from a random element of $Z^*_{N^2}$.

How to encrypt?

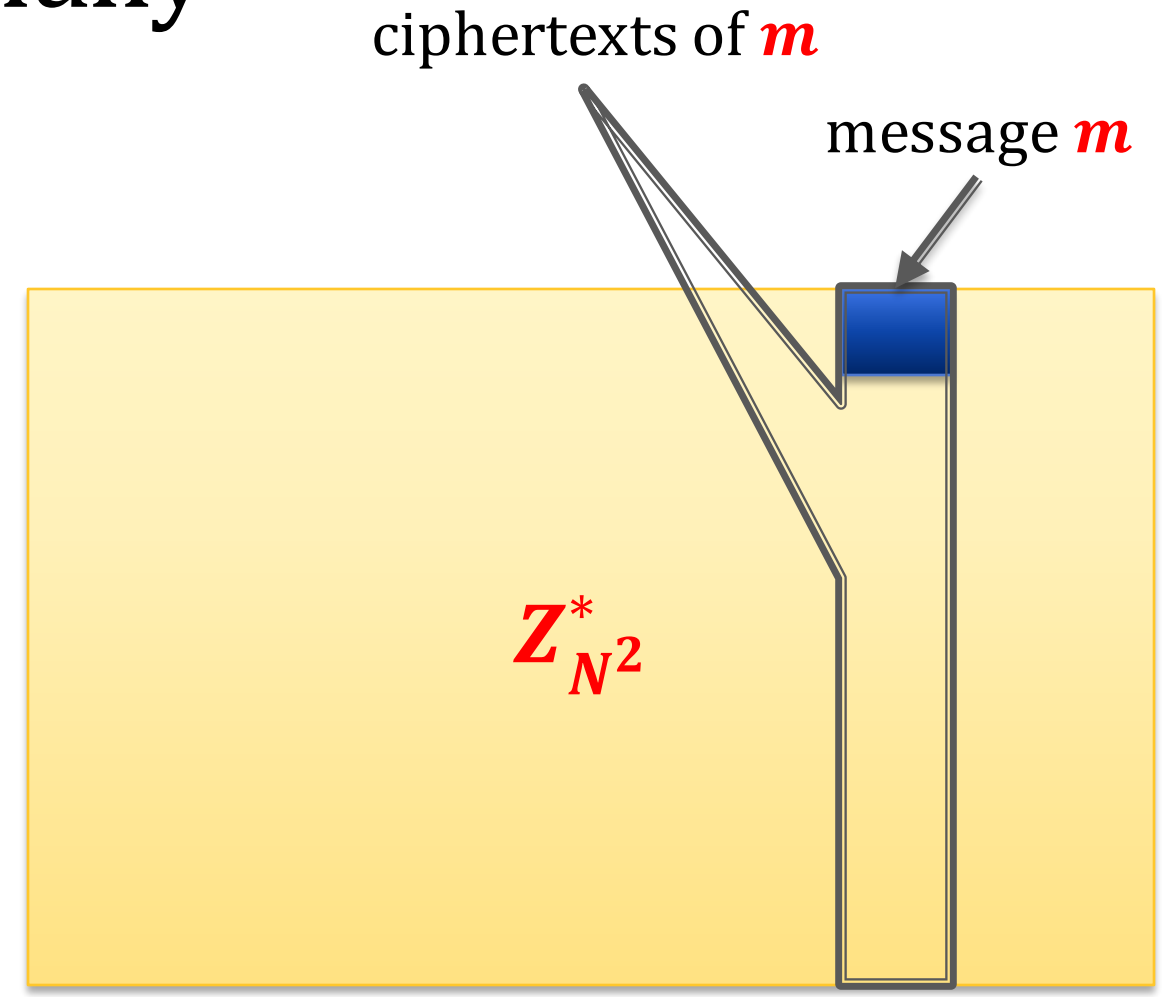
Main idea: messages are elements $x \leftrightarrow (a, 1)$ (for $a \in Z_N$)



To encrypt a message m multiply it by a random $r \leftarrow \mathbf{Res}(N^2)$:

$$\mathbf{Enc}_N(m) = m \cdot r$$

Pictorially



Two questions

1. Is this **secure**?
2. How to **decrypt**?

Security follows from the **DCR** assumption

Proof (sketch):
Take the original scheme

$$\mathbf{Enc}_N(m) = m \cdot r \text{ where } r \leftarrow \mathbf{Res}(N^2)$$

and modify it as follows:

$$\mathbf{Enc}_N(m) = m \cdot r \text{ where } r \leftarrow Z^*_{N^2}$$

Easy to see:

1. the **modified scheme hides the message completely** (it's a "generalized one-time pad")
2. if these **two schemes can be distinguished then the DCR assumption is broken**.

How to decrypt?

$\mathbf{Enc}_N(m) = m \cdot r$ where $r \leftarrow \mathbf{Res}(N^2)$

Let's view encryption as a function in $Z_N \times Z_N^*$:

$\mathbf{Enc}_N(a, 1) \leftrightarrow (a + 0, 1 \cdot b)$ where $b \leftarrow Z_N^*$

$= (a, b)$

Problem:
the receiver can only see $f(a, b)$.
How can he "extract" a from it?

Observation

$$\big(f(a,b)\big)^{\varphi(N)} \bmod N^2 \leftrightarrow (\varphi(N)\cdot a \bmod N,\ b^{\varphi(N)} \bmod N)$$
$$= (\varphi(N)\cdot a \bmod N,\ 1)$$
$$\leftrightarrow f(\varphi(N)\cdot a \bmod N,\ 1)$$
$$= (1+N)^{\varphi(N)\cdot a \bmod N}\cdot 1^n \bmod N^2$$
$$= (1+N)^{\varphi(N)\cdot a \bmod N} \bmod N^2$$
$$= \underbrace{1+(\varphi(N)\cdot a \bmod N)\cdot N}_{<N^2} \bmod N^2$$
$$= 1+(\varphi(N)\cdot a \bmod N)\cdot N$$

here we use the fact that

$$(1+N)^a = 1 + a\cdot N \ (\bmod N^2)$$

So:

$$\varphi(N)\cdot a \bmod N = \frac{\big(f(a,b)\big)^{\varphi(N)} \bmod N^2 - 1}{N}$$

Continued:

denote it z

We got that

$$\varphi(N) \cdot a \bmod N = \frac{\left(f(a,b)\right)^{\varphi(N)} \bmod N^2 - 1}{N}$$

Therefore

$$a = z \cdot \left(\varphi(N)\right)^{-1} \bmod N$$

Paillier encryption

Key generation: let $N := pq$ like in RSA

public key: N

private key: (p, q)

Encryption:

$\mathbf{Enc}_N(m) = (1+N)^m \cdot r^N \bmod N^2$ where $r \leftarrow Z_N^*$

Decryption:

$$\mathbf{Dec}_{p,q}(c) = \frac{(c^{\varphi(N)} \bmod N^2) - 1}{N} \cdot \varphi(N)^{-1} \bmod N$$

Why is this additively homomorphic?

$\boldsymbol{c = \mathbf{Enc}_N(m) \leftrightarrow (m, r)}$ where $\boldsymbol{r \leftarrow Z_N^*}$

$\boldsymbol{c' = \mathbf{Enc}_N(m') \leftrightarrow (m', r')}$ where $\boldsymbol{r' \leftarrow Z_N^*}$

We have:

$$\begin{aligned} \boldsymbol{c \cdot c'} &\leftrightarrow \boldsymbol{(m, r) \cdot (m', r')} \\ &= \boldsymbol{(m + m', r \cdot r')} \\ &\leftrightarrow \boldsymbol{\mathbf{Enc}_N(m + m')} \text{ with randomness } \boldsymbol{r \cdot r'} \end{aligned}$$

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. CCA security
9. Practical considerations
10. Theoretical overview

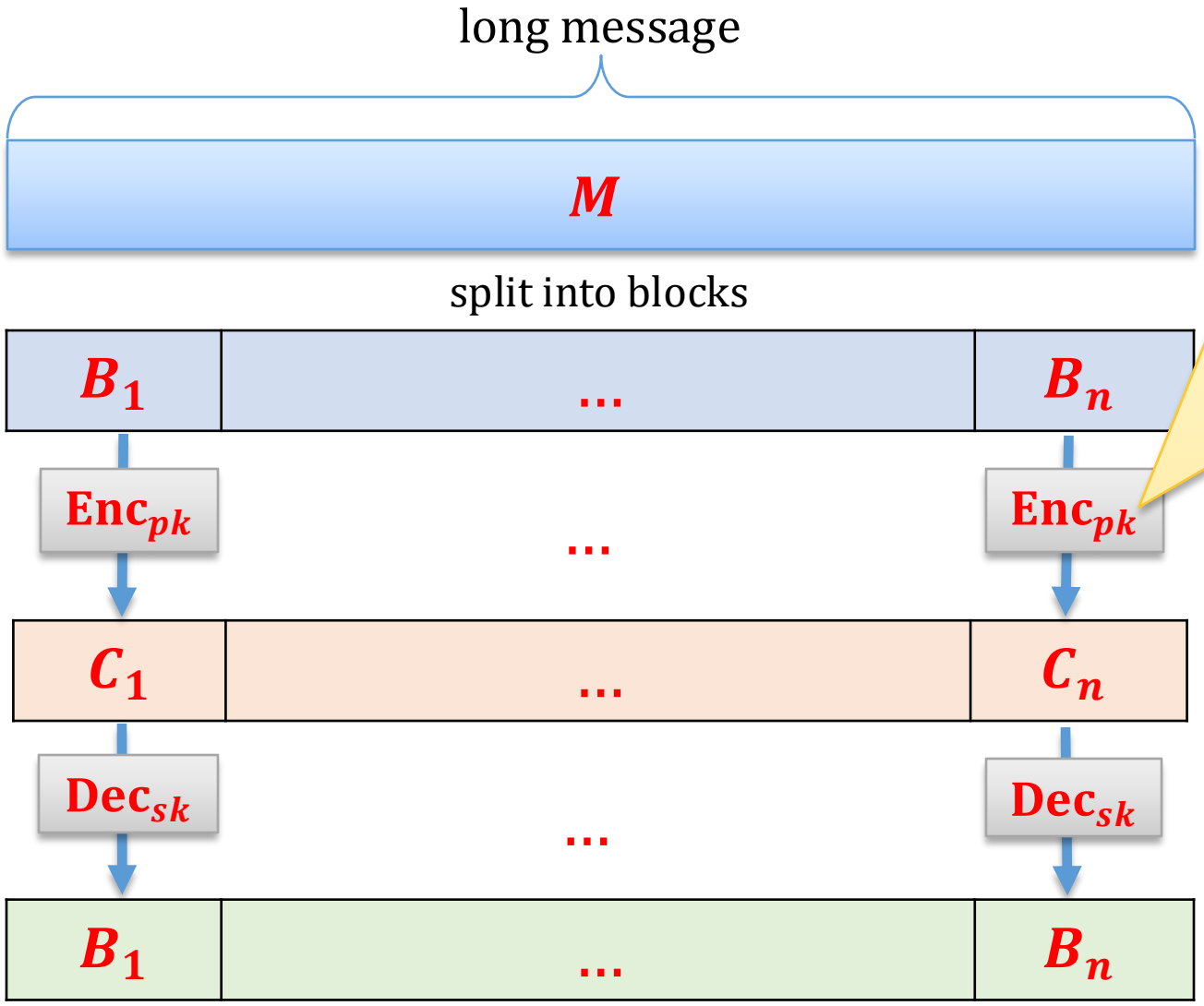
How to encrypt longer messages?

Two options:

1. divide the message in blocks and **encrypt each block separately**.

2. combine the **public-key techniques** with the **private-key encryption**.

Encrypting block-by-block



note: this is **randomized**, so we don't have the same problem as with the **ECB mode**

A problem with this solution

It's rather inefficient (the number of public-key operations is proportional to $|M|$)

A more efficient solution:

KEM/DEM paradigm

Ingrediens

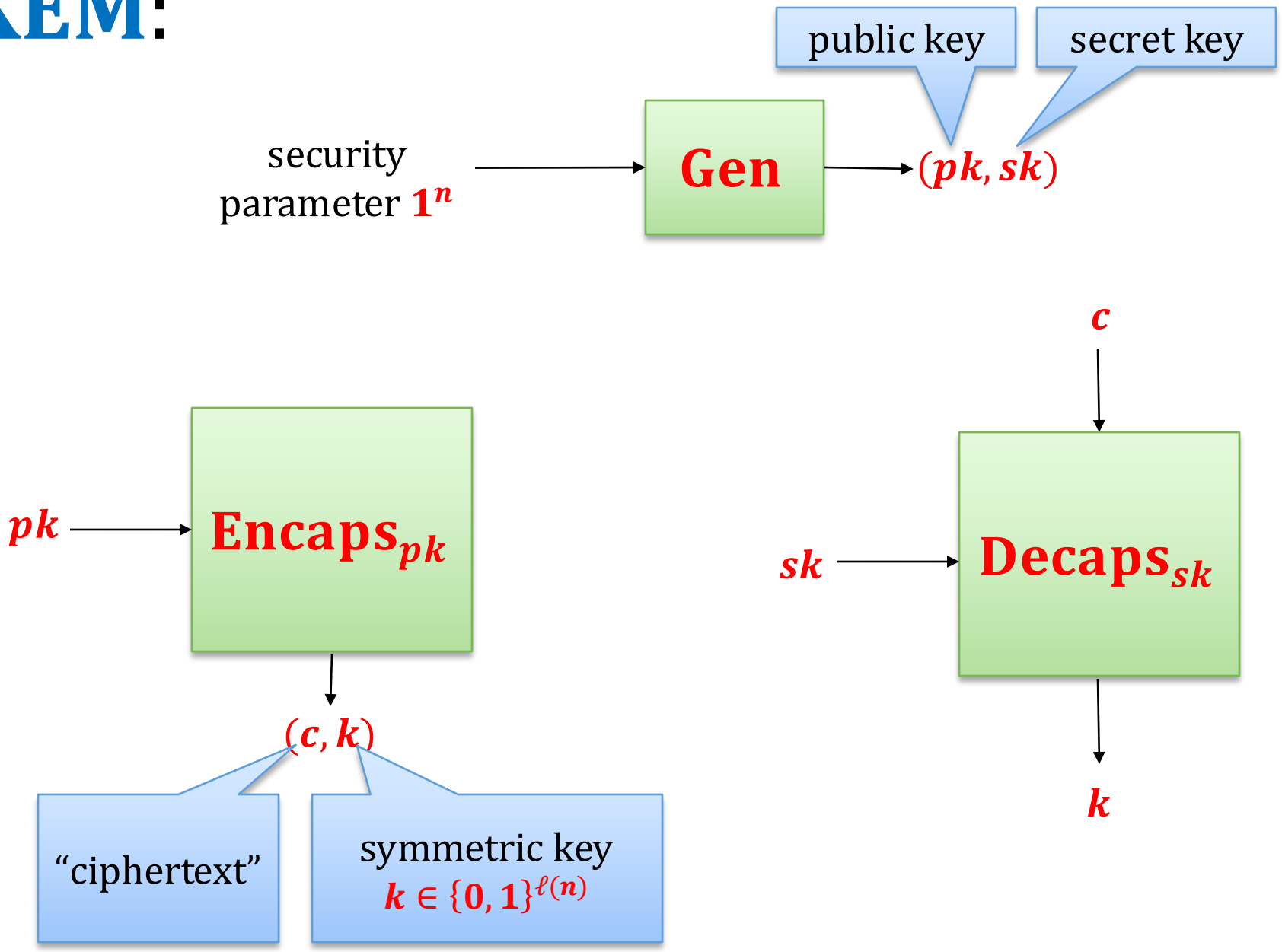
DEM – Data Encapsulation Mechanism

= private key encryption

KEM – Key Encapsulation Mechanism
consists of the following algorithms:

- **key generation algorithm Gen** – as in **PKE**,
- **encapsulation algorithm Encaps**,
- **decapsulation algorithm Decaps**.

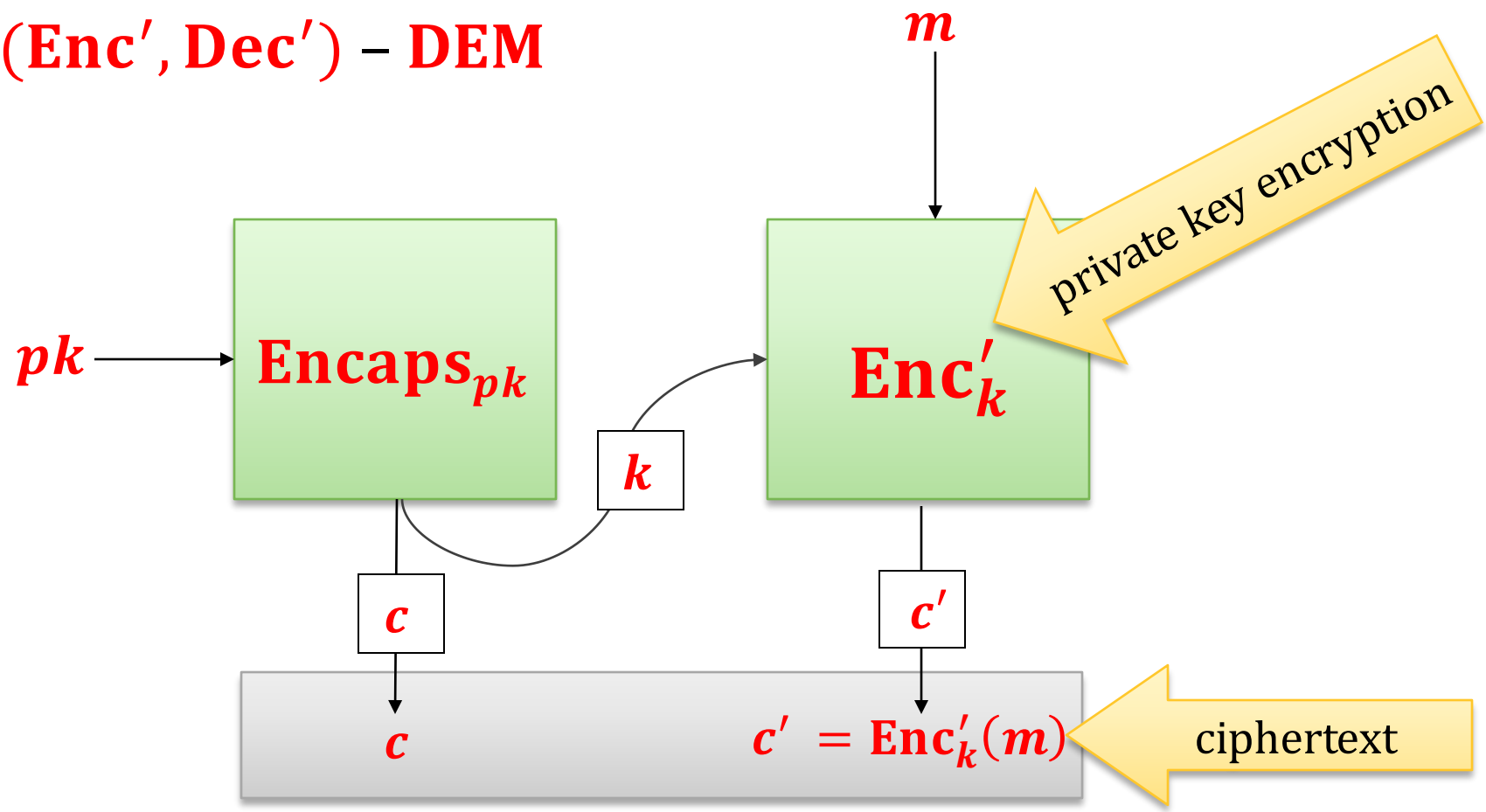
KEM:


public key
secret key
security parameter 1^n
Gen
(pk, sk)
pk
Encaps_{pk}
(c, k)
"ciphertext"
symmetric key $k \in \{0,1\}^{\ell(n)}$
c
sk
Decaps_{sk}
k

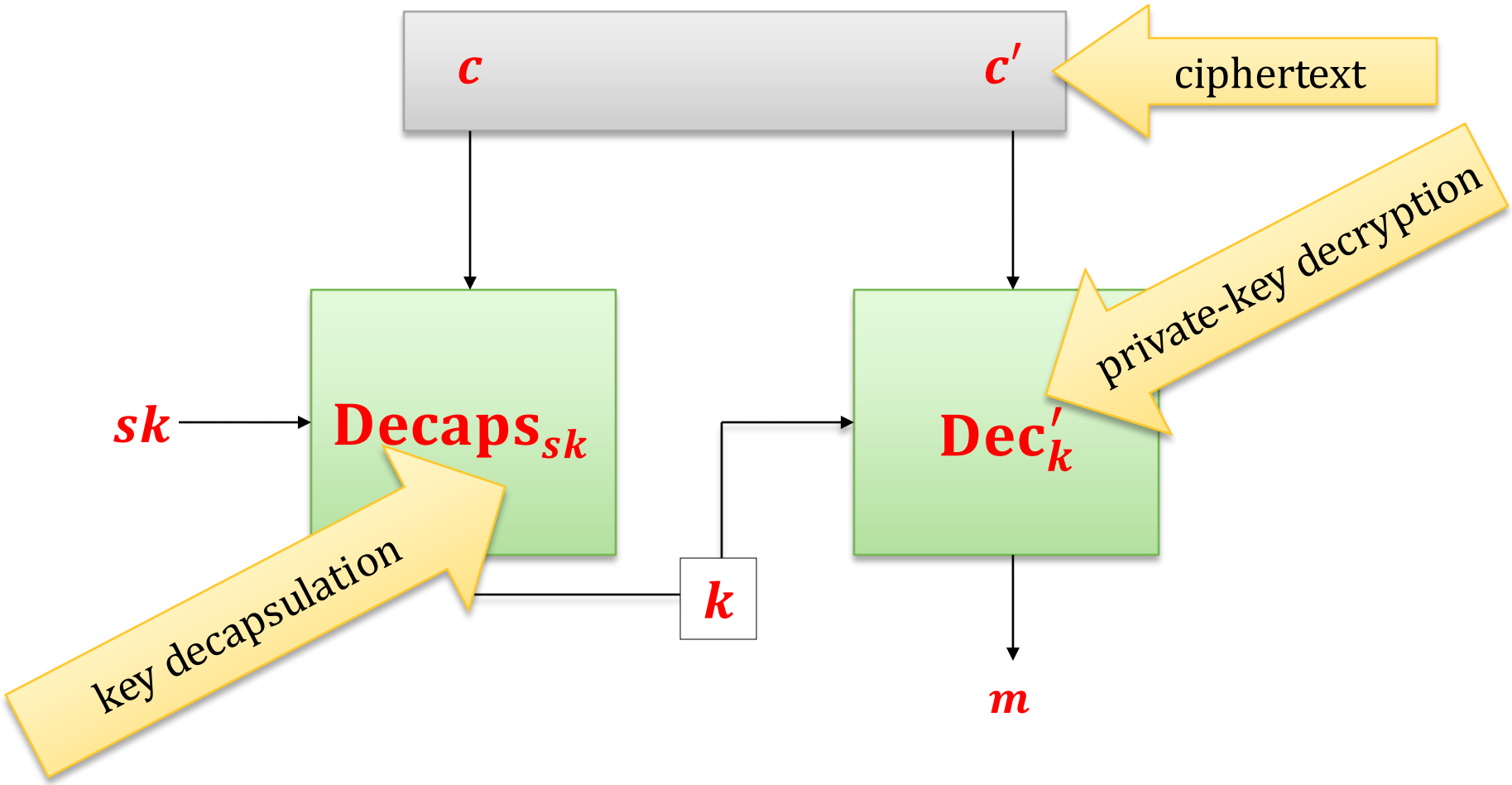
How to encrypt?

Gen – as in **KEM**

$(\mathbf{Enc}', \mathbf{Dec}')$ – **DEM**



How to decrypt?



One method to implement **KEM**

Hybrid encryption:

Take a **public-key encryption** scheme **(Gen, Enc, Dec)**.

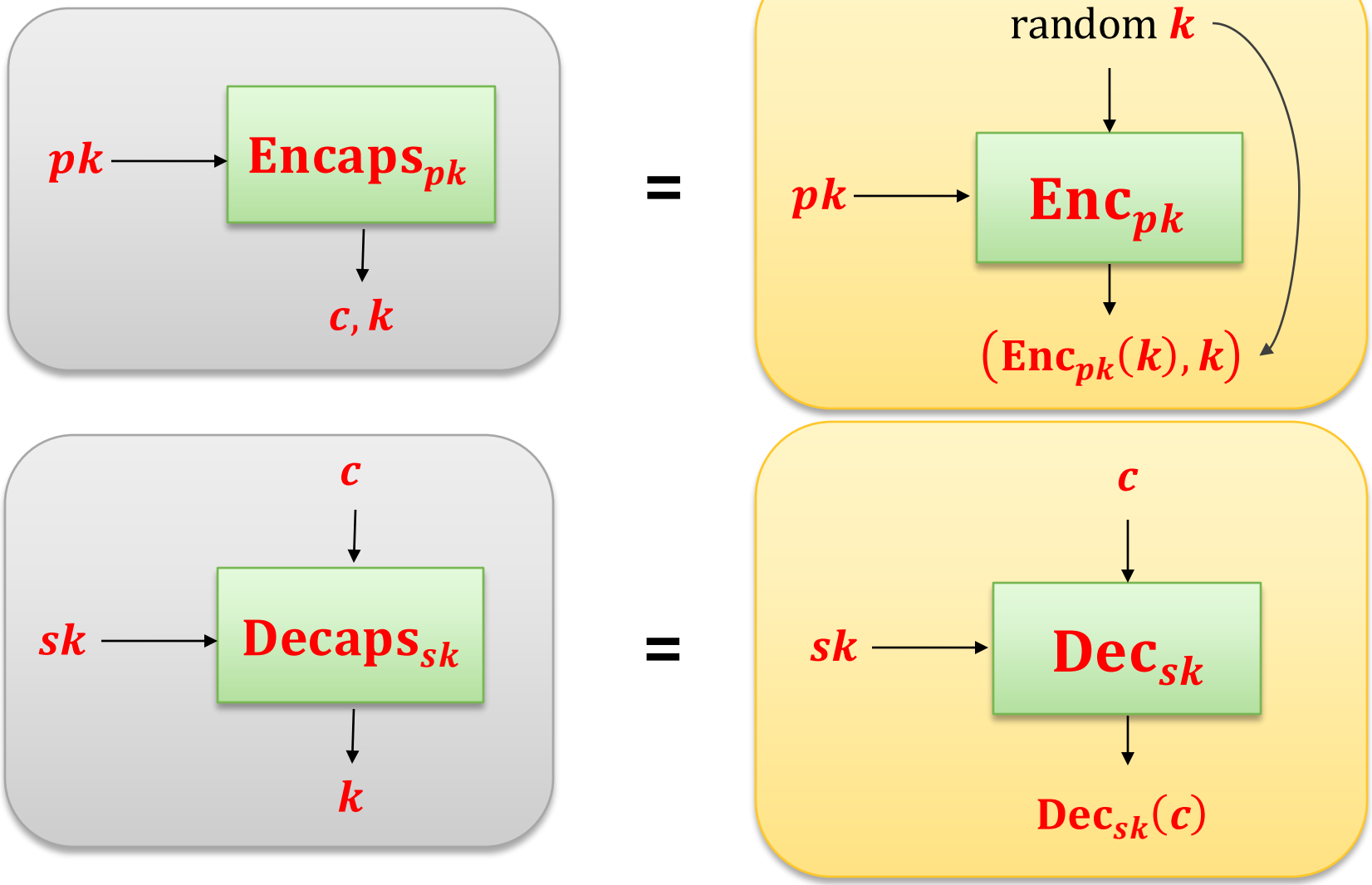
Define **KEM** as follows:

- **Gen** is the same
- $\mathbf{Encaps}_{pk}$ = generate a **random symmetric key** k and **output**

$$\left(\mathbf{Enc}_{pk}(k), k\right)$$

- $\mathbf{Decaps}_{sk}$ = on input c **output** $\mathbf{Dec}_{sk}(c)$

Pictorially:



Other methods exist.

Consequences of these approaches

For longer messages the cost of encryption is **dominated by the cost of symmetric operations**.

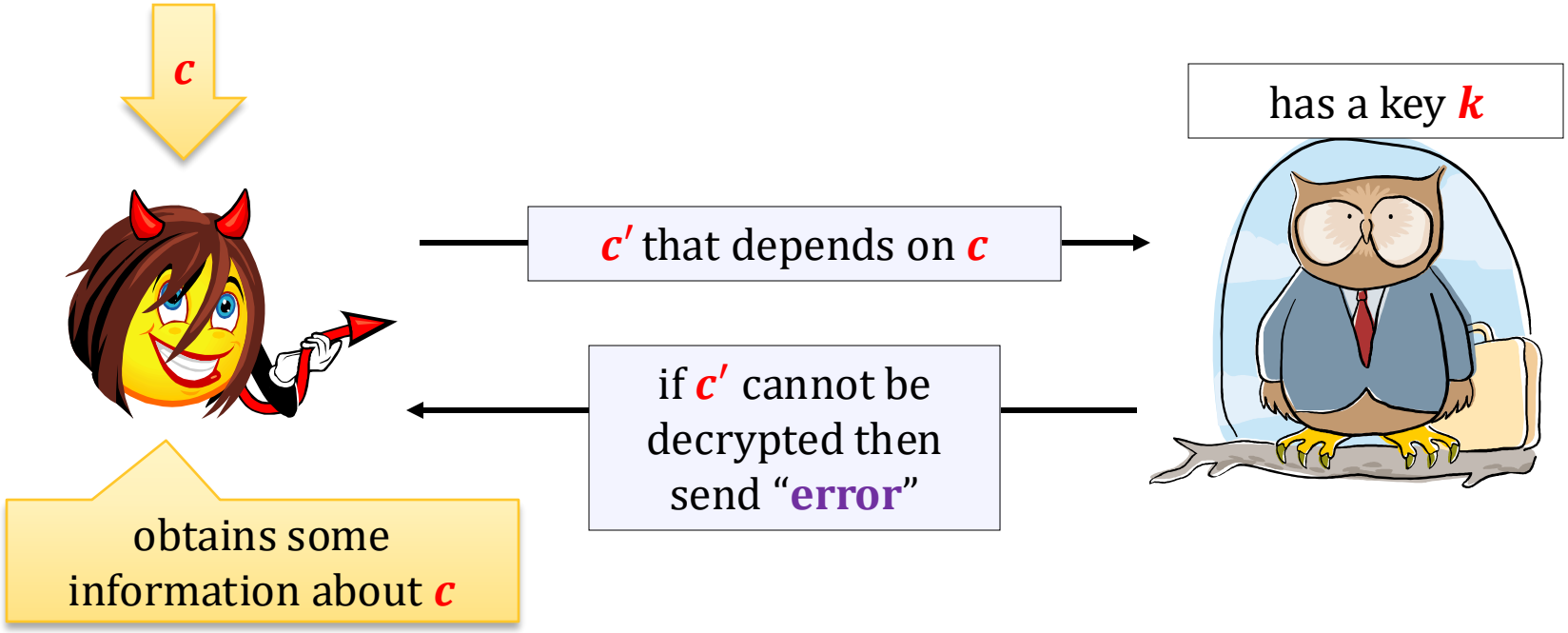
Hence: the public-key operations (amortized over the length of the messages) are almost “for free”.

Plan

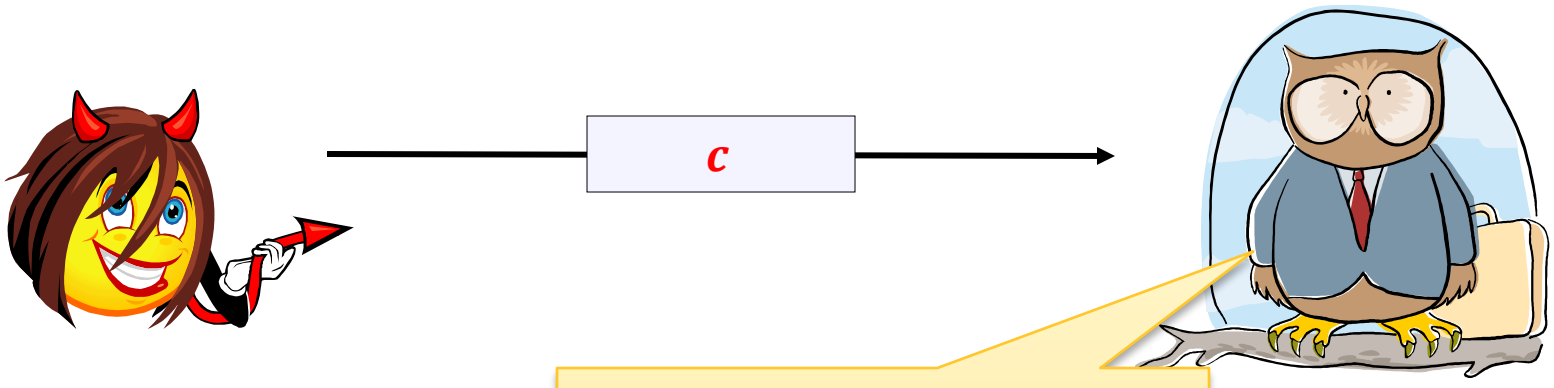
1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. CCA security
    1. Definition of the **CCA security**
    2. Constructions of **CCA-secure** symmetric encryption
    3. Constructions of **CCA-secure RSA** encryption schemes
9. Practical considerations
10. Theoretical overview

Chosen-ciphertext attacks – motivation

Consider the following scenario:

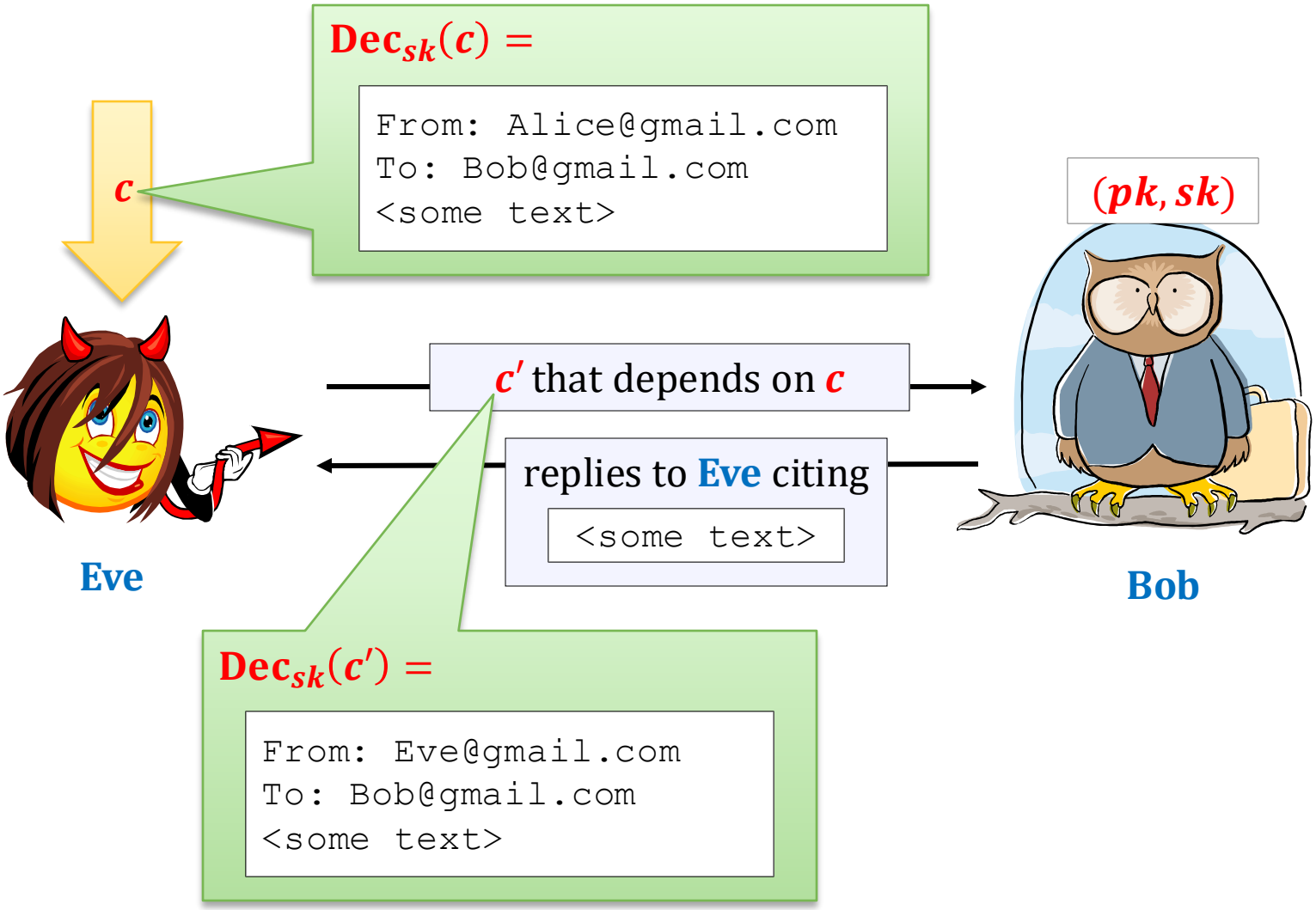


Another scenario



if $\mathbf{Dec}_{sk}(c)$ = "**alarm**"
then announce
"**alarm**" to everybody.

A more advanced example



Note

IND-CPA security does not imply that such attacks are impossible.

We need a **stronger** security definition.

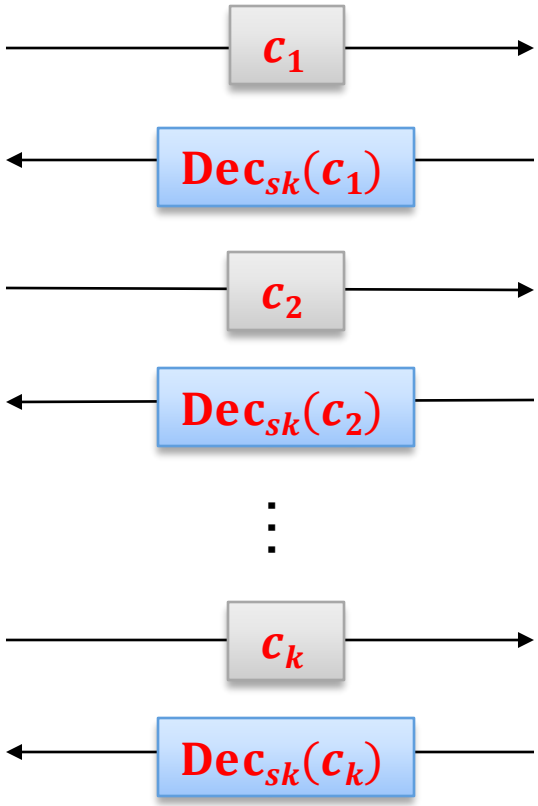
This will be called:

chosen-ciphertext security (IND-CCA)

It can be defined both for the **symmetric** and **asymmetric** case.

Decryption oracle

To define the IND-CCA security we consider a **decryption oracle**.



convention:

$\mathrm{Dec}_{sk}(c_i) := \perp$ if c_i cannot be decrypted

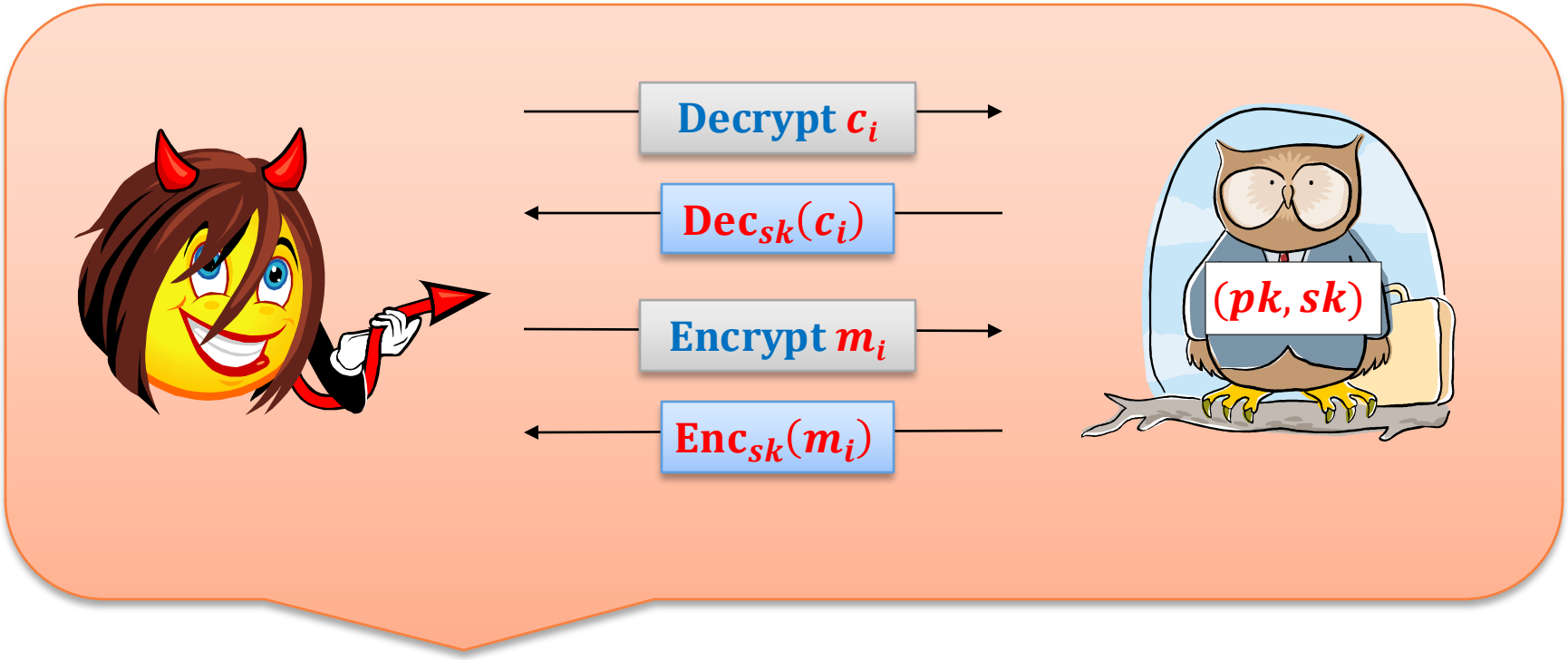
we call such a ciphertext "**invalid**"

Decryption/encryption oracle

We assume that **also** **IND-CPA** is allowed.

this will be used in the symmetric case

Two types of queries:

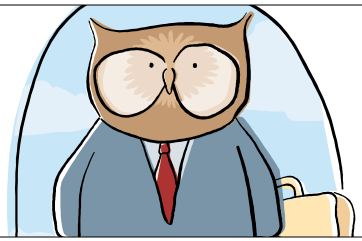


this is called a **IND-CCA-attack**

IND-CCA security– the game in the symmetric case

security parameter 1^n

1. generates $k \leftarrow \mathbf{Gen}(1^n)$
2. chooses random $b \leftarrow \{0,1\}$



IND-CCA-attack

decryption/encryption oracle

challenge phase:

chooses m_0, m_1

m_0, m_1

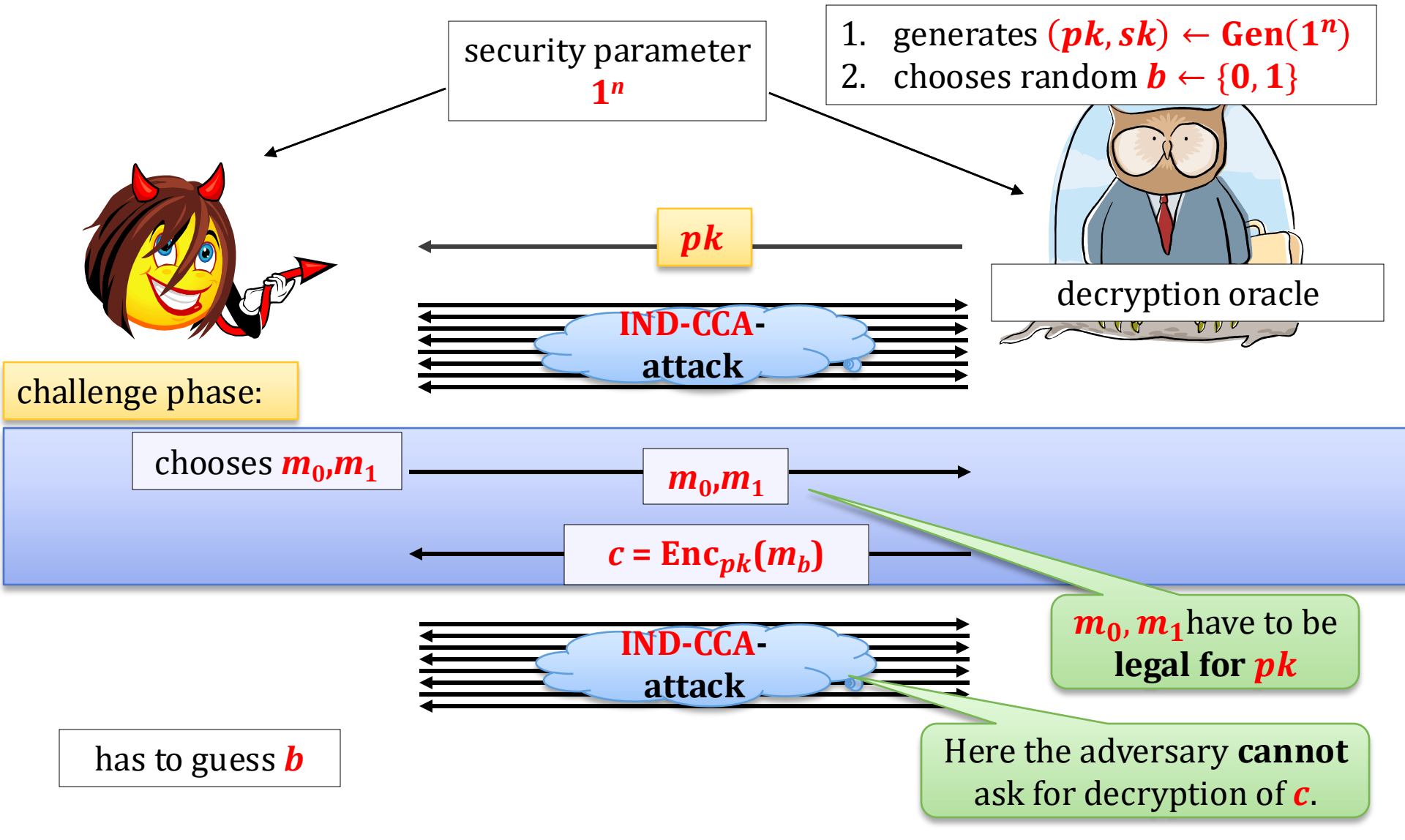
$c = \mathbf{Enc}_k(m_b)$

IND-CCA-attack

has to guess b

Here the adversary **cannot** ask for decryption of c.

IND-CCA security– the game in the **asymmetric** case



IND-CCA security

Alternative name:
IND-CCA-secure

Security definition (in the **asymmetric** case):

In the **symmetric** case: **(Enc, Dec)**

We say that **(Gen, Enc, Dec) has indistinguishable encryptions under a chosen-ciphertext attack (IND-CCA)** if any

randomized polynomial time adversary

guesses b correctly

with probability at most $1/2 + \varepsilon(n)$, where ε is negligible.

Easy to see

IND-CCA security implies **IND-CPA security**

(because the adversary in the "**IND-CCA game**" is at least as powerful as the one in the "**IND-CPA game**")

What about the implication in the other direction?

IND-CPA security does **not** imply the IND-CCA security

m_0, m_1

$c = \mathbf{Enc}(k, m_b)$

here ask about the "related" c'

Here **Eve** cannot ask for decryption of c.

Informally:

To win the game it is enough that Eve is computes some c' such that $\mathbf{Dec}_k(c')$ is "*related to*" $\mathbf{Dec}_k(c)$.
(**Why?** Because then she is allowed to ask for it.)

For example: it is possible for any stream cipher!

if $c' = c \oplus (1, \ldots, 1)$ then $\mathbf{Dec}_k(c') = \mathbf{Dec}_k(c) \oplus (1, \ldots, 1)$

How to construct IND-CCA-secure schemes?

- in the **symmetric case**: **easy**
- in the **asymmetric case**: usually **harder** (also: in this case the "**IND-CCA attacks**" are more realistic).

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. IND-CCA security
    1. Definition of the **IND-CCA security**
    2. Constructions of **IND-CCA-secure** symmetric encryption
    3. Constructions of **IND-CCA-secure** **RSA** encryption schemes
9. Practical considerations
10. Theoretical overview

Symmetric case

Simplest method: **authenticate** every ciphertext with a MAC.

Ingredients:

- **(Enc, Dec)** – a **IND-CPA-secure symmetric encryption** scheme
- **(Tag, Vrfy)** – a **message authentication code** that is **strongly secu**

A MAC is **strongly secure** if the adversary cannot produce a valid tag t' on a message m even if he saw a valid pair (m, t) (where $t' \neq t$)

The method from previous lectures

encrypt-then-authenticate:

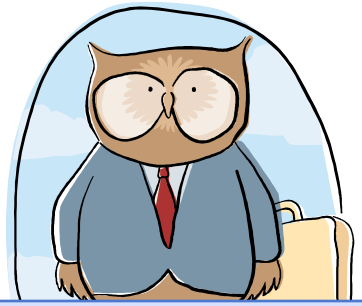
key: a pair (k_1, k_2)

- to **encrypt** m compute $c := \mathbf{Enc}_{k_1}(m)$ and $t := \mathbf{Tag}_{k_2}(c)$, and output (c, t)
- to **decrypt** (c, t):

  if $\mathbf{Vrfy}_{k_2}(c, t) = \mathbf{no}$ then output $\perp$

  otherwise output $\mathbf{Dec}_{k_1}(c)$

Why is this secure?



chooses m_0, m_1

m_0, m_1

$c = \mathrm{Enc}_k(m_b)$

The adversary **cannot "produce himself" a valid ciphertext.**

The only decryption queries **Decrypt** c' on which he doesn't get $\perp$ are such that he received c' from the oracle before.

But he already knows the decryptions of such c's.

So: the **IND-CCA** attack does not help him!

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. IND-CCA security
    1. Definition of the **IND-CCA security**
    2. Constructions of **IND-CCA-secure** symmetric encryption
    3. Constructions of **IND-CCA-secure RSA** encryption schemes
9. Practical considerations
10. Theoretical overview

PKCS #1 v.2 is not IND-CCA-secre

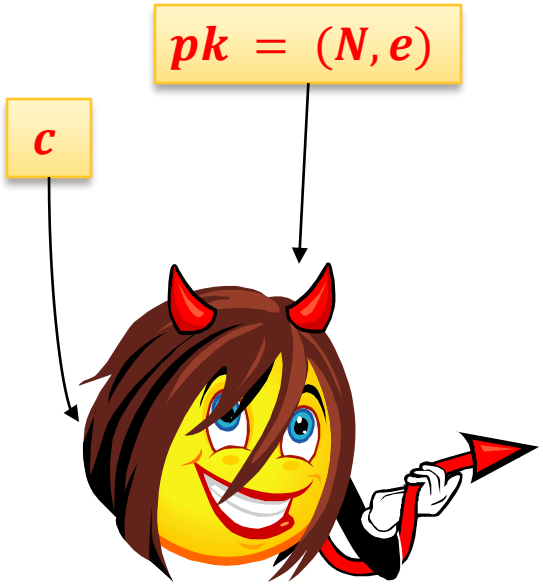
Bleichenbacher [1998] showed a “practical” chosen ciphertext attack on encoding proposed for the PKCS #1 v.2 standard.

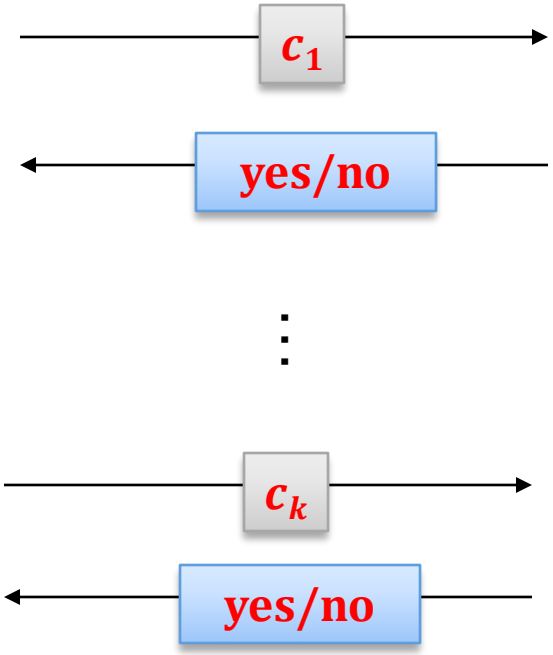
[see also: Bleichenbacher, D., Kaliski B., Staddon J., "Recent results on PKCS #1: RSA encryption standard", *RSA Laboratories' bulletin #7,* ftp://ftp.rsasecurity.com/pub/pdfs/bulletn7.pdf]

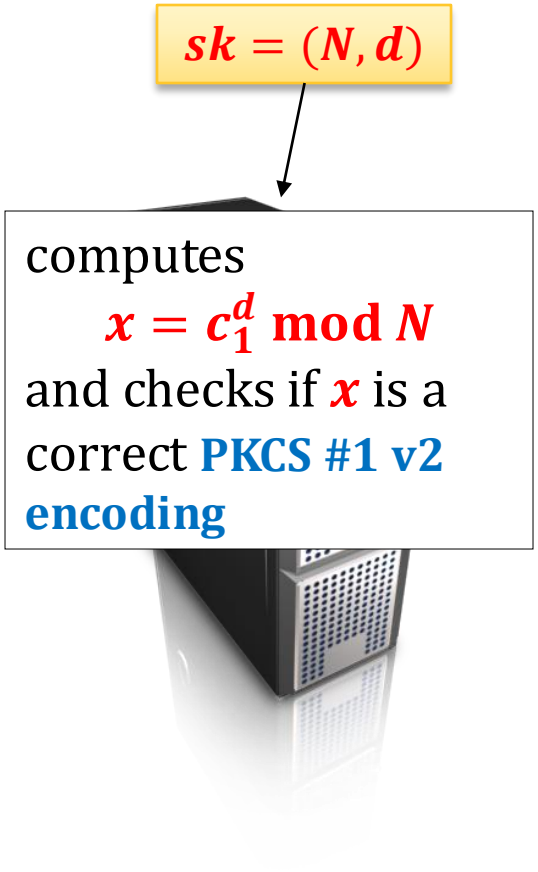
Why is Blaichenbacher’s attack practical?

Because it assumes that the adversary can get only one bit of information about the plaintext...

Bleichenbacher's attack – the scenario







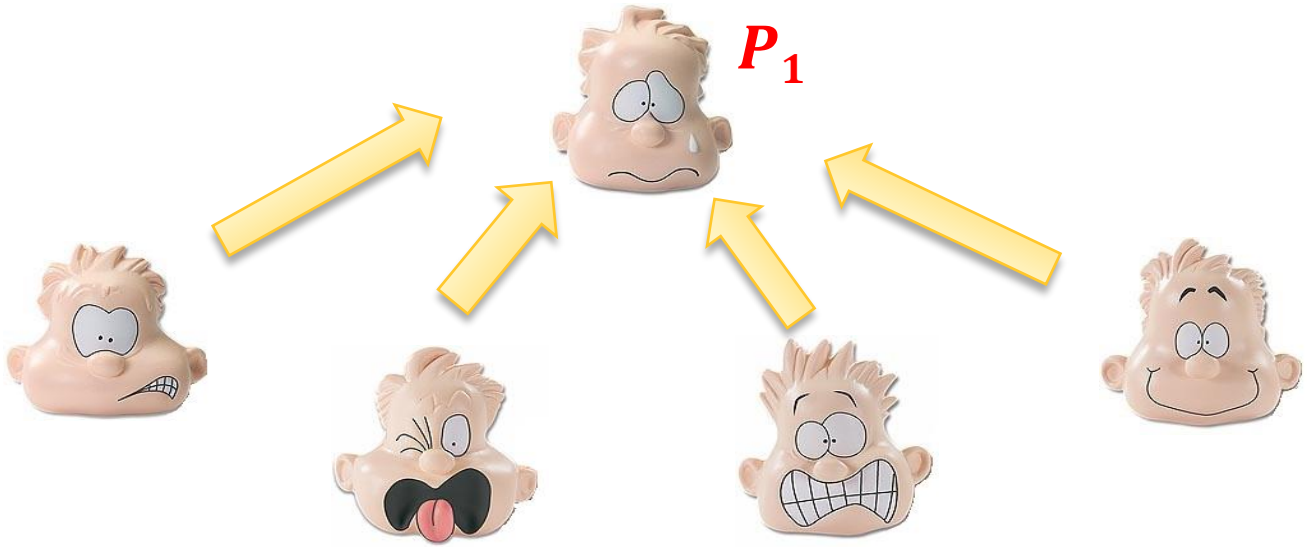
Goal:
compute $c^d \bmod N$

Bleichenbacher [1998]:
There exists a successful attack that requires $k = 2^{20}$ questions for $|N| = 1024$.

How to construct IND-CCA-secure encryption scheme from RSA?

Observation: MACs don't help (at least directly).

Because in the asymmetric case the parties don't share a key for a MAC.



First attempt

Idea: take a **symmetric-key IND-CCA-secure** scheme **(Enc', Dec')** and use it in the **KEM/DEM** method.

r **is random from** Z_N^*

public key: (N, e) **private key:** (N, d)

$$\mathbf{Enc}((N, e), m) := (r^e \bmod N, \mathbf{Enc}'(r, m))$$

$$\mathbf{Dec}((N, d), (c_0, c_1)) := \mathbf{Dec}'(c_0^d \bmod N, c_1)$$

Problem

$$\mathbf{Enc}((N, e), m) := (r^e \bmod N, \mathbf{Enc}'(r, m))$$

$|N|$ is normally much larger than the length of a key for symmetric encryption.

Typically $|N| = 1024$ and length of the symmetric key is 128.

First idea: **truncate**.

But is it secure?

It may be the case that

- **RSA** is hard to invert, but
- **128** first bits are easy to compute...

Idea: instead of truncating – hash!

t – length of the symmetric key
$H: \{0,1\}^* \to \{0,1\}^t$ – a hash function

$$\mathbf{Enc}((N,e),m) := (r^e \bmod N, \mathbf{Enc}'(H(r),m))$$

$$\mathbf{Dec}((N,d),(c_0,c_1)) := \mathbf{Dec}'\left(H\left(c_0^d \bmod N\right), c_1\right)$$

But can we prove anything about it?

depends...

Which properties should *H* have?

If we just assume that *H* is collision-resistant we cannot prove anything...

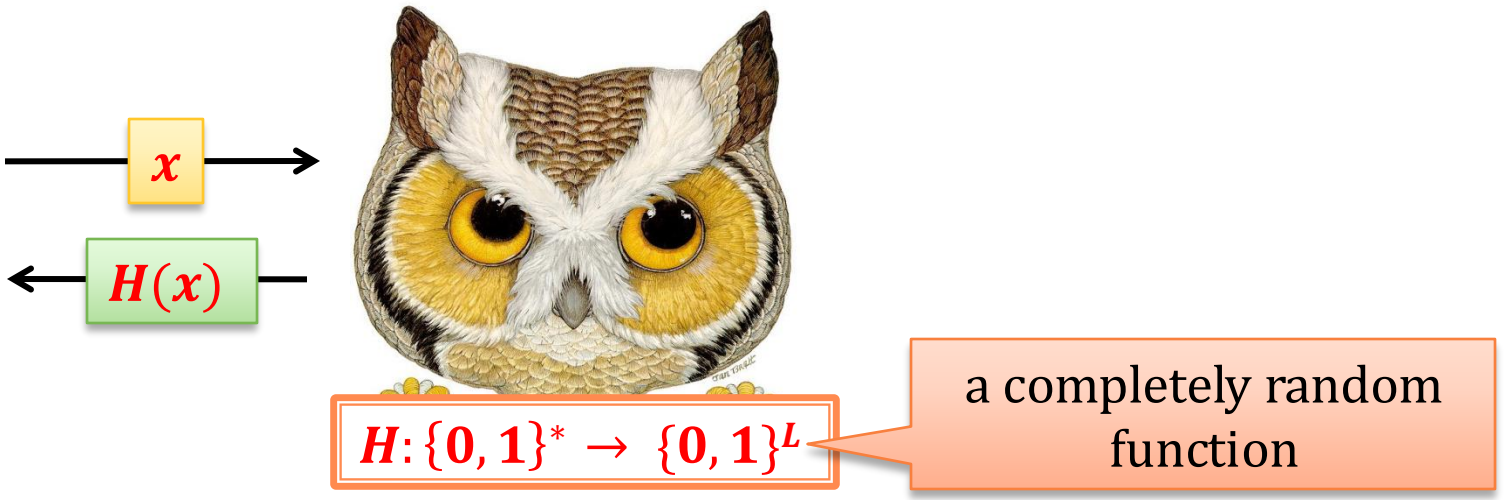
We have to assume that *H* "outputs **random values** on different inputs".

This can be formalized by modeling *H* as **random oracle.**

Remember the **Random Oracle Model**?

Random oracle model

hash functions ≈ **random oracles**



Security proof – the intuition

H – a hash function $\mathbf{Enc}((N,e),m) := (r^e \bmod N, \mathbf{Enc'}(H(r),m))$

Why is this scheme secure in the **random oracle model**?

Because, as long as the adversary did not query the oracle on r, the value of $H(r)$ is completely random.

To learn r the adversary would need to compute it from $r^e \bmod N$, so he would need to invert **RSA**.

So (with a very high probability) from the point of view of the adversary $H(r)$ is random.

Therefore, the **IND-CCA security** of **(Enc, Dec)** follows from the **IND-CCA security** of **(Enc', Dec')**.

A drawback of this method

$$\mathbf{Enc}((N, e), m) := (r^e \bmod N, \mathbf{Enc}'(H(r), m))$$

The ciphertext is longer than N even if the message is short.

Therefore in practice another method is used:

Optimal Asymmetric Encryption Padding (OAEP).

Optimal Asymmetric Encryption Padding (OAEP) – the history

- **Introduced in**:
  [M. Bellare, P. Rogaway. *Optimal Asymmetric Encryption -- How to encrypt with RSA.* Eurocrypt '94]

- **An error in the security proof was spoted in**
  [V. Shup. *OAEP Reconsidered.* Crypto '01]

- **This error was repaired in**
  [E. Fujisaki, T. Okamoto, D. Pointcheval, and J. Stern. *RSA-OAEP is secure under the RSA assumption*. Crypto '01]

It is now a part of a **PKCS#1 v. 2.0** standard.

OAEP

N – RSA modulus

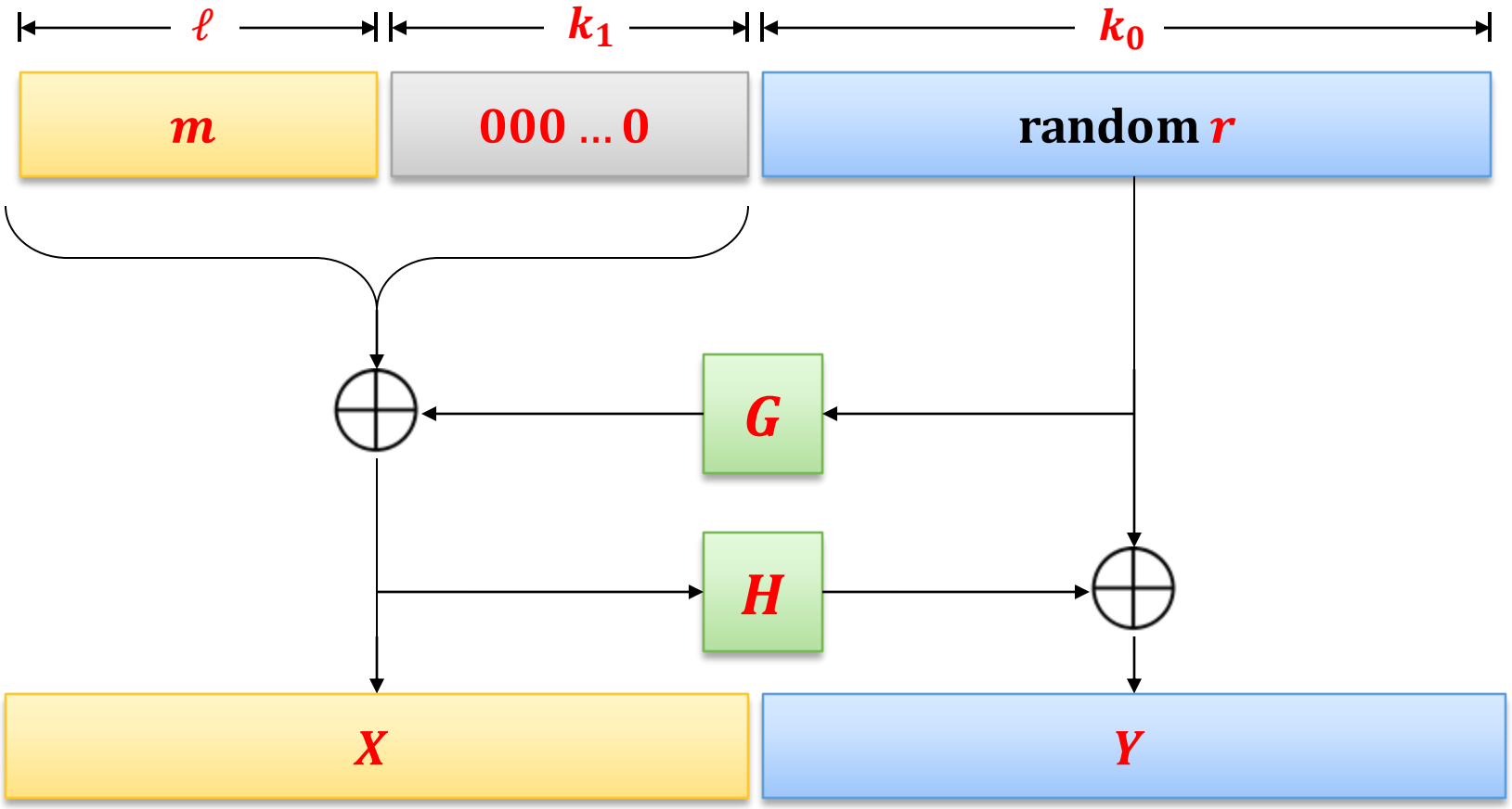
ℓ, k_1, k_2 – parameters such that

$\ell + k_1 + k_2 \leq \lfloor \log_2 N \rfloor$

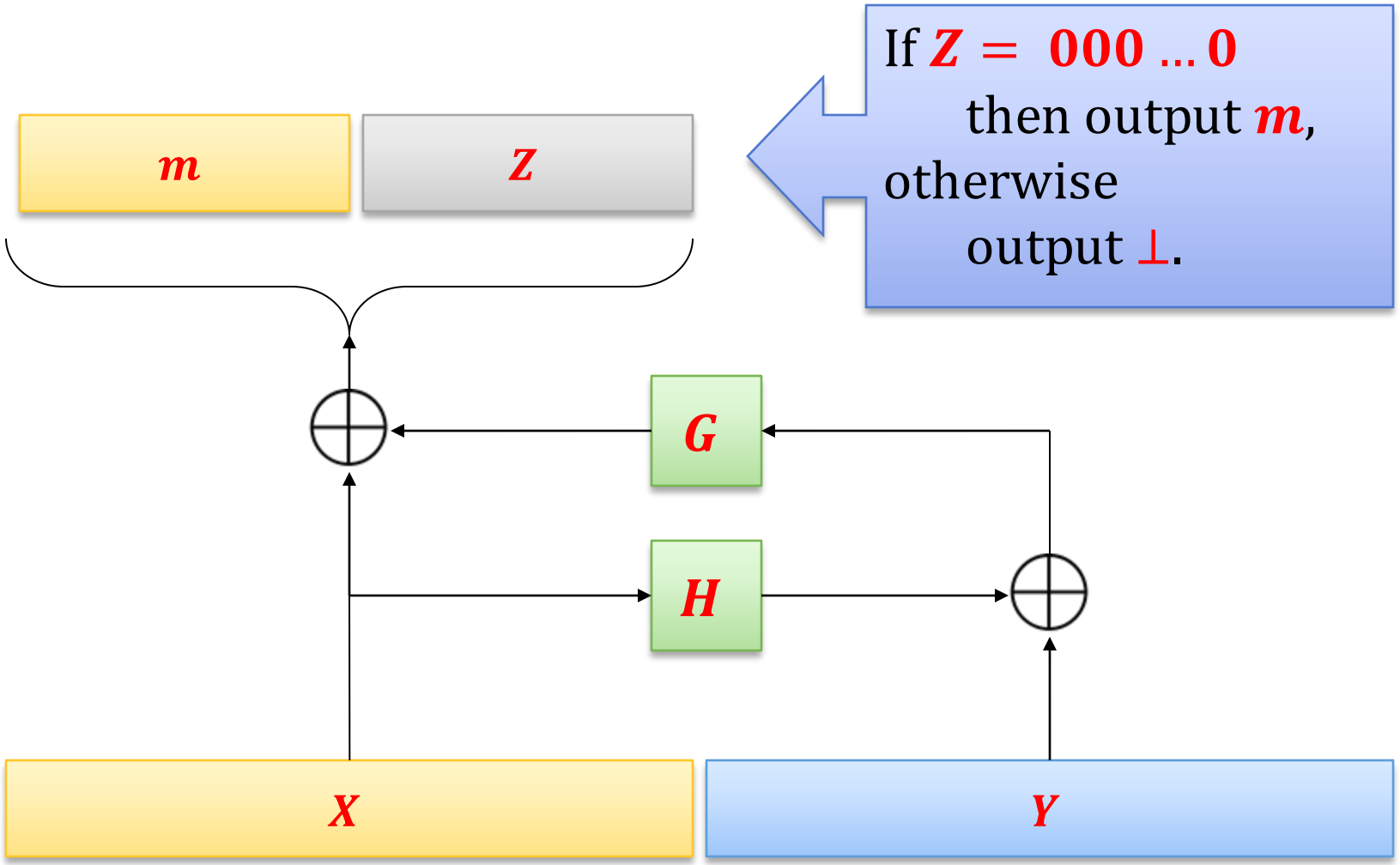
hash functions:

- $G: \{0,1\}^{k_0} \to \{0,1\}^{\ell + k_1}$,
- $H: \{0,1\}^{\ell + k_1} \to \{0,1\}^{k_0}$

$\mathbf{OAEP}(m) :=$



How to invert?



RSA-OAEP

key pair like in the **handbook RSA**:

private key: (N, d)

public key: (N, e)

$$\mathbf{Enc}\big((N,e),m\big) := (\mathbf{OAEP}(m))^e \bmod N$$

$$\mathbf{Dec}\big((N,e),c\big) := \text{let } x := c^d \bmod N$$

if $x > 2^{\ell+k_1+k_2}$ then output $\perp$

otherwise output $\mathbf{OAEP}^{-1}(x)$

Security of RSA-OAEP

Security of **RSA-OAEP** can be proven

- if one models ***H*** and ***G*** as random oracles
- assuming the **RSA assumption** holds.

We do not present the proof here.

We just mention some **nice properties** of this encoding.

Nice properties of OAEP
(for the right choice of parameters)

good for the **IND-CPA security**

- it is **invertible**
- but **to invert you need to know (X, Y) completely**
- for every message m the encoding **$\mathrm{OAEP}(m)$** is uniformly **random**

good for the **IND-CCA security**

- It is hard to produce a valid (X, Y) "without knowing m first"

OAEP is hard to invert if you don't know ***X*** and ***Y*** completely.

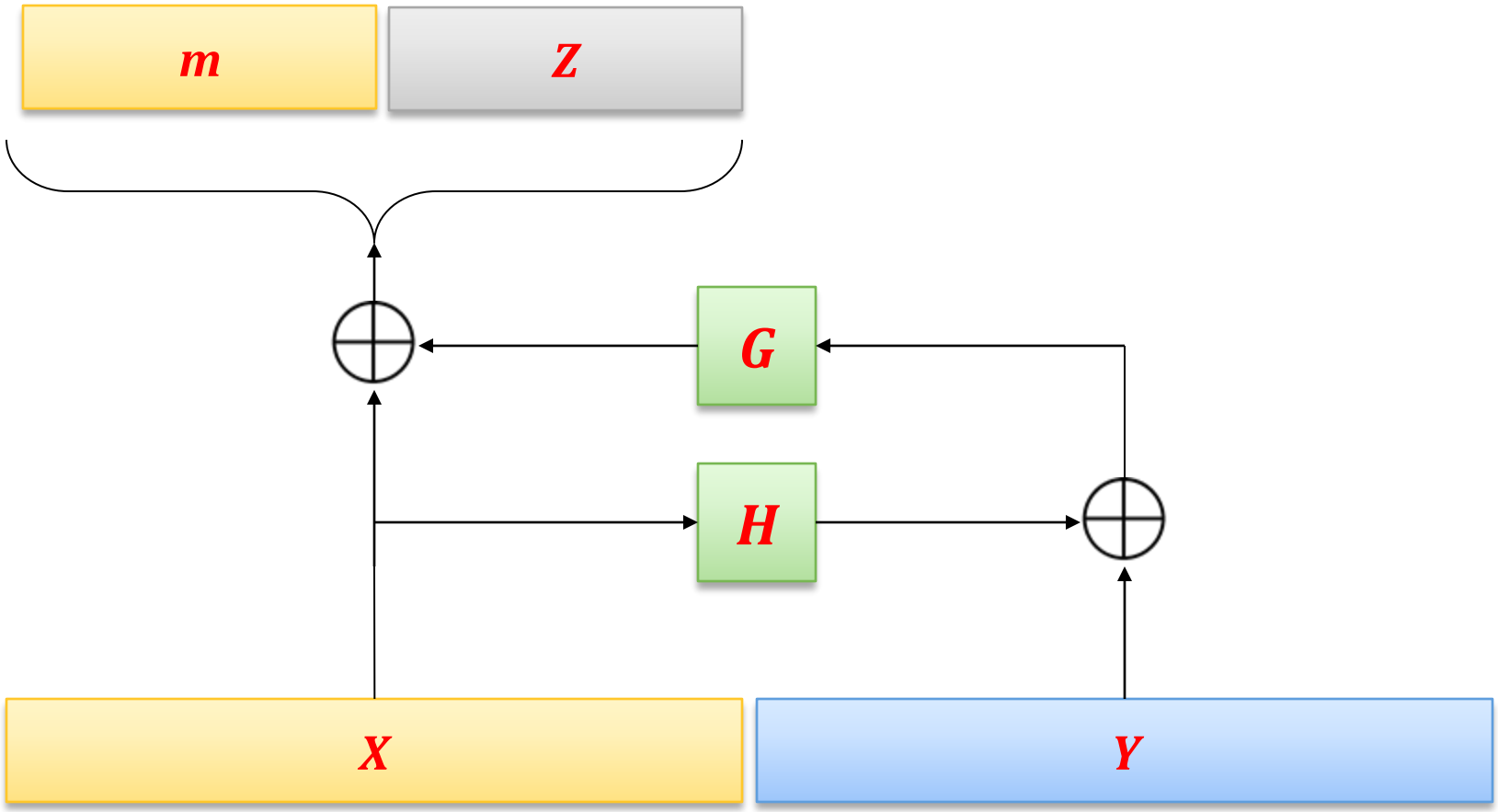
Actually:

m is completely hidden in such a case.

(assuming ***G*** and ***H*** are random oracles)

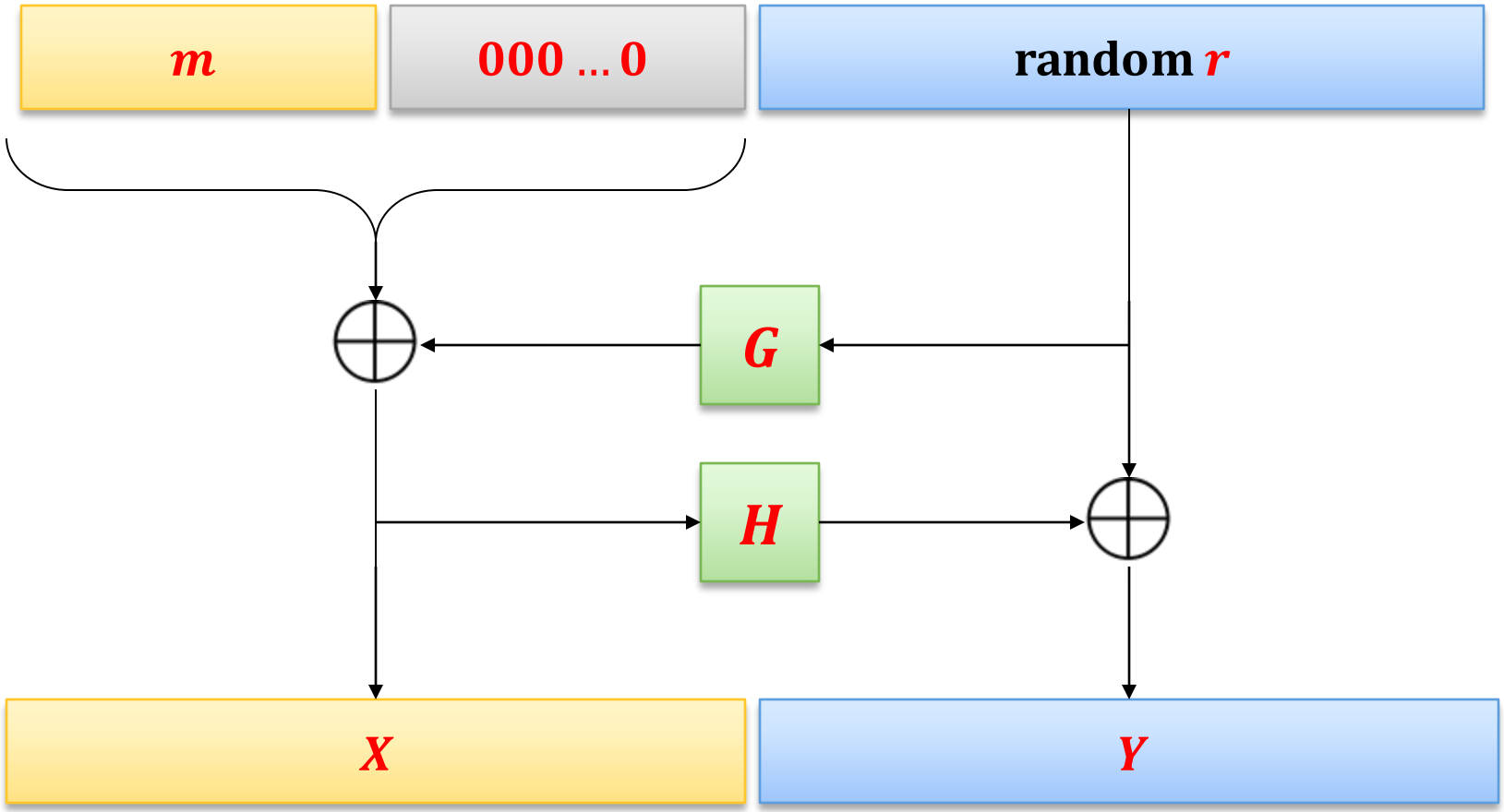
Why?

Look at the picture:



The encoding **OAEP(m)** is uniformly **random**

Again look at the picture:



Why are these two properties useful for **IND-CPA security**?

The adversary obtains

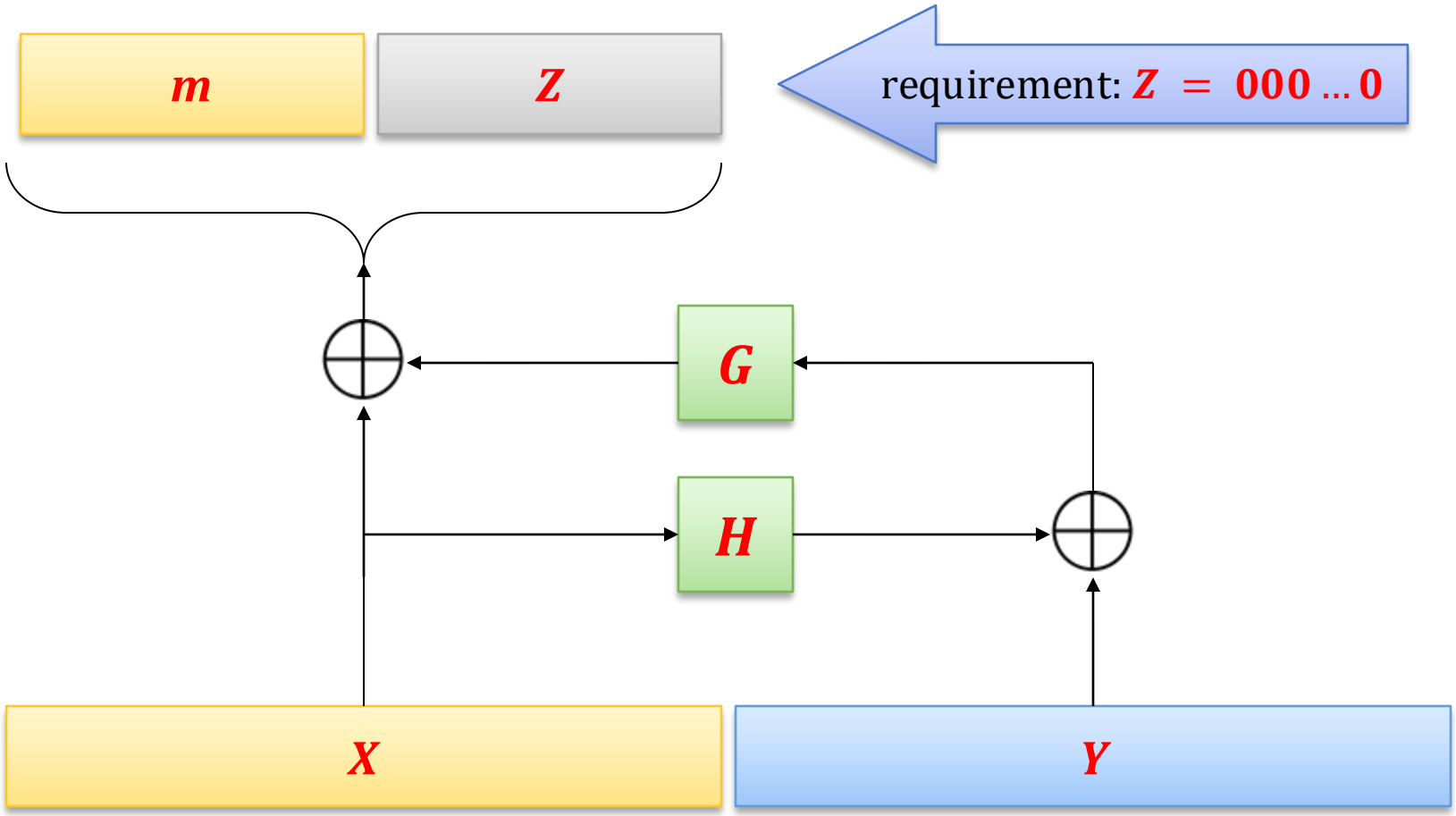
$$\mathbf{Enc}_{pk}(m_b) = x^e \bmod N$$

where $x = \mathbf{OAEP}(m_b)$.

In order to get **any information about m_b** needs to compute the **entire value of x**, where x is uniformly random.

Hardness of this problem is equivalent to the **RSA assumption.**

It is hard to produce a valid (X, Y) "without knowing m first"



This last property is useful for IND-CCA security

Why?

Informally:

Eve can produce valid ciphertexts only of those messages that she knows...

The only way to produce a valid ciphertext is to do the following:

- choose $\boldsymbol{m}$
- compute $\boldsymbol{c} := \left(\mathbf{OAEP}(\boldsymbol{m})\right)^{\boldsymbol{e}} \mathbf{mod}\ \boldsymbol{N}$.

Note

In “handbook RSA” this is not the case since every $c \in Z_N^*$ is a valid ciphertext.

Also in the **PKCS #1: RSA Encryption Standard Version 1.5** standard the probability of producing a valid ciphertext is noticeable.

An interesting attack on OAEP

J. Manger: **A Chosen Ciphertext Attack on RSA Optimal Asymmetric Encryption Padding (OAEP) as Standardized in PKCS #1 v2.0.** CRYPTO 2001

Based on the following fact:

the decryption algorithm outputs ⊥ in two cases:

1. "$x > 2^{\ell+k_1+k_2}$",
2. or $Z \neq 000 \ldots 0$.

The attack exploits the fact that in the **PKCS #1 v2.0** standard the **error messages in these two cases were different** and (1) was checked before (2).

Moral: implementation details matter!

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. IND-CCA security
9. Practical considerations
10. Theoretical overview

ElGamal vs. RSA

In practice **RSA** and **ElGamal** (in $\boldsymbol{Z_p^*}$) have similar security for equivalent key lengths.

- **RSA** is slightly more efficient
- **ElGamal** has a ciphertext twice as long as the plaintext
- But **ElGamal** can be generalized to other groups (e.g., the **elliptic curves**) where it is much more efficient!

NIST recommendations

bits of security	RSA modulus length	discrete log in order q subgroups of Z_p^*	discrete log in elliptic curves of order:
≤ 80	1024	$\|p\| = 1024$ $\|q\| = 160$	160
112	2048	$\|p\| = 2048$ $\|q\| = 224$	224
128	3072	$\|p\| = 3072$ $\|q\| = 256$	256
192	7680	$\|p\| = 7680$ $\|q\| = 384$	384
256	15360	$\|p\| = 15360$ $\|q\| = 512$	512

[NIST Special Publication 800-57 Part 1 Revision 4 Recommendation for Key Management]

Quantum attacks

All the schemes presented so far can be broken by quantum computers using Shor's algorithm.

[Peter W. Shor "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer" 1995]

Peter Shor
1959—

There exists public-key encryption schemes that are believed to be secure against quantum computers (see **post-quantum cryptography**)

Plan

1. Problems with the “handbook RSA”
2. Definition of the IND-CPA security
3. RSA encryption
4. Rabin encryption
5. ElGamal encryption
6. Homomorphic encryption and Paillier cryptosystem
7. The KEM/DEM paradigm
8. IND-CCA security
    1. Definition of the **IND-CCA security**
    2. Constructions of **IND-CCA-secure** symmetric encryption
    3. Constructions of **IND-CCA-secure RSA** encryption schemes
9. Practical considerations
10. Theoretical overview

A natural question

Is public-key encryption a member of **Minicrypt**?

Answer: **NO** (as far as we know).

More precisely: nobody knows how to construct **PKE** from **one-way functions**.

However, the following implication is known:

This is proven using the **hardcore predicates**.

Hard-core predicates

Hard-core **predicates** are a generalization of hard-core **bits**.

Definition (informal)

$\boldsymbol{\pi}: \{\mathbf{0},\mathbf{1}\}^{\boldsymbol{n}} \to \{\mathbf{0},\mathbf{1}\}$ is a hard core predicate for a trap-door permutation $\boldsymbol{f}: \{\mathbf{0},\mathbf{1}\}^{\boldsymbol{n}} \to \{\mathbf{0},\mathbf{1}\}^{\boldsymbol{n}}$ if it is hard to guess $\boldsymbol{\pi}(\boldsymbol{f}^{-\mathbf{1}}(\boldsymbol{y}))$ from $\boldsymbol{y}$ (with probability significantly better than $\mathbf{1/2}$).

A fact

Does every trap-door permutation have a hard-core predicate?

Almost:

Suppose that f is a trap-door permutation.

It can be used to build a trap-door permutation g that has a hard-core predicate.

How to encrypt with such an g?

Encryption for messages of length **1**:

public key: description of g

private key: trapdoor t for g

$$\mathbf{Enc}_g(b) = (\pi(x) \oplus b, g(x))$$

where $x \in Z_N^*$ is random.

$$\mathbf{Dec}_t(b', y) = \pi\left(g^{-1}(y)\right) \oplus b'$$

The general picture